Web Technologies and Applications

Prof. Dr. Jan Stehr

WS 2025/26

Dr. rer. nat. Jan Stehr

1971 in Kassel geboren, lebt seit 1994 in Paderborn.
Broterwerb in der Informatik seit gut 30 Jahren.

Ausbildung zum Mathematisch-technischen Assistenten,
Studium Wirtschaftsinformatik,
Promotion in Informatik an der Uni Paderborn.

Professor für Informatik, AI und Cyber Security an der FHDW,
CISO, Systemarchitekt, Security Architect, Leitung Softwareentwicklung
in verschiedenen Unternehmen

Verwendungshinweise

Bitte beachten Sie:

- Dieses Dokument ist Grundlage der Vorlesung, *ersetzt sie aber nicht*.
- Dieses Dokument und die im Rahmen der Veranstaltung ausgegebenen weiteren Dokumente sind nur für den internen Gebrauch an der FHDW durch Studierende und Mitarbeiter der Hochschule bestimmt.
- Sie dürfen Dritten, auch auszugsweise, nur mit vorheriger schriftlicher Zustimmung der FHDW zugänglich gemacht werden.
- Dies gilt für die Weitergabe in jedweder Form, ob elektronisch, z. B. Internet, oder Papier.

Organisatorisches

- 40 Kontaktstunden
- 85 Stunden Selbststudium
- 5 Credit Points
- Prüfungsleistung als Klausur, Dauer 90 Minuten.
- Das Skript + Lösungen zu Übungen bekommen Sie in elektronischer Form (Ordner in Teams) immer nach unseren Veranstaltungsterminen.
- Ihren Dozenten erreichen Sie unter jan.stehr@fhdw.de

- Warum findet die Veranstaltung virtuell statt?
- Unterbrechen Sie bitte jederzeit bei Fragen, Unklarheiten, wenn Sie etwas vermissen oder ergänzen möchten.
- Im E-Learning dürfen Sie die Kameras ausgeschaltet lassen.
- Melden Sie sich mit Fragen auch gerne außerhalb der Veranstaltung.

Literatur, Quellen, Systeme

Kurose, Ross: Computer Networking: A Top-Down Approach, Global Edition. Pearson, immer aktuelle Ausgabe.

Gough et al.: Mastering API Architecture. O'Reilly, aktuelle Ausgabe.

Mezzalira: Building Micro-Frontends. O'Reilly, aktuelle Ausgabe.

Burns et al.: Kubernetes Up & Running. O'Reilly, aktuelle Ausgabe.

Zu HTTP: `https://developer.mozilla.org/en-US/docs/Web/HTTP`

Zu HTML, CSS etc.: `w3schools.com`

Software zum praktischen Ausprobieren

Vagrant unter `vagrantup.com`

VirtualBox unter `virtualbox.org`

Linux Mint unter `linuxmint.com`

Docker unter `docker.com`

Postman unter `postman.com`

Minikube unter `minikube.sigs.k8s.io/`

Bitte immer die Hinweise der Produkte zu Ihrem Betriebssystem beachten.

Die Übungen werden irgendwann etwas komplexer und setzen Gruppenarbeit und/oder den Einsatz von AI-Hilfsmitteln voraus.

Falls Sie damit noch nicht, oder noch nicht in technischen Aufgaben gearbeitet haben: In MS365 ist z. B. Copilot enthalten.

Abgrenzung: Wieso fehlt hier das wichtige [Produkt Ihrer Wahl]?

Die grundlegende Architektur, Prozesse und der Technologie-Stack sind im Web Frontend und Backend stabil gesetzt.

Entwicklungswerkzeuge und Frameworks entwickeln sich jedoch schnell, auch entwickeln sie selbst wieder eigene Stacks.

Eine Vorlesung wie WTA, die über die Zeit stabile Hintergründe und Konzepte vorstellt, gibt daher keine umfassende Schulung zu einem bestimmten Produkt – die findet man bei den jeweiligen Anbietern. Natürlich nutzen wir beispielhaft konkrete Software.

Die Kenntnisse der Grundlagen sind notwendige Bedingungen für eine schnelle Einarbeitung in neue Produkte und insbesondere auch für die Beurteilung ihrer Qualitäten und Vor-/Nachteile für bestimmte Anwendungen.

Auch bilden sich die Frameworks für Frontend und Backend am Ende immer auf die hier etablierten Grundlagen ab: Browser rendern HTML und haben RTEs für JS (z. B. kann WebAssembly nicht das DOM ändern).

„move fast and break things“ ;-/

ACHTUNG

Um so höher im Stack heutiger Infrastrukturen wir klettern, desto wackeliger wird die Stabilität.

Es gibt eine sehr schnelle Entwicklung, viele dynamische Akteure und eine sehr hohe Komplexität der beteiligten Softwaresysteme.

Das bedeutet für uns:

- Die Softwares, die wir hier einsetzen, funktionieren auch mal nicht. Oder nach dem nächsten Update arbeitet die eine mit der anderen nicht mehr zusammen.
- Es kommt schnell vor, dass man sich eine Systemkonfiguration zerschießt. Daher ist es empfehlenswert, möglichst mit VMs zu arbeiten, von wegen der Möglichkeit „Spielstände“ zu sichern, oder ein System einfach mal schnell komplett neu aufzusetzen, z. B. als *Infrastructure as Code*.

Zu den praktischen Übungen bekommen Sie daher auch Screenshots, wie es aussehen sollte. Wenn etwas in der Vorlesung nicht klappt, versuchen Sie das nachzuholen. Es ist nervig, wenn Dinge nicht laufen, man kann aber viel daraus lernen für die betriebliche Realität.

Ziele unserer Veranstaltung

- Die Infrastrukturen aus CI nutzen: Anwendungen mit Containern und dem Protokoll-Stack oberhalb von TCP.
- Aufbau und Abläufe der verteilten Anwendungen und Dienste im Internet verstehen:
  - Wie funktionieren die?
  - Wie bauen wir sie?
  - Wie betreiben wir sie?

Das sind Grundlagen für

- einen fundierten Umgang mit aktuellen IT-Architekturen im kommerziellen Betrieb,
- die Abschätzung von Chancen und Risiken in der Service- und Produktentwicklung,
- die Entwicklung von stabiler und sicherer Software zum Einsatz im Internet.

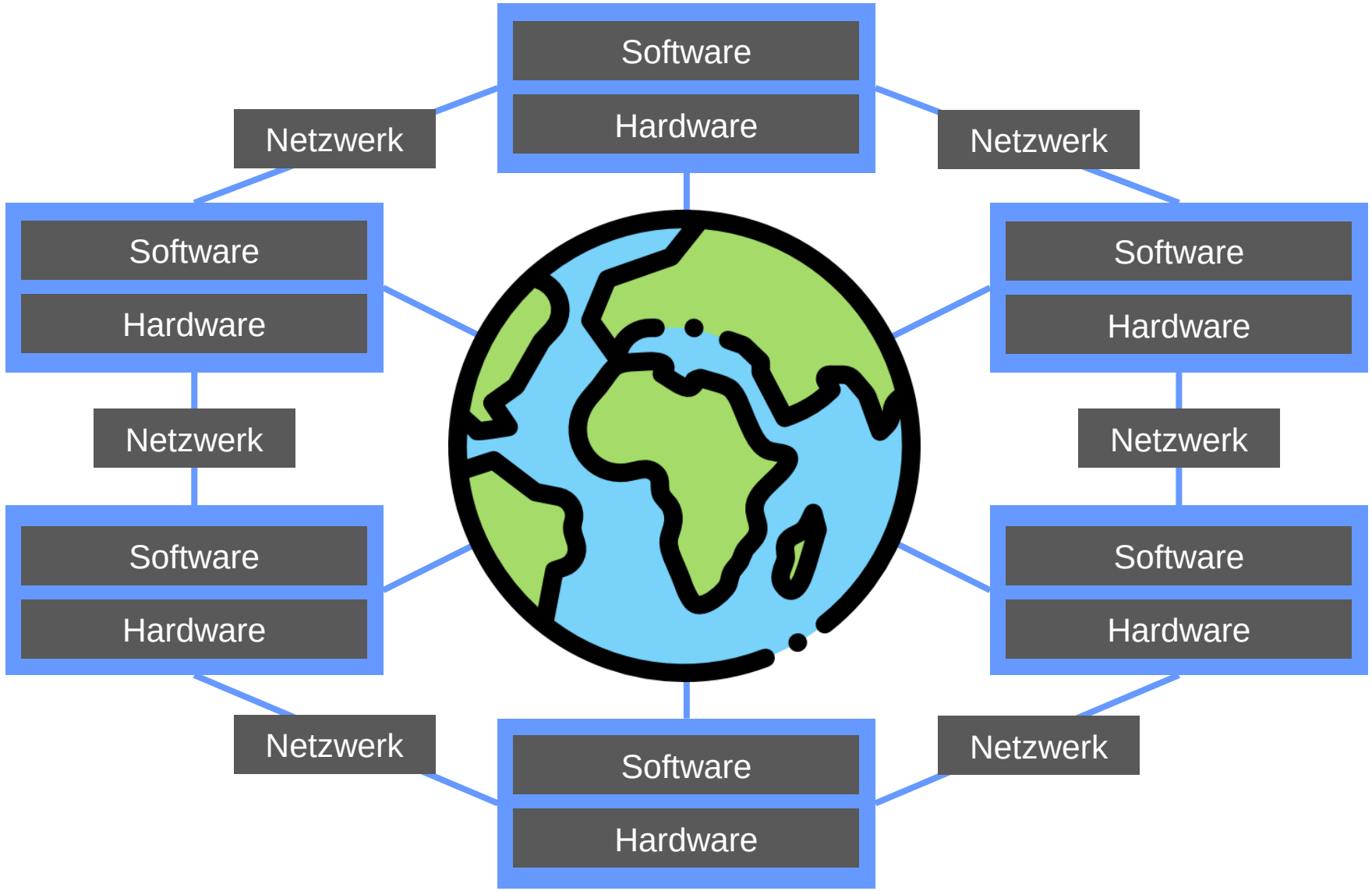
Unser Vorgehen setzt sich auf der Anwendungsschicht 5 fort: Interaktionen zwischen Web Client und Server, Austausch und Darstellung von Content.

Inhalte dieses Moduls

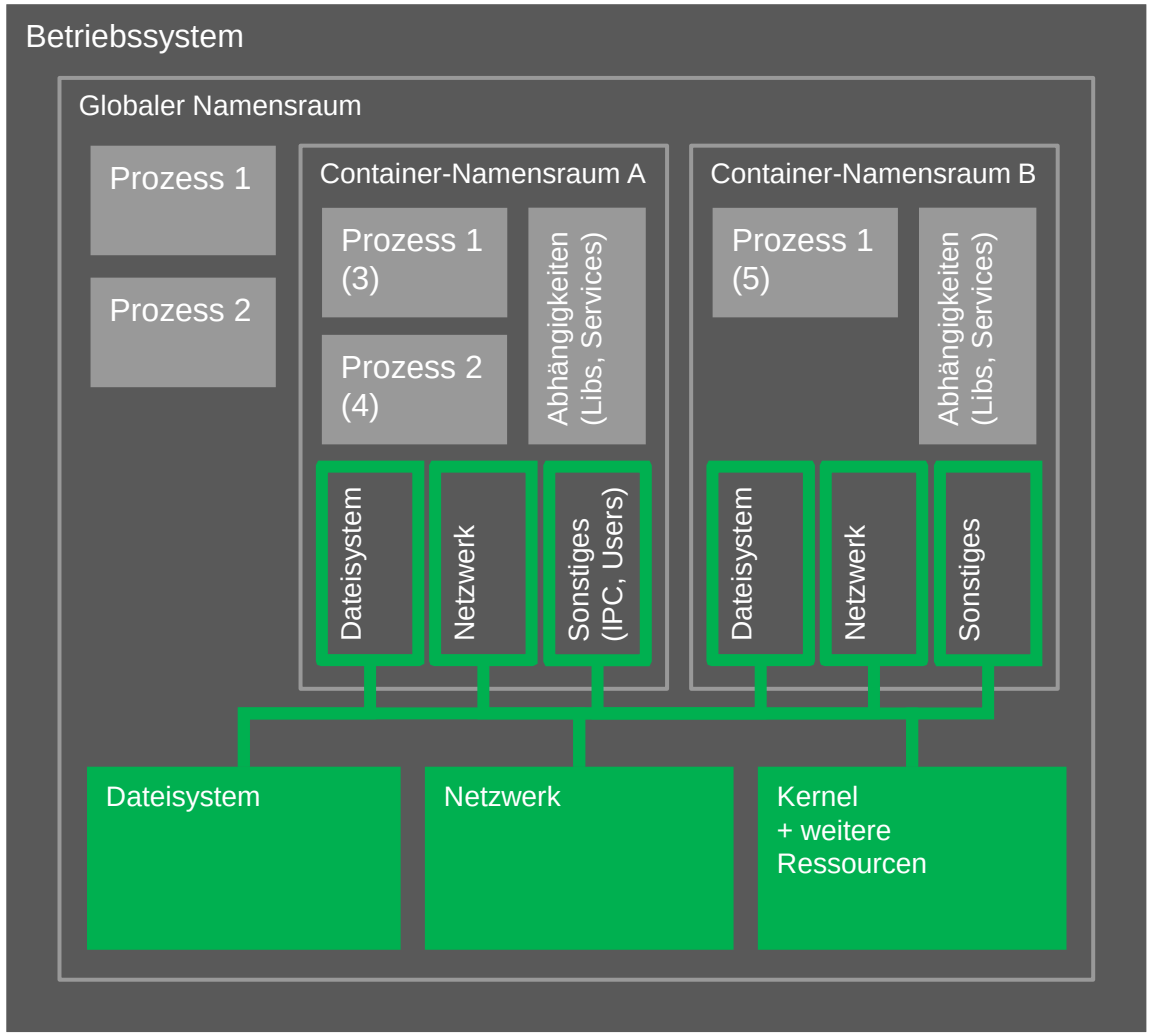
Infrastruktur – kurze Wiederholung	Elemente von Betriebssystemen und Internet, auf die wir aufbauen.
TLS und DNS – ein paar Ergänzungen	Absicherung der Datenflüsse, Vereinfachung der Host-Adressierung.
Hyper Text Transfer Protocol (HTTP)	Die populärste Kommunikation des Web auf der Anwendungsschicht 5.
JSON, XML, HTML+CSS, JavaScript	Content im Web, seine Formate und Eigenschaften.
Architektur und Umsetzung	Konzepte, Komponenten, Schnittstellen von Web-Anwendungen und -Diensten.
Deployment und Betrieb	Orchestrierung und Skalierung von containerisierten Anwendungen.
Absicherung	Identity and Access Management auf den Schnittstellen von Web-Diensten.

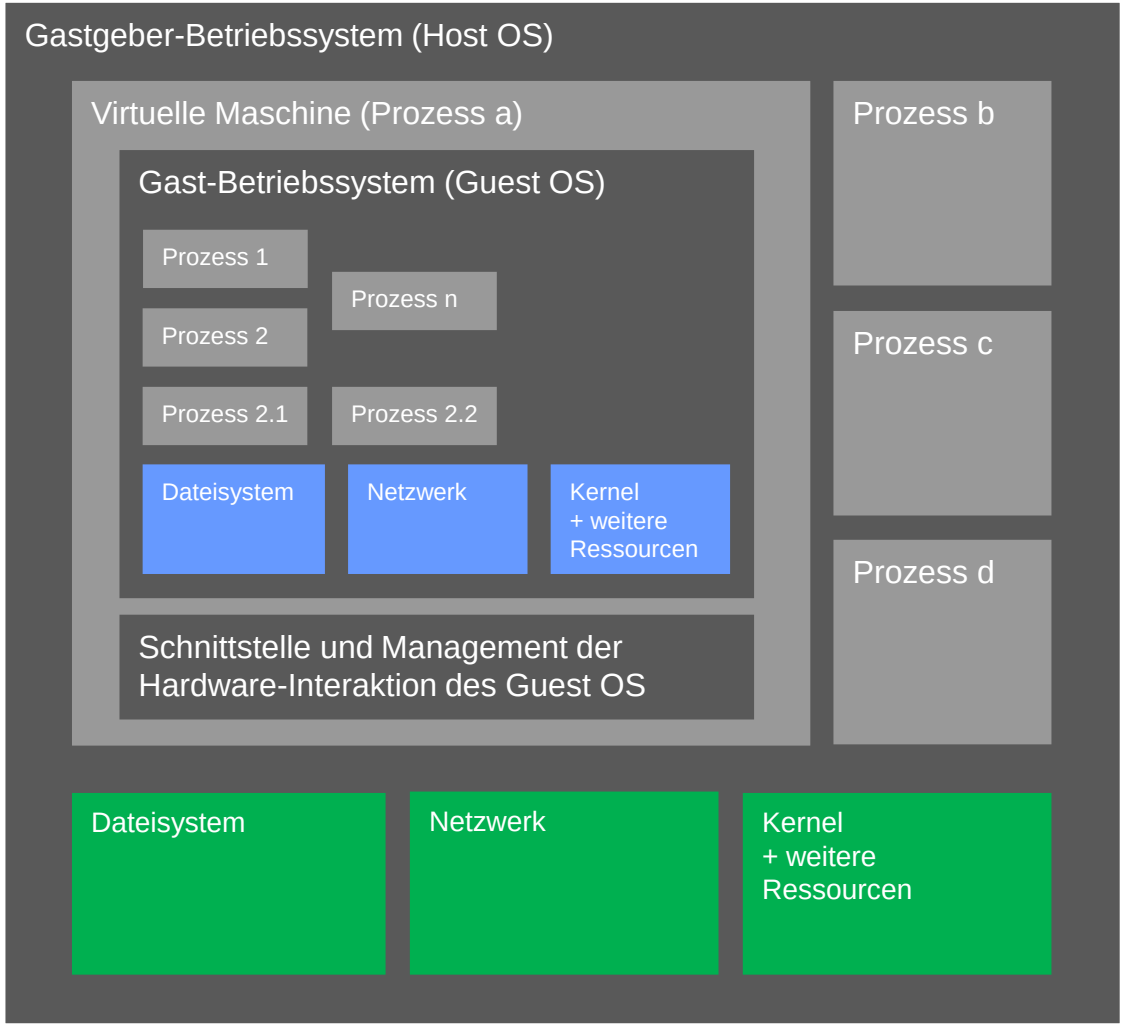
Kurze Wiederholung:
Die Infrastruktur unter dem Web

Die Infrastruktur unserer Welt: Technische Elemente

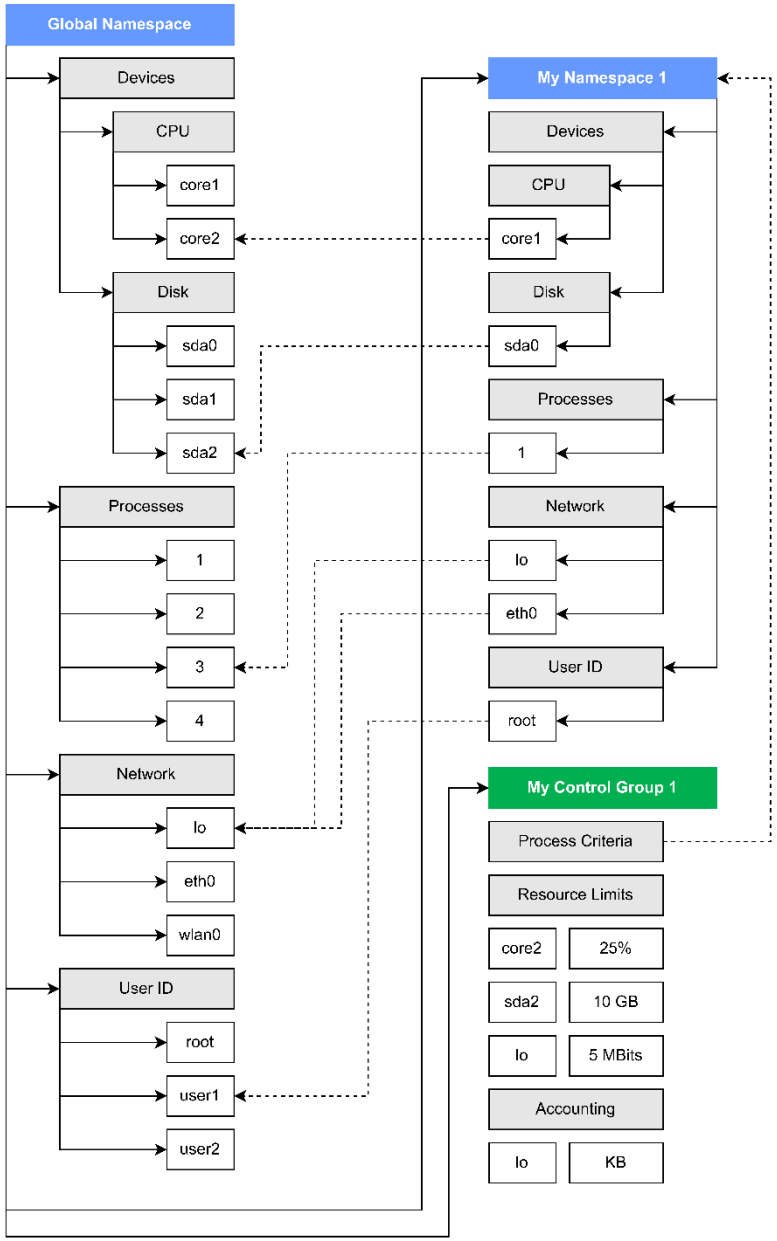


Container und virtuelle Maschinen





Container: Namensräume + Kontrollgruppen



Unter dem globalen Namensraum können wir uns unsere eigenen **Namespaces** definieren.

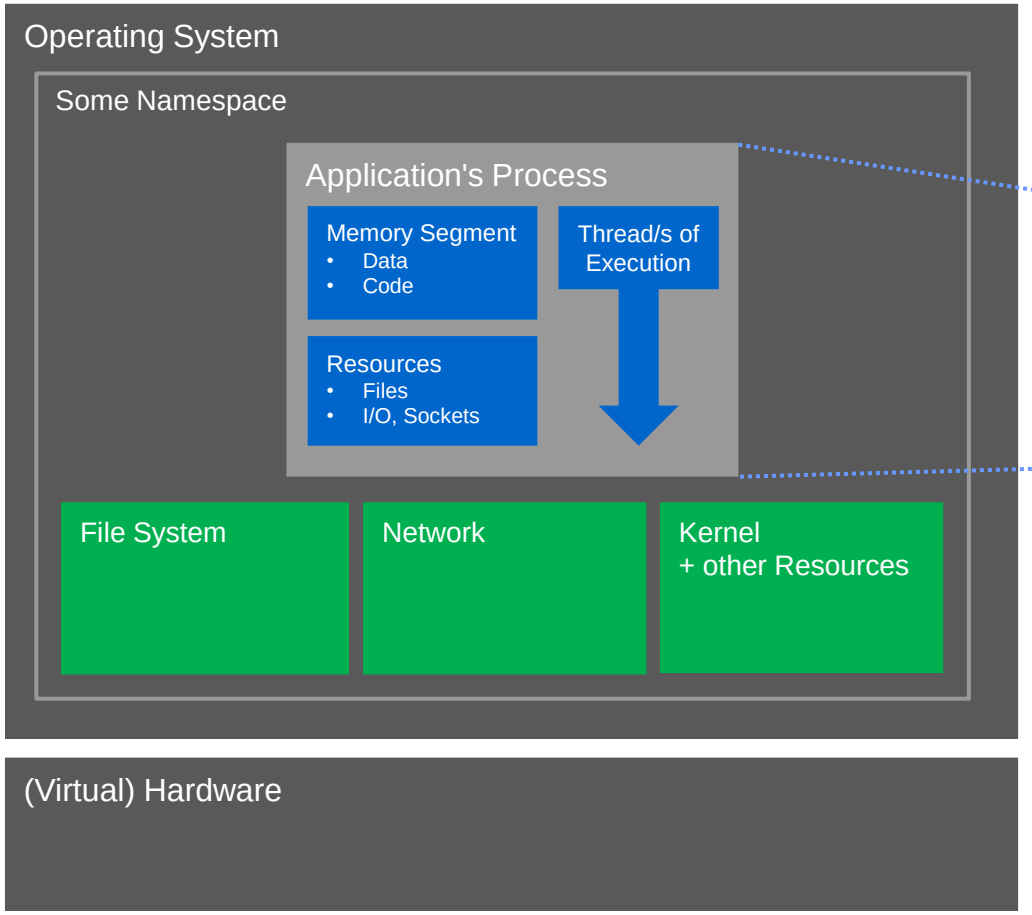
- Struktur ist die gleiche,
- Ressourcen legen wir an, wie wir wollen,
- Namen für Ressourcen können wir vergeben, wie wir wollen,
- *sie referenzieren aber immer Ressourcen des hierarchisch übergeordneten Namensraums*.

Zusätzlich können wir **Kontrollgruppen** bezogen auf bestimmte Prozesse definieren, z. B. für alle Prozesse in einem Namensraum.

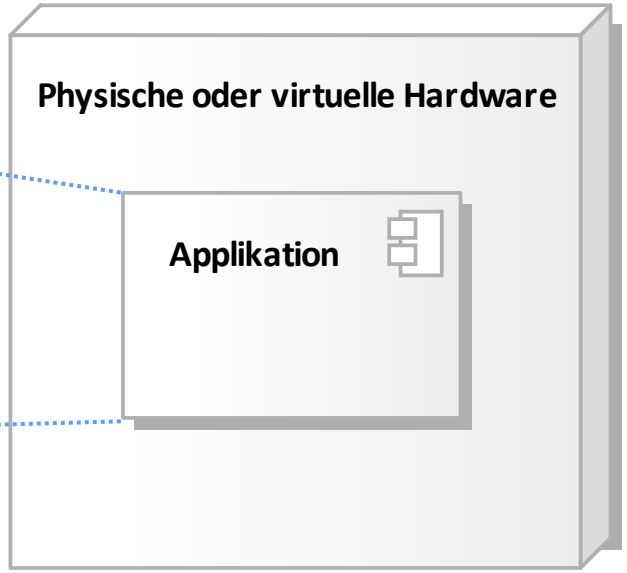
- Ressourcengrenzen / *resource limits* legen fest, wie viel Rechenzeit eines Kerns die Prozesse nutzen können, wie viel Speicher einer Partition, wie viel Bandbreite einer Verbindung usw.
- „Buchführung“ / *accounting* legt fest, welche Verbrauchsgrößen welcher Ressource protokolliert werden, z. B. übermittelte KB an Daten über eine Netzschnittstelle.

Prozesse als Anwendungen

Zu diesem Bild einer Anwendung waren wir in CI gekommen:

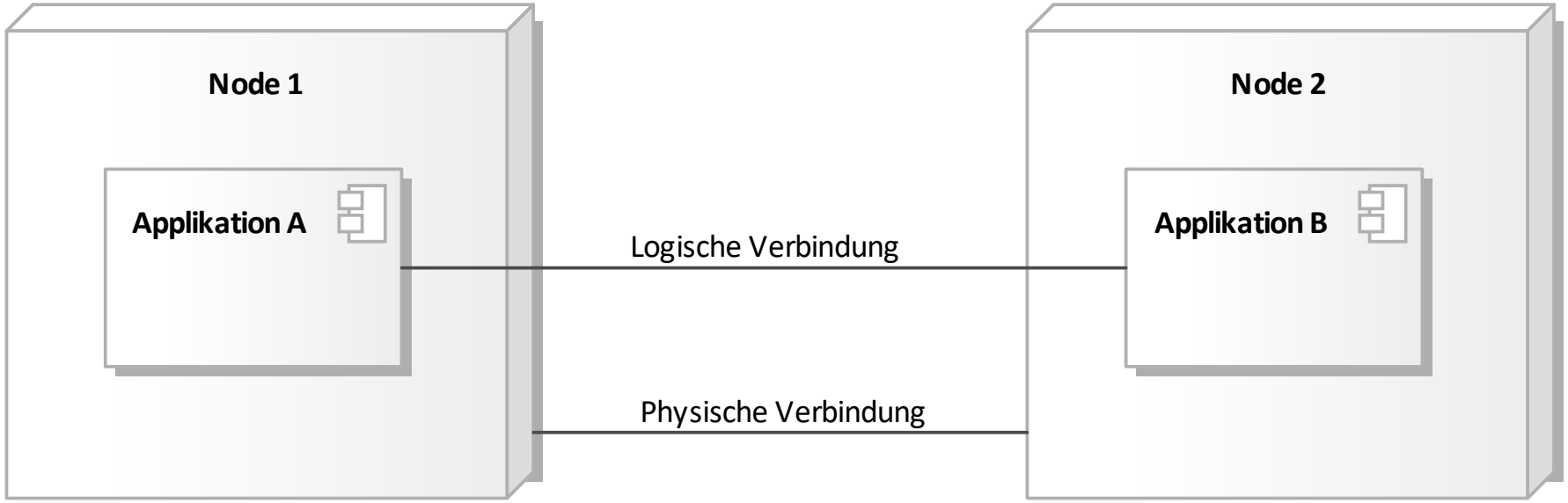


Das haben wir – für das Internet – zusammengefasst:



Verbindung zwischen Prozessen

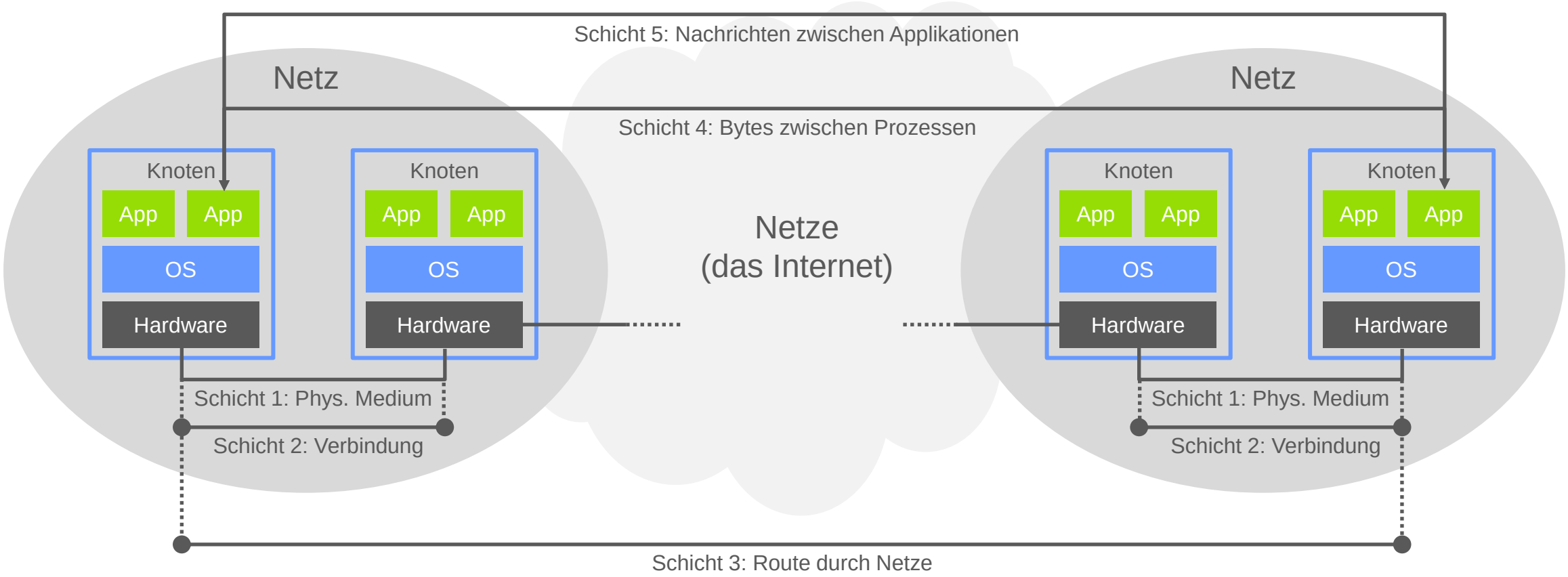
Dieses Schema nutzen wir in unserem Alltag ganz selbstverständlich:



Z. B.
- App auf Smartphone
- Applikation auf Notebook
- Dienst im Auto
- Spiel auf Konsole

Z. B.
- App auf Smartphone
- Dienst auf physischem Server
- Spiel auf Konsole
- Service in der Cloud

Applikationen im Internet: Überblick der Layer-Aufgaben

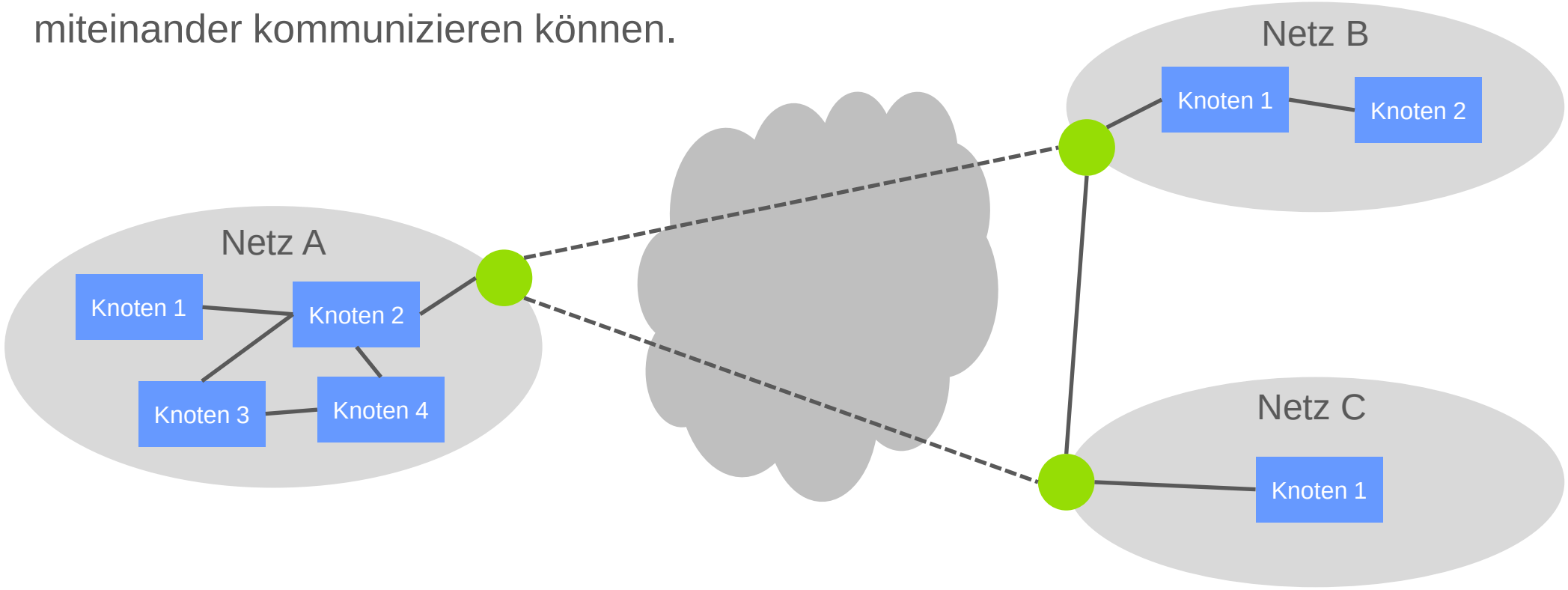


Netze

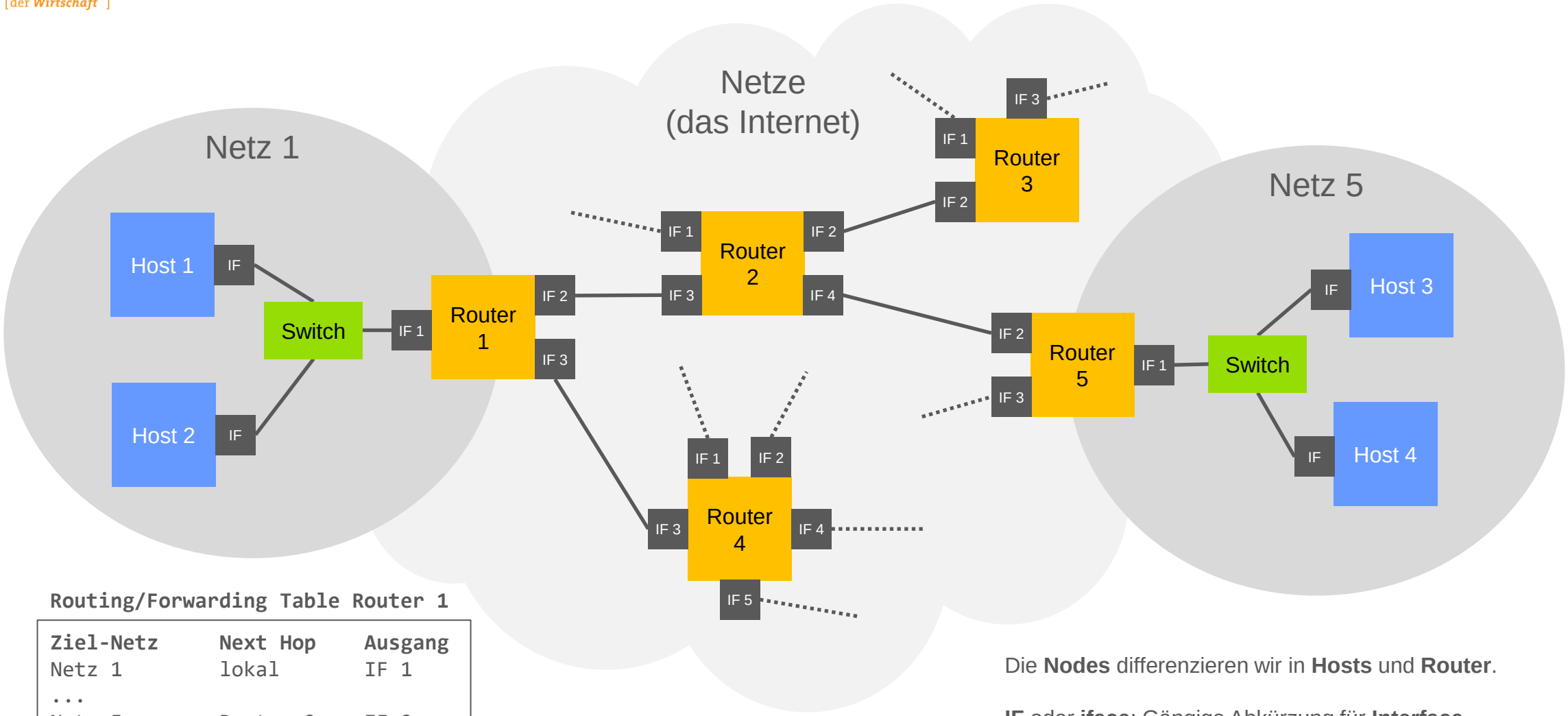
Die Kombination vieler Rechnerknoten (Nodes) führt zu Netzen, in denen

- nur direkt verbundene Nodes direkt,
- aber entfernte Knoten indirekt

miteinander kommunizieren können.



Routing und Forwarding



Routing/Forwarding Table Router 1

Ziel-Netz	Next Hop	Ausgang
Netz 1	lokal	IF 1
...		
Netz 5	Router 2	IF 2
...		
default	Router 4	IF 3

Die **Nodes** differenzieren wir in **Hosts** und **Router**.

IF oder **iface**: Gängige Abkürzung für **Interface**. Identifiziert durch eine *IP-Adresse*.

Switches leiten Pakete auf Schicht 2 an die lokalen MAC-Adressen der Geräte weiter.

TCP und Ports: Adressierung einer Anwendung im Internet

Bei TCP identifiziert das 4-er-Tupel

```
(Quell-IP, Quell-Port, Ziel-IP, Ziel-Port)
```

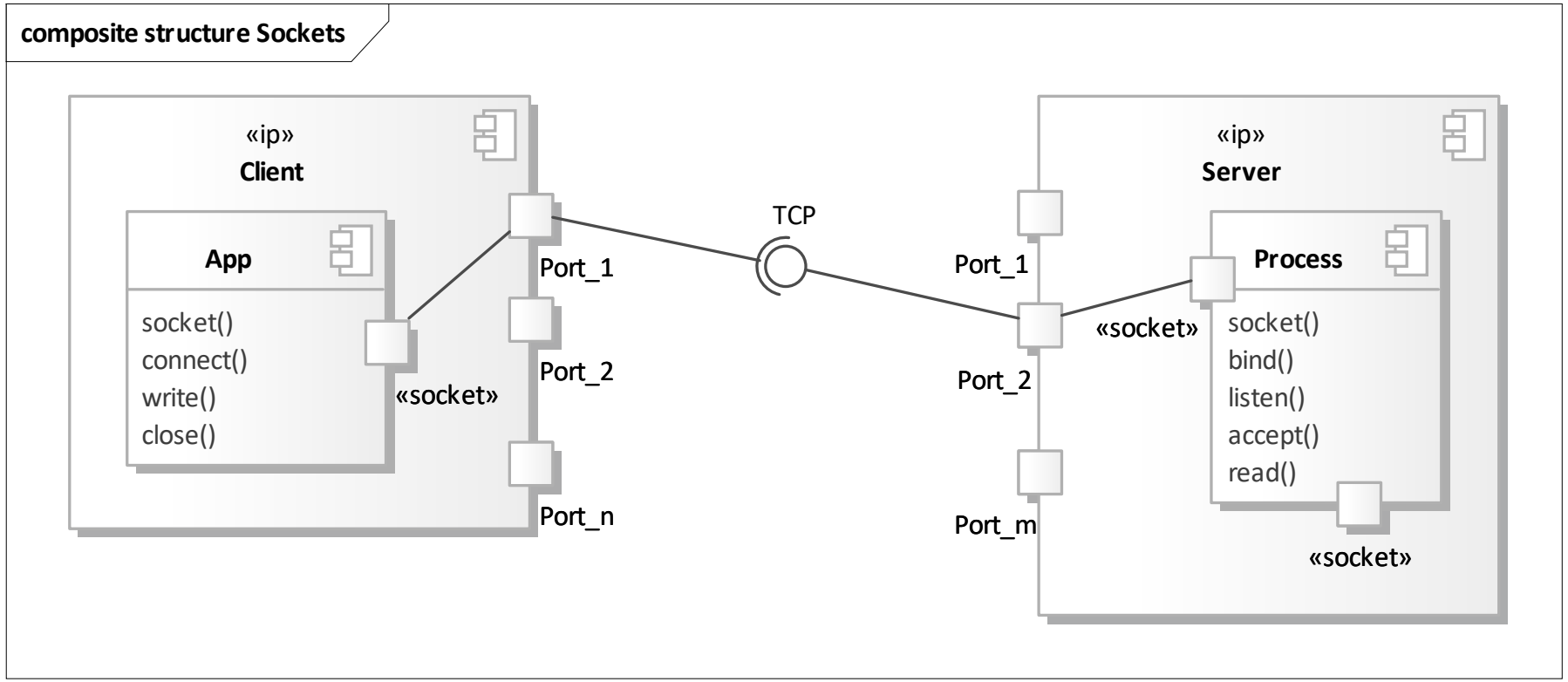
eindeutig einen Socket und damit die gewünschte **Verbindung zwischen Prozessen**.

Beim Multiplexing muss sich also nur ein Port unterscheiden.

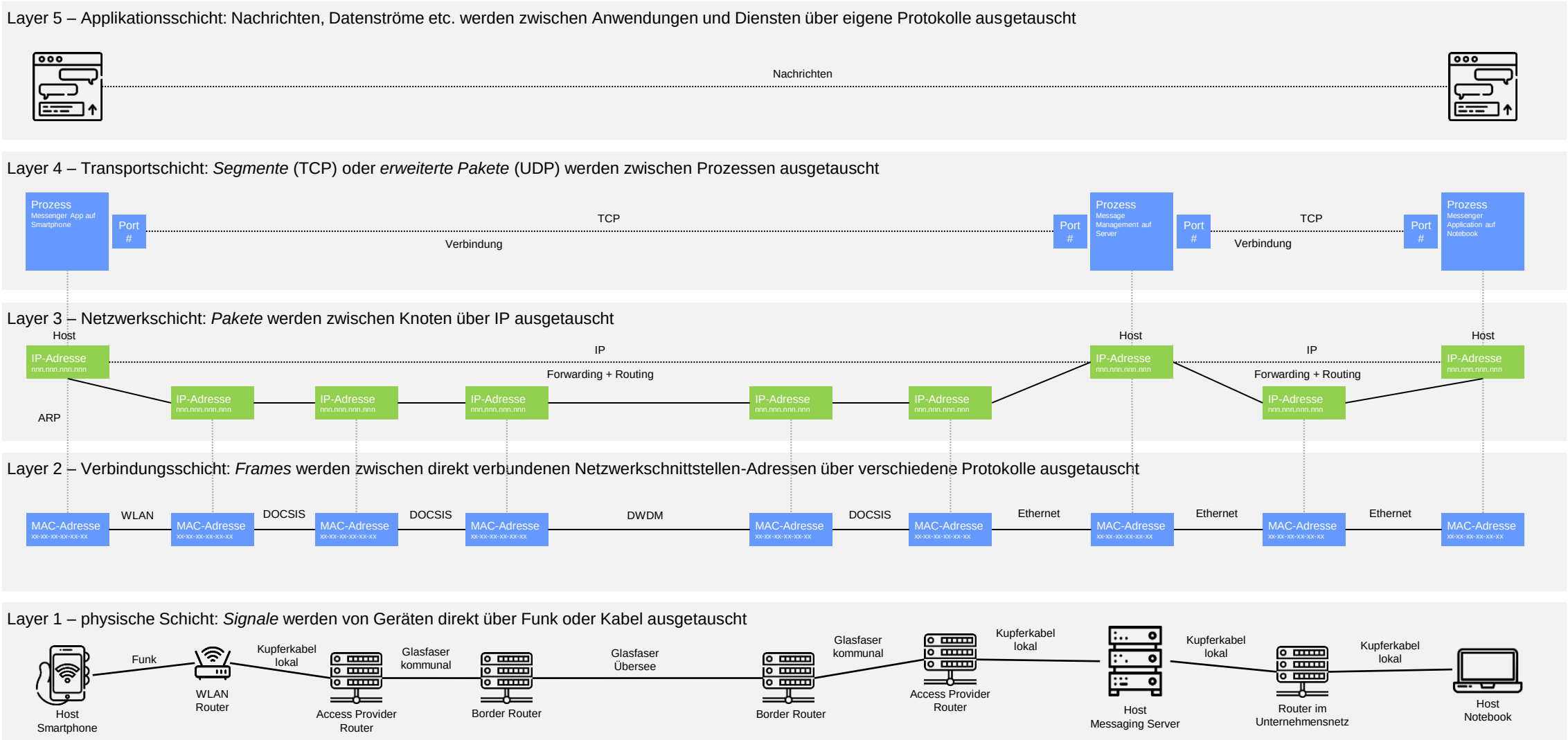
Die möglichen Ports sind eingeteilt:

- **Well-known** oder **kontrollierte** Ports von 1-1023
  - z. B. Port 80 für HTTP, 443 für HTTPS, 143 für Mailboxes (IMAP), 135 für Remote-Dienste.
  - Die Allokierung dieser Ports erfordert im Allgemeinen spezielle Rechte.
- **Registrierte** (aber frei verwendbare) Ports von 1024-49151.
- **Dynamische** bzw. **private** Ports von 49152-65535.

IP-Adressen, Sockets und Ports: Verbindung im Internet



Schichten im Internet



Icons made by Freepik, Eucalyp, xnimrodx from www.flaticon.com, Creative Commons BY 3.0

Ergebnisse der Infrastrukturen

Prozesse lassen unsere Anwendungen isoliert und voreinander geschützt laufen.

In **Containern** können wir unsere Anwendungen mit ihren Ressourcen (voreinander) isolieren und bündeln, um sie einfacher einsetzen zu können. Gerade auf verteilten Hosts im Internet.

All das ermöglicht ein **Kernel** als maßgebliche Managementinstanz eines Betriebssystems.

Mit **IP** können wir alle Arten von Rechnern als *Hosts* lokal und aus der Ferne adressieren und Datenpakete mit ihnen austauschen.

TCP verbindet unsere *Prozesse auf diesen Hosts* miteinander und sorgt für einen zuverlässigen und vollständigen Austausch von Byte-Strömen zwischen den Anwendungen.

Zwei Sachen wollen wir noch anmerken:

1. TCP ist zuverlässig, aber nicht sicher im Sinne von *authentisch*, *vertraulich* und *integer*.
2. IP-Adressen sind schwer zu merken und lästig einzugeben.

Was nutzen uns diese mühsam erlangten Kenntnisse nun in der Web-Welt?

Ergänzung: Infrastructure as Code

Die leistungsfähige Hardware *on-premise* und in der *Cloud* nutzen wir heute aus praktischen Gründen meist mit einer weiteren Schicht zur **Virtualisierung**.

Schon die Betriebsysteme abstrahieren unsere lokale Hardware und stellen virtuelle Umgebungen für unsere Anwendungen bereit (Prozesse).

Die weitergehende Virtualisierung erlaubt uns, viele vernetzte Rechner als flexible, also anpassbare Umgebung zu verwalten. Im Rahmen der physischen Ressourcen können wir darauf

- virtuelle **Rechnerknoten** und
- virtuelle **Netze**
- mit **Ressourcen und Nutzern nach Bedarf** einrichten.

Um diese komplexe **Verwaltung** und **Provisionierung** (möglichst) zu

- automatisieren,
- konsistent, fehlerfrei und
- reproduzierbar zu machen,

beschreiben wir die virtuellen Systeme als Skripte:

- **Imperativ / prozedural** – wie erreichen wir den Zielzustand unseres Systems?
- **Deklarativ / funktional** – wie sieht der Zielzustand unseres Systems aus?

Beispiele für Werkzeuge und Technologien für IaC

- *Ansible*, Konfig-Management, App-Deployment
- *Terraform*, Cloud scale, aber auch on-prem
- *Puppet, Chef*, Konfig-Management
- *Vagrant*, eher lokal, für Entwicklung.

Ergänzung: Everything as Code

Die heutigen komplexen, verteilten Anwendungen, Plattformen und Dienste unseres beruflichen und privaten Alltags lassen sich wirtschaftlich – effizient – (fast) nur in virtuellen Umgebungen betreiben.

- Entwicklung und Konfiguration erfolgt **as Code**,
- Management, Deployment und Nutzung erfolgt **as a Service**.

Unsere Web-Technologien und Applikationen sind heute Bestandteile dieser Konzepte und Architekturen, oder ermöglichen sie sogar erst.

Das gilt in großen kommerziellen Systemen, aber auch in kleinen, eigenen Übungen. Daher nutzen wir das hier.

Übung 1: Eine kleine Infrastruktur, praktisch mit Vagrant

1. Installieren Sie *VirtualBox* (oder nutzen Sie eine bereits bei Ihnen vorhandene, kompatible Virtualisierungslösung).
2. Installieren Sie *Vagrant*. Bitte beachten: Es gibt immer mal Inkompatibilitäten zwischen Vagrant und Provider der VMs.
3. Legen Sie ein `Vagrantfile` in einem geeigneten Verzeichnis an:

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/bionic64"
  config.vm.network "private_network", ip: "10.0.0.10"
end
```

4. Wechseln Sie via Shell in das Verzeichnis, dann `vagrant up`
5. Nehmen Sie ersten Kontakt zur VM über den Internet Stack auf:
   `ping 10.0.0.10`
6. Loggen Sie sich in der VM ein mit `vagrant ssh` und schauen Sie sich die Netzwerkschnittstellen dort an: `ifconfig`
7. Was für Prozesse laufen in der VM? `ps` und `ps -ef`
8. Probieren Sie verschiedene Vagrant-Kommandos aus:
   `vagrant`, `vagrant status`, `vagrant box list`…
9. Am Ende `vagrant halt`

Übung 1: Eine kleine Infrastruktur, praktisch, alternativ mit Docker

Sollte Vagrant nicht funktionsfähig/gewünscht/nötig sein:

1. Installieren Sie Docker (oder einen kompatiblen Verwandten davon).
2. Legen Sie eine Datei `docker-compose.yml` in einem geeigneten Verzeichnis an.
3. Starten Sie den Container mit `docker compose up -d` (Parameter d für *detached*).
4. Schauen Sie sich das Netzwerk zwischen den Containern an: `docker network ls` und `docker network inspect`
5. Starten Sie eine Shell in einem Container mit `docker exec -it container1 /bin/bash`
6. Was für Prozesse laufen im Container? Und außerhalb? `ps` und `ps -ef`
7. Wie sieht es mit Namespaces aus? `lsns`
8. Schauen Sie sich die Netzwerkschnittstellen mit `ifconfig` an. `ping`-en Sie den anderen Container über das Netz zwischen beiden an.
9. Verlassen mit `exit`, Container beenden mit `docker compose down`

docker-compose.yml

```yaml
services:
  container1:
    image: ubuntu:latest
    container_name: container1
    networks:
      small_internal_network:
        ipv4_address: 10.0.0.10
    command: /bin/bash -c "apt-get update && apt-get install -y
iputils-ping net-tools && /bin/bash;while true; do sleep 30;
done"
    tty: true

  container2:
    image: ubuntu:latest
    container_name: container2
    networks:
      small_internal_network:
        ipv4_address: 10.0.0.11
    command: /bin/bash -c "while true; do sleep 30; done"
    tty: true

networks:
  small_internal_network:
    driver: bridge
    ipam:
      config:
        - subnet: 10.0.0.0/24
```

Was bedeutet das alles?

Infrastructure/Platform/Software as a Service

On-Premise(s)	Infrastructure as a Service	Platform as a Service	Software as a Service
Anwendungen	Anwendungen	Anwendungen	Anwendungen
Daten	Daten	Daten	Daten
Laufzeitumgebung	Laufzeitumgebung	Laufzeitumgebung	Laufzeitumgebung
Middleware	Middleware	Middleware	Middleware
Betriebssystem	Betriebssystem	Betriebssystem	Betriebssystem
Virtualisierung	Virtualisierung	Virtualisierung	Virtualisierung
Server	Server	Server	Server
Massenspeicher	Massenspeicher	Massenspeicher	Massenspeicher
Netzwerk	Netzwerk	Netzwerk	Netzwerk
Selbst verwaltet			Extern verwaltet / Cloud

Eine Ergänzung zur Sicherheit auf Schicht 4: Transport Layer Security (TLS)

TLS: Ansatz

Die Protokolle der Ebene 4 liefern in den gängigen Fällen eine Kommunikation, die

- *unverschlüsselt* und
- *ohne Identitätsprüfung* abläuft.

Schon Mitte der 90er-Jahre wurde mit *Secure Sockets Layer* (SSL) eine Bibliothek auf Ebene 5 entwickelt, die eine sichere Kommunikation erlauben sollte.

Seit 1999 existiert der Nachfolger **Transport Layer Security** (TLS), dessen Versionen 1.2 und v. a. 1.3 heute alleine genutzt werden sollten.

TLS selbst erwartet einen „reliable, in-order data stream“ (RFC 8446), so dass Anwendungen TLS als Zwischenschicht meist vor TCP verwenden und TCP so nur verschlüsselte Bytes überträgt. TLS-Nachrichten sind dann aber TCP-Segmente.

Üblicherweise läuft TLS auf *Port 443 TCP* oder *UDP*. Darüber läuft dann z. B. der gesicherte Datenverkehr zwischen Browser und Webserver über HTTP*S*, wenn HTTP-Daten in TLS verpackt sind.

TLS: Eigenschaften

TLS besteht aus zwei Teil-Protokollen:

Handshake

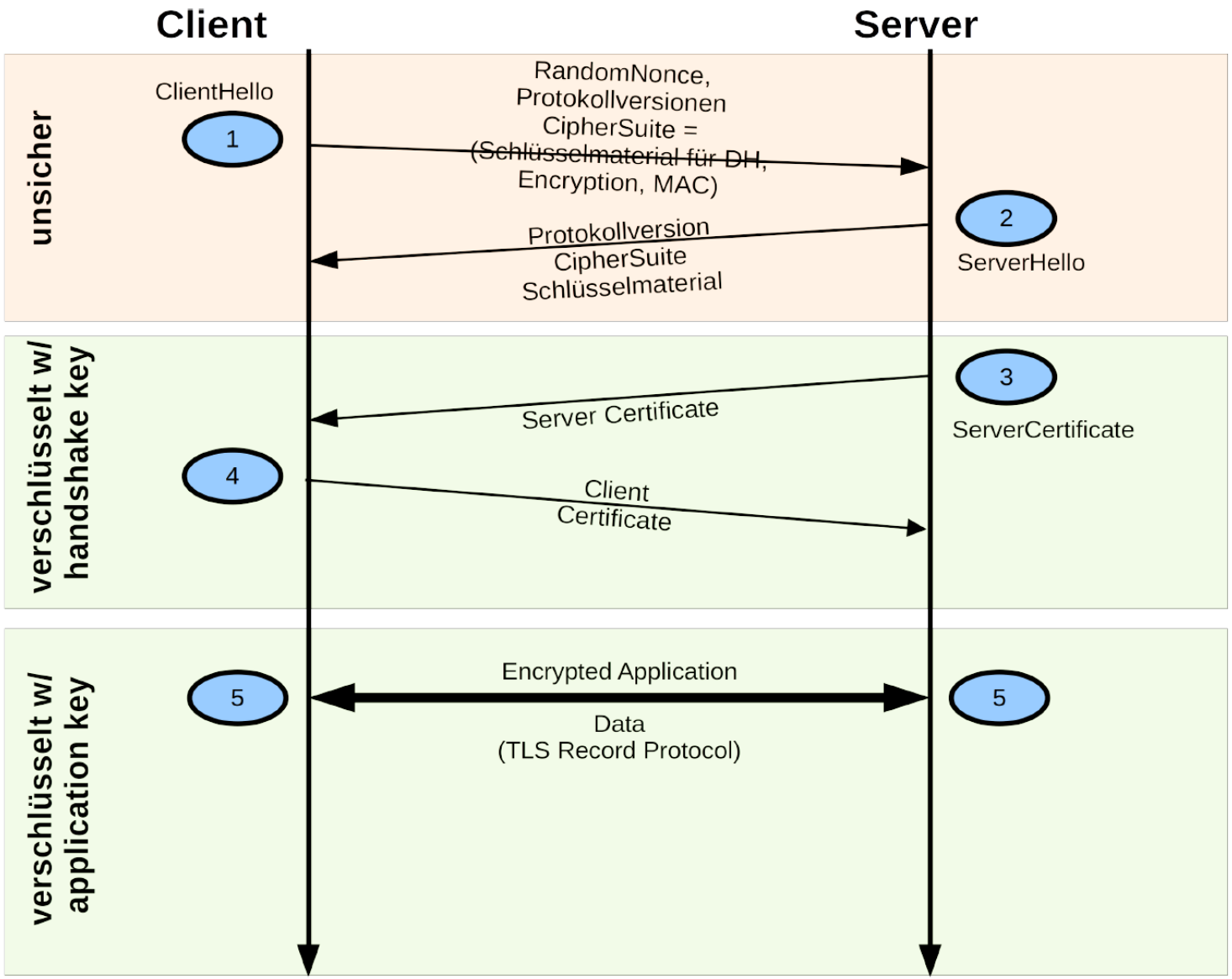
- *Authentifizieren* der Partner
- Aushandeln der Security-Verfahren
- Aufbau der gemeinsamen Keys

Record

- Nutzen der oben ausgehandelten Verfahren und Keys zum
- *Verschlüsseln* und *Verifizieren* einzelner Pakete, Records.

In TLS 1.3 garantieren die Verfahren zusätzlich eine *Perfect Forward Secrecy*: Es gibt für jede Verbindung einen neuen *Session Key*, der ohne einen dauerhaft gültigen Schlüssel zwischen den Kommunikationspartnern vereinbart wird. Bei Angriffen müssten die Daten jeder einzelnen Verbindung teuer entschlüsselt werden, die geknackten Schlüssel einer Verbindung lassen keine Rückschlüsse auf andere Verbindungen zu.

TLS: (Ein möglicher und vereinfachter) Ablauf



Client: „Hallo, das sind meine Kryptoparameter und TLS-Version, die ich nutzen kann und eine Zufallszahl als *Nonce* (number used once), um diesen Handshake eindeutig zu machen.“

Server: „Hallo, das ist die stärkste TLS-Version, die wir nutzen können, hier meine Kryptoparameter und meine Nonce.“

Der Server schließt die Begrüßung ab, indem er sein Zertifikat schickt.

Der Client antwortet mit seinem Zertifikat. Er kann außerdem das Server-Zertifikat bei einer CA überprüfen.

Die öffentlichen Schlüssel in den Zertifikaten und die Nonces fließen in die Generierung eines Master Secrets ein, was wiederum zur Erzeugung der Session Keys verwendet wird.

Client und Server nutzen die Session Keys zur Verschlüsselung der ausgetauschten Nachrichten.

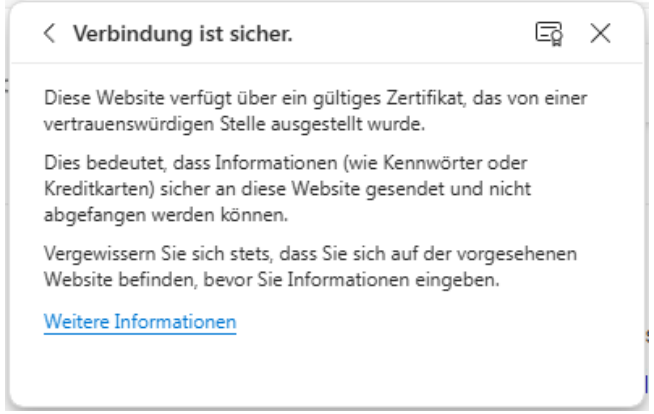
Durch die Nonces sind Replay-Angriffe verhindert.

Schafft es ein Angreifer, das Master Secret für eine Verbindung zu ermitteln, da er die Nonces abgefangen hat und die Schlüssel knacken konnte, ist es nutzlos für alle weiteren Verbindungen.

Übung 2: Verständnisfragen zu TLS

Welche Aussagen sind richtig oder falsch und warum?

1. Die ausgetauschten Daten zwischen meinem Browser und dem Server sind durch TLS für Dritte nicht lesbar. Es sieht z. B. also niemand, was ich auf einer Website mache.
2. Das Zertifikat des Servers überprüft mein Client bei einer Zertifizierungsstelle (CA; Certificate Authority). Dadurch weiß ich genau, dass ich mit dem richtigen Server kommuniziere, z. B. dem meiner Bank.



Ein Hilfsmittel auf Schicht 5: Domain Name Service (DNS)

DNS: Einführung

Die Adressierung von Hosts im Internet über IP-Adressen, z. B. `217.29.41.139` oder sogar `2001:db8:0:8d3:0:8a2e:70:7344`, ist für Menschen unbequem:

- Wir wollen Hosts, entfernte Rechner, durch „sprechende“ Namen finden.
- Dafür muss eine Umsetzung von diesen Namen auf die bzw. eine IP-Adresse geschehen.
- Ursprünglich gab es dazu nur die Datei `hosts` (ausnahmsweise mal unter Unix *und* Windows). Zeilenweise einfach eintragen
  `[IP address] [DNS name(s)]`

Leichter macht es uns das **Domain Name System (DNS)**:

- Das DNS ist ein Client-Server-System mit vielen Servern.
- Ein Server wird als **Nameserver** bezeichnet.
- Sein Protokoll läuft als Anwendung auf Schicht 5, gängig auf *Port 53 UDP*.

DNS: Prinzip und Aufbau

Der Aufbau des DNS-Systems kann aus folgender Logik erschlossen werden:

1. Eine Tabelle für die Zuordnung *Name* $\rightarrow$ *IP* muss existieren.
2. Eine Tabelle für das gesamte Internet ist zu groß
   $\Rightarrow$ aufteilen auf viele Rechner (*partitionieren*).
3. Immer noch zu viel Last pro Rechner
   $\Rightarrow$ verteilen (*replizieren*).
4. Es ergibt sich eine sehr große partitionierte und replizierte Tabelle.

DNS kann als verteilte Datenbank mit vielen Servern betrachtet werden.

Folgerungen aus diesem Ansatz:

- – Konsistenzwahrung
- + hohe Performance
- + hohe Ausfallsicherheit

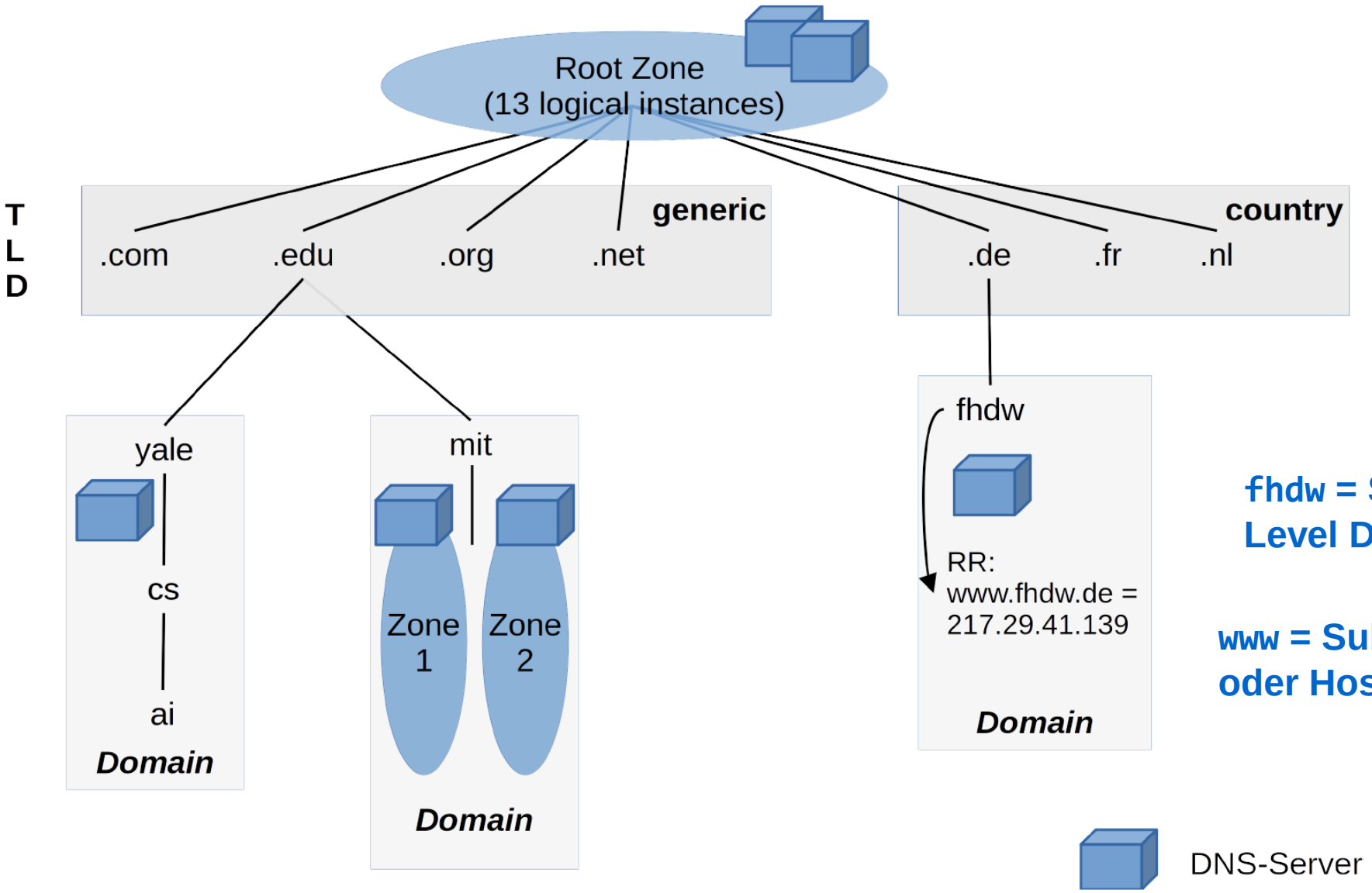
DNS: Begriffe

Im DNS sind folgende Begriffe etabliert:

- Der (case-insensitive) (**DNS) Name** ist der „sprechende“ Name eines Hosts, z. B. `fhdw.de`
- Die DNS-Namen sind hierarchisch aufgebaut und bilden damit einen **Namensbaum**.
- Eine **Domain** ist ein zusammenhängender Teilbaum davon.
- Viele **Nameserver** bearbeiten die Anfragen.
- Eine **Zone** ist eine Verwaltungseinheit des Domänenbaums, für die ein Nameserver verbindliche („authoritative“) Antworten geben kann. Die **root**-Zone verwaltet die Wurzel des Baums.
- **Resource Records** (RR) halten die DNS-Daten auf den Nameservern.
- Das Auffinden einer IP-Adresse aus dem DNS-Namen ist ein **Lookup**.

DNS: Domains und Zones

Top Level Domains



`fhdw` = Second Level Domain

`www` = Subdomain(s) oder Host

DNS: Server-Seite

Es existieren verschiedene Typen von Nameservern:

- **Primäre** Nameserver – verantwortlich (*authoritative*) für bestimmte Zonen.
- **Sekundäre** Nameserver – spiegeln die Daten der primären Nameserver.
- **Caching-only** Nameserver – besitzen keine eigenen persistenten Daten, sondern speichern nur erhaltene Ergebnisse zu Lookups.

Die Konfiguration eines Nameservers geschieht über **Resource Records**:

- Darin liegt die gesamte Information des DNS.
- Werden häufig in lokalen Dateien auf dem – primären oder sekundären – Nameserver gehalten.

DNS: Resource Records

Der Aufbau eines Resource Record:

`[name] [ttl] [class] [type] [optional data length] [data]`

- `type = SOA` – Start of Authority, beschreibt und definiert eine Zone bzw. deren Parameter, z. B. primärer Nameserver, Email der verantwortlichen Person etc.
- `type = NS` – Name Server, welche Server (in `data`) sind verantwortlich (authoritative) für Zone `name`.
- `type = A` bzw. `AAAA` – gibt IPv4 bzw. v6-Adresse (in `data`) für Host `name` an.
- `type = PTR` – Pointer, gibt Name (in `data`) zu IP-Adresse `name` an; *reverse lookup*.
- `type = MX` – Mail Exchange, gibt Mail-Empfänger (in `data` als DNS-Namen) für die jeweilige Domain in `name` an.
- `type = CNAME` – Canonical Name, gibt (in `data`) den „echten“ Namen zu einem Alias in `name` an. Es können mehrere Domains auf eine IP-Adresse zeigen.
- `type = TXT` – Textinformation zu der Domain, z. B. zum Inhaber oder zu Sicherheitsfeatures.

DNS: Ablauf Query

Der typische Ablauf einer DNS-Anfrage ist damit wie folgt:

1. Ein Client, z. B. Browser, Mailer, oder sonstige App, benötigt die IP-Adresse für einen DNS-Namen.
2. Er findet seinen Default-Nameserver (DNS Resolver) und stellt die Anfrage.
3. Wenn der Nameserver die Antwort kennt, gibt er sie zurück.
4. Wenn der Nameserver die Antwort *nicht* kennt,
   - dann fragt ein **rekursiver** Nameserver seinerseits weitere (u. a. **Root**-) Nameserver – bis die Antwort dem Client gegeben werden kann.
   - liefert ein **nicht-rekursiver** Nameserver eine Referenz auf einen weiteren – übergeordneten – Nameserver zurück. Der Client muss folgende Anfragen selbst stellen.

Die Abfragen werden normalerweise über UDP gestellt.

Übung 3: Praktische Nutzung von DNS

Eine DNS-Abfrage können wir auch manuell mit `nslookup` oder `dig` auf der Kommandozeile durchführen, z. B.:

`nslookup fhdw.de` – „nicht autorisierende Antwort“ folgt darauf (die ersten beiden Zeilen sind Ihr DNS Resolver).

`nslookup –type=NS fhdw.de` – verantwortliche Nameserver finden.

1. Fragen Sie Informationen über eine Website Ihrer Wahl mit den unterschiedlichen Parametern ab (z. B. den RR Types vorne für `-type`, `-debug`, oder weiteren über `-?` oder `--help`).
2. Was bedeutet es, wenn wir mehrere IP-Adressen für eine Domain bekommen? Wozu dient DNS auch?
3. Was sind potentielle Probleme, z. B. mit Sicherheit und Verfolgbarkeit, bei DNS in der einfachen, vorgestellten Form?

Die populärste Kommunikation auf der Anwendungsschicht 5: Hyper Text Transfer Protocol (HTTP)

HTTP: Einführung

Die Grundlage für den Großteil des heutigen Internet, d. h. der als „World Wide Web“ (WWW) sichtbare Teil, wurde Ende der 80er Jahre im CERN gelegt:

- Die Grundfrage: *„Wie können Dokumente vernetzt werden?“*
- Das ist für die effiziente Zusammenarbeit in einer dezentralen Organisation notwendig.

In diesem Zusammenhang wurden Protokolle für die **Datenübertragung** und Sprachen für die **Struktur der Dokumente** definiert:

- Datenübertragung – **Hyper Text Transfer Protocol (HTTP)**
- Inhalte – **Hyper Text Markup Language (HTML)**, heute mit CSS, Java-/TypeScript etc., außerdem für komplexe Schnittstellen und Strukturen mit REST und JSON.

Die Architektur des daraus resultierenden **World Wide Web (WWW)** basiert auf einem einfachen **Client/Server**-Modell.

HTTP: Eigenschaften

HTTP ist das grundlegende Protokoll für den Datenaustausch im Web:

- Es ist ein Protokoll der **Anwendungsschicht, Layer 5**.
- Es ist ursprünglich ein **Klartext- also ASCII-kodiertes** Protokoll, das auf TCP aufsetzt.
- Arbeitet über den Austausch von **HTTP Requests** vom Client und **Responses** vom Server, C/S-Prinzip.
- Wird zunächst als **zustandslos** bezeichnet – der Server hält *gemäß Protokoll* keine Informationen über den Client.
- Mit TLS für die Absicherung wird **HTTPS** daraus.

HTTP: Versionen

HTTP/1.0/1.1

- Klartext-Protokolle.
- Kommunikationsbeginn nur vom Client (*pull*).
- Eigene TCP-Verbindung für jedes Objekt in 1.0, in 1.1 TCP parallel möglich, aber sequentielle Abarbeitung vom Server, also *head-of-line blocking* möglich.

HTTP/2 (2015 standardisiert)

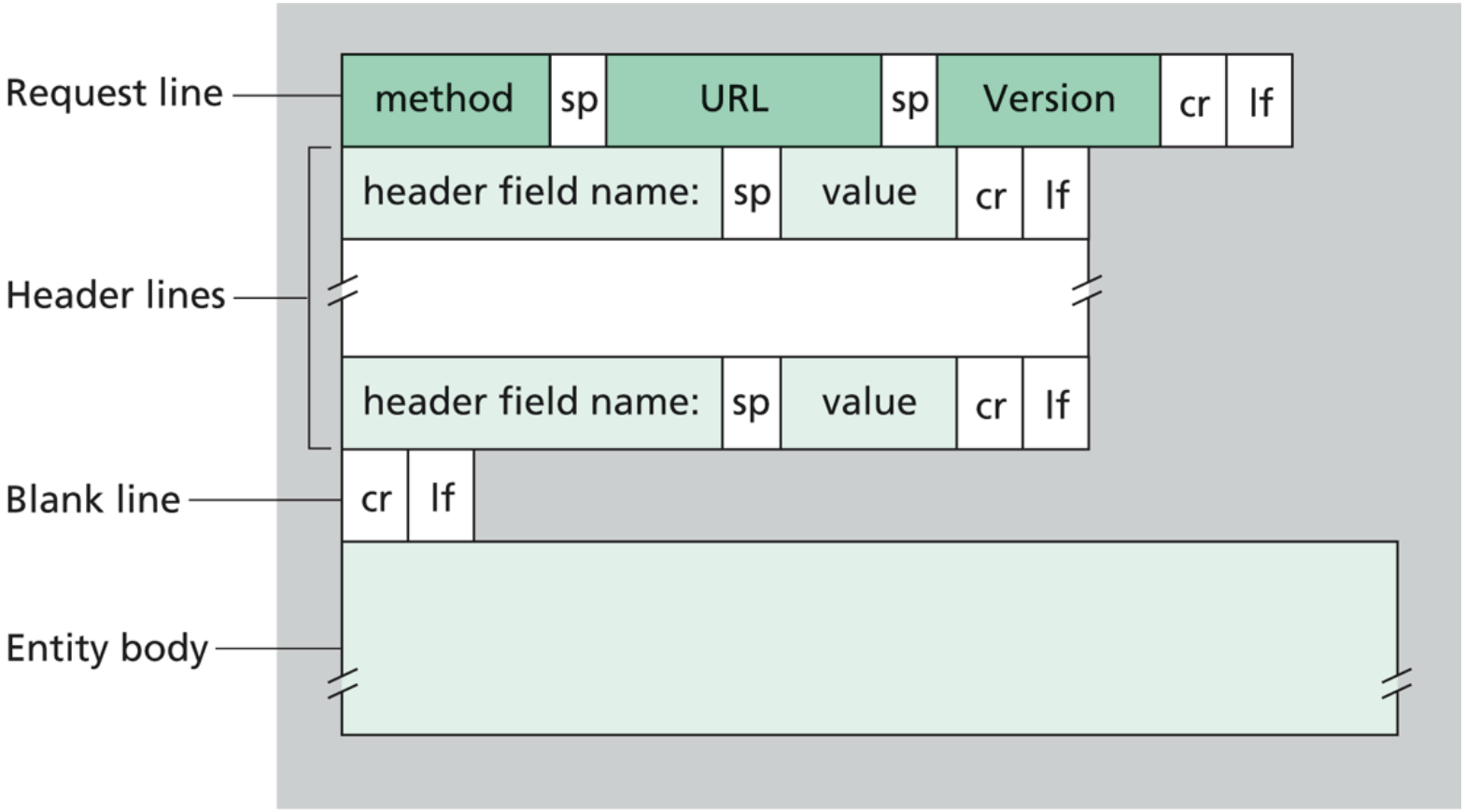
- Requests und Responses binär als *Frames*. Logik und Aufbau aber wie gewohnt.
- Multiplexed, mehrere Objekte parallel als *Streams* über *eine* TCP-Verbindung.
- Aber damit verschärftes *head-of-line blocking* auf TCP-Ebene.
- *Server Push* kann zusätzliche Daten im Response liefern.

HTTP/3

- Basiert auf *QUIC* (Quick UDP Internet Connections) statt TCP.
- Logik und Aufbau der Requests/Responses wie gewohnt.
- Immer verschlüsselt, effizientere TLS Handshakes.
- Wegen UDP kein head-of-line blocking beim Multiplexing.

HTTP Request

Die syntaktische Struktur einer HTTP-Anfrage:



Quelle: Kurose/Ross

Methode steuert die Reaktion und Verarbeitung auf dem Web Server.

URL spezifiziert den Ort einer Ressource.

Version ist `HTTP/1.1` etc.

General Headers beziehen sich auf die gesamte Nachricht.

Request Headers detaillieren die Anfrage, oder schränken sie ein.

Representation Headers beziehen sich auf das Format der Daten im Rumpf.

In echten Nachrichten sind die Headers nicht unbedingt nach Arten sortiert.

Body bei GET i. d. R. leer, bei POST, PUT, PATCH werden hier Daten an den Server übergeben.

HTTP Request: Beispiel

```
GET /index.html HTTP/1.1
Host: example.de
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,
*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Upgrade-Insecure-Requests: 1
```

HTTP Request: Methoden

Methode	Zweck	Sicher	Idempotent	Cachable
GET	Ressourcen anfordern	ja	ja	ja
HEAD	Nur den Header anfordern	ja	ja	ja
PUT	Bestehende Ressource aktualisieren	nein	ja	nein
PATCH	Teilweise Aktualisierung	nein	ja	nein
DELETE	Bestehende Ressource löschen	nein	ja	nein
POST	Neue Ressource anlegen	nein	nein	nein
OPTIONS	Interaktionsmöglichkeiten für eine Ressource abfragen	ja	ja	nein

URI, URL und URN

Uniform Resource Identifier (URI) – Zeichenkette, die eine Ressource im Internet

- mit *Schema* (für Zugriff oder Identifikation) und
- *Identifier* eindeutig identifiziert.

Als Ort: **Uniform Resource Locator (URL)**, mit

- *Zugriffsmethode*, z. B. Protokoll,
- *Host* als Domain oder IP-Adresse,
- *Pfad* auf dem Host, wo die Ressource zu finden ist,
- *optionalen Komponenten*, z. B. Port oder Parameter.

Wo ist die Ressource und *wie* greife ich darauf zu, z. B. `https:example.com/page1.html`, `mailto:jan.stehr@fhdw.de`

Als Name: **Uniform Resource Name (URN)**, mit

- `urn` als Identifikationsschema,
- Namensraum für die Ressourcenart, z. B. *International Standard Book Number, Internet Engineering Task Force RFCs, Object Identifier* etc.,
- dem eindeutigen Namen in diesem Namespace.

Wie ist eine Ressource im Internet *persistent identifiziert*, z. B. `urn:isbn:0-451-45052-3`

Alle URLs und URNs sind URIs, aber nicht alle URIs sind URLs oder URNs.

Für später: URL und Zugriffsmethode in REST

Die heute beliebten und verbreiteten **RESTful APIs** definieren **Endpoints** durch

URLs, die eine Ressource oder Funktion repräsentieren

+ *Methoden, die auf die Ressource ausgeführt werden können*.

Beispiel:

`POST meineapi.beispiel.de/users/1234` – User 1234 anlegen

`GET meineapi.beispiel.de/users/1234` – Daten von User 1234 abfragen

`PUT meineapi.beispiel.de/users/1234` – Daten von User 1234 aktualisieren

`...`

HTTP Header Lines (Auszug)

Header mit Beispiel	Zweck
`Host: beispiel.de:8080`	DNS-Name und Port des Servers.
`User-Agent: Mozilla/5.0...`	Informationen über den anfragenden Client, Browser, Betriebssystem, Gerät.
`Accept: text/html`	Formate, die der Client als Antwort akzeptiert.
`Accept-Encoding: gzip`	Vom Client akzeptierte Komprimierungsmethoden.
`Accept-Language: en-US`	Bevorzugte Sprache der Antwort.
`Connection: keep-alive`	Mitteilung an den Server, ob die Verbindung für weitere Anfragen geöffnet bleiben soll.
`Set-Cookie: sessionId=abc123; Path=/`	Server: Weist den Client an, einen Cookie mit dem gegebenen Wert zu speichern.
`Cookie: sessionId=abc123; someToken=xyz456`	Schickt Cookies in Form von `key=value`-Paaren an den Server. Genutzt für Sitzungsmanagement, Tracking und „User Experience“.
`Content-Type: application/json`	Format der Daten im Body, für POST und PUT Requests.
`Content-Length: 250`	Größe der Daten im Body in Bytes.

Header mit Beispiel	Zweck
`Referer: https://example.com/page1.html`	URL und Pfad einer Seite, auf der der Client ein Request erzeugte, zur Verfolgung der Navigation, z. B. bei Klick auf Link.
`Range: bytes=400-999`	Fordert Teile einer Ressource an, z. B. um einen Download fortzusetzen.
`Cache-Control: no-cache`	Steuerung der Zwischenspeicherung im Browser, als auch z. B. auf Proxies oder CDNs beim Response.
`Upgrade-Insecure-Requests: 1`	Der Client möchte von HTTP auf HTTPS wechseln.
`Origin: https://example.com:8080`	Spezifikation der Herkunft eines Requests mit Schema/Protokoll, Domain und Port für *Cross-Origin Requests*, oft für *Cross-Origin Resource Sharing* (CORS), z. B. API-Zugriffe oder Formular-Daten-POST in einer anderen Domain. Der empfangende Server kann prüfen, ob er von dieser Domain Anfragen akzeptiert.

Präferenzen mit *quality/q-values* gewichten (siehe Beispiel vorne).

Interaktion mit dem Webserver

Wie die Requests aussehen, wissen wir nun. Wie können wir praktisch eine Anfrage an den Server schicken? Viele Optionen, z. B.:

1. Aus einer **eigenen Software**, indem wir die Nachricht wie beschrieben als String kodieren und dann über einen passenden TCP Socket schicken. Oder wir nutzen Bibliotheken, die uns da helfen.
2. Durch Aufruf der Adresse und Port über einen **Browser**. Die meisten Browser bieten auch „Development Tools“, mit denen Sie sich die Request/Response-Nachrichten anschauen können.
3. Über das Tool **Command Line URL** mit `curl -v` (`verbose`, damit wir sehen, was passiert) mit Adresse und Port.
4. Interaktiv über den **Postman API Client** (wenn man dort einen Account anlegen möchte).

Übung 4: Wir bauen uns einen kleinen Webserver

Unsere minimale virtuelle Infrastruktur bauen wir jetzt aus: Wir lassen (die VM) einen Container mit einem Webserver ausführen. Also, wir „deployen“ einen Container auf unserem (virtuellen) Knoten. Legen Sie in einem passenden Verzeichnis die Dateien unten an. Wenn der Webserver läuft, machen Sie ihn ansprechbar z. B. unter `wta.meinedomain.com` (Hinweis in Abschnitt DNS vorne).

webserver.py

```python
from flask import Flask

webserver = Flask(__name__)

@webserver.route('/')
def home():
  return '<h1>Willkommen auf der minimalen WTA-Homepage!</h1>'

if __name__ == '__main__‘:
  webserver.run(host='0.0.0.0', port=8000)
```

In einem Python-Programm setzen wir die Bibliothek *Flask* ein, die uns einen Webserver implementiert.

Über *Decorators* @ können wir sehr einfach Routen auf unserer Website auf Funktionen abbilden, die bei HTTP Request des Pfades ausgeführt werden.

Nutzen Sie gerne auch andere Implementierungen in Go oder Java-/Typescript.

Dockerfile

```dockerfile
FROM python:3.9.20-slim

RUN pip install flask
RUN mkdir /server
ADD webserver.py /server
WORKDIR /server

CMD ["python", "webserver.py"]
```

Wenn direkt als Container, ohne Vagrant/VM (ohne Zeilenumbrüche!):

`docker buildx build –t webserver .`

`docker run –p 8001:8000 webserver`

`curl -v localhost:8001`

Vagrantfile

```vagrant
Vagrant.configure("2") do |config|

  config.vm.box = "ubuntu/bionic64"
  config.vm.provision :docker

  config.vm.network "private_network", ip: "10.0.0.10"

  config.vm.provision "docker" do |d|
    d.build_image "/vagrant", args: "-t webserver"
    d.run "webserver", args: "-p 8001:8000"
  end

end
```

Um komplexere, verteilte Infrastrukturen lokal zu entwickeln und zu testen, bieten sich VMs an, denen wir lokale Adressen zuweisen und in denen wir die Applikationscontainer dann ausführen.

`vagrant up`

`curl -v 10.0.0.10:8001` – was sehen Sie?

Loggen Sie sich wieder in die VM ein und schauen Sie, was dort läuft. Auch mit **`docker ps`**

Wie sieht das entstandene Deployment als Diagramm aus?

Applikationsdaten vom Client an den Server: Query Parameter

Bisher sind die Requests festgelegte Interaktionen zum Zugriff auf Ressourcen des Servers. Wie kann ein Client eigene *Applikations*daten mitschicken?

Query Parameter der URL in GET:

- `?` nach dem Pfad der URL
- `key=value`-Paare
- Getrennt durch &

Z. B.

```
GET /search?query=beispiel&lang=de HTTP/1.1
Host: example.com
```

Dabei beachten:

- Die Parameter sind in der URL sichtbar. Wir können sie also bookmarken, sie landen in der Browser History, sie sollten also – auch bei TLS – keine, oder nur verschlüsselte Secrets enthalten.
- URLs besitzen Längenbeschränkungen. Die sind auf Client und Server implementierungs-abhängig, aber Sie sollten von höchstens 2.048 Zeichen ausgehen.
- GET Requests sind cacheable auf Browser, Proxy und Server, da sie eigentlich idempotent sein sollten.

Applikationsdaten vom Client an den Server: POST Request

Wenn wir größere Datenmengen an den Server schicken möchten, trennen wir die Daten von der URL und nutzen den ***Body* einer POST-Anfrage**.

Das Format der Daten deklarieren wir im `Content-Type`, z. B.:

- `application/x-www-form-urlencoded` – das Format von eben mit `key=value`-Paaren.
- `multipart/form-data` – für Datei-Uploads, auch binäre Daten *as-is*, einzelne Felder durch Boundary Strings markiert.
- `application/json` – Daten im strukturierten JSON-Format.

Z. B.

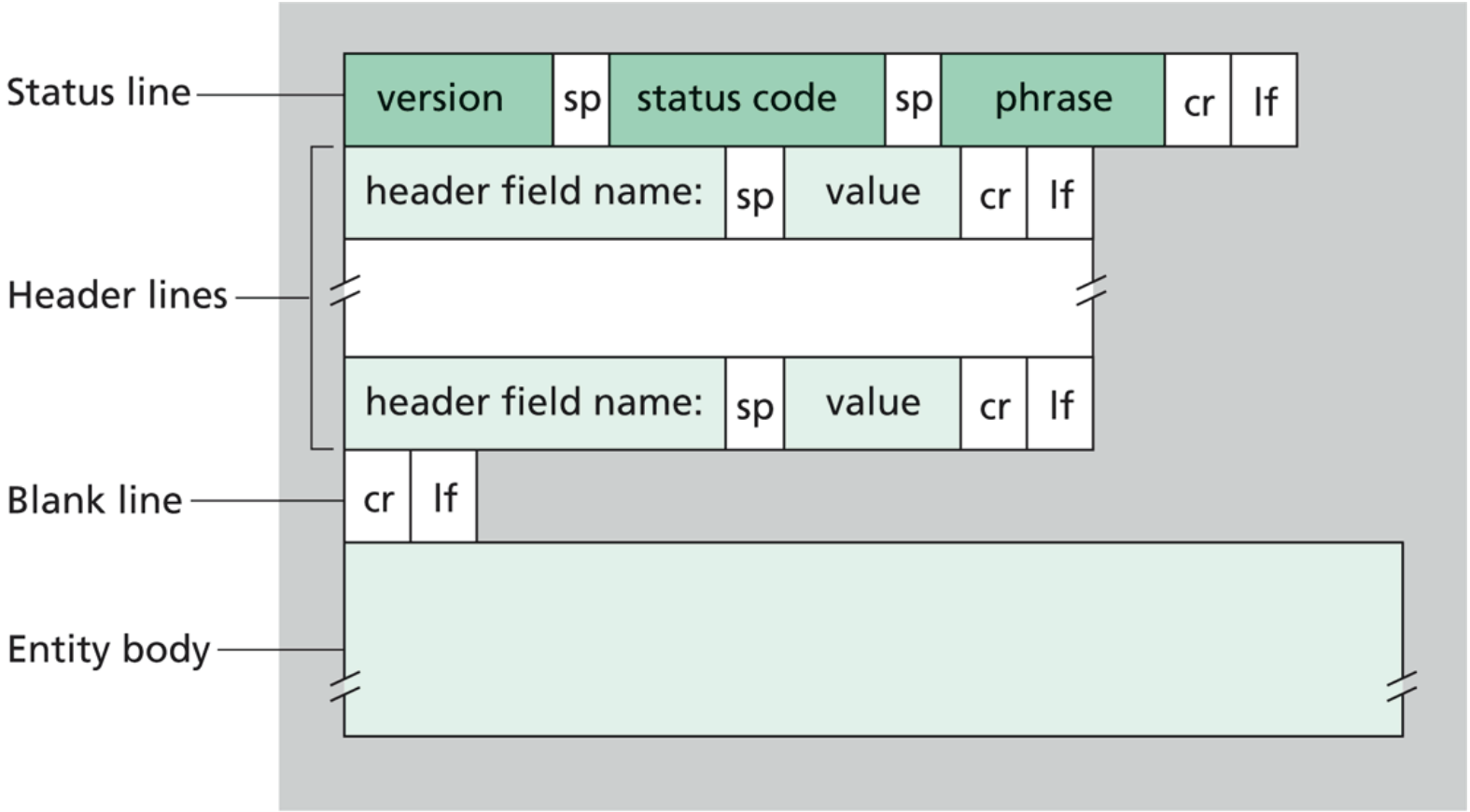
```
POST /login HTTP/1.1
Host: www.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 28

username=jan&password=geheim
```

Dabei beachten:

- I. d. R. keine expliziten Größen-beschränkungen.
- Daten sind in der URL nicht sichtbar.
- Ohne weitere Maßnahmen sind die Daten im Body aber nicht verschlüsselt.
- POST-Anfragen landen grundsätzlich nicht im Cache, da keine Idempotenz angenommen wird, denn es wird eine Ressource angelegt, also der Zustand des Servers verändert.

HTTP Response

Die syntaktische Struktur einer HTTP-Antwort:



Quelle: Kurose/Ross

Version ist `HTTP/1.1` etc. wie vorne.

Status liefert das Ergebnis der Anfrage als Zahl und **Phrase** beschreibt es textuell.

General und **Representation Headers** wie beim Request. **Response Headers** liefern weitere Metadaten zum Antwortstatus, zum Server und Anweisungen an den Client.

Body liefert Daten in unterschiedlichen Formaten wie eben beim POST Request gezeigt: HTML, XML, JSON, Binärdaten...

HTTP Response: Beispiel

```
HTTP/1.1 200 OK
Date: Tue, 10 Sep 2024 15:32:00 GMT
Server: Apache/2.4.41 (Ubuntu)
Last-Modified: Tue, 10 Sep 2024 10:15:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 5678
Connection: keep-alive
Vary: Accept-Encoding
Content-Encoding: gzip

<html>
<head>
  <title>Welcome to Example.de</title>
</head>
<body>
  <h1>Welcome to Example.de!</h1>
  <p>This is a sample HTTP response to demonstrate how a server responds to a GET
request.</p>
  <!-- More HTML content -->
</body>
</html>
```

HTTP Response: Status

Werte-bereich	Typ	Nutzung	Beispiel
1xx	Information	Vorläufige Antwort	`101 Switching Protocols`
2xx	Success	Erfolgreiche Verarbeitung	`200 OK`
3xx	Redirection	Mehr als eine Antwort, weitere Requests nötig	`303 See Other`
4xx	Client Error	Fehler in der Anfrage	`404 Not Found`
5xx	Server Error	Fehler beim Server	`503 Service Unavailable`

Ergänzung: Pushing vom Server

Im klassischen Web geschieht eine Content-Aktualisierung nur auf Anfrage (Pull) des Browsers bzw. Clients:

- Einfach für den Server, aufwändiger für Client und Netzwerk.
- Applikationsabläufe können keine bi-direktionale Kommunikation nutzen.
- Oft viele TCP-Verbindungen zur Statusabfrage.
- Hoher Overhead durch HTTP und seine Header.

Seit ca. 2000 gab es deshalb Ansätze

- nicht mehr die komplette Seite zu laden,
- Nachladen von Komponenten im Hintergrund durchzuführen.

Asynchronous JavaScript and XML (AJAX) z. B. verwendet JS auf dem Client, um Daten per XML/JSON nachzuladen.

Heute sind **WebSockets** oder **gRPC** (gRPC Remote Procedure Calls) die bevorzugten Wege für Pushes vom Server.

Die **WebSockets**

- wollen bestehende Infrastruktur und Transaktionen (Proxies, Authentifizierung etc.) möglichst nutzen, gehen also von einer HTTP-Kommunikation aus,
- nutzen `wss://beispiel.de/socket` als Schema der URL (WebSocket Secure),
- implementieren darin bi-direktionale Kommunikation,
- arbeiten auf der Ebene von Anwendungs-definierten Nachrichten,
- sind damit event-basiert zu programmieren (*on open*, *on message*, ...).

```
GET /chat HTTP/1.1
Host: beispiel.de
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key:
dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept:
s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

Die Content-Formate: JSON, XML, HTML+CSS, JavaScript

Datenaustausch

Textuelle, applikationsspezifische Formate der ausgetauschten Daten sind immer möglich, aber oft ineffizient: Das Format ist nicht allgemein bekannt, individuelle Parser sind nötig.

Daher haben sich zwei Formate (nicht nur) für die Datenübertragung etabliert:

eXtensible Markup Language (XML) für baum-orientierte, attributierte Dokument-Strukturen. Abstraktion von HTML, XML ist eine Metasprache. Klartext.

JavaScript Object Notation (JSON) für *key-value*-Strukturen und Arrays. Klartext.

Für beide Formate gibt es auch binär kodierte Versionen und Möglichkeiten, erlaubte Strukturen zu definieren (*Document Type Definition/DTD, XML Schema Definition/XSD, JSON Schema*).

JSON Syntax

Ein JSON-Datensatz besteht

- aus einem **Objekt** mit einer öffnenden und schließenden geschweiften Klammer, oder
- aus einem **Array** von Werten.

Die Attribute des Objekts sind als Komma-separierte *key-value*-Paare angegeben.

```
{
   key_1: value_1,
   ...
   key_n: value_n
}
```

Ein Schlüssel ist eine Zeichenkette in Anführungszeichen `"..."`.

Werte können

- Strings,
- Zahlen,
- `true` oder `false` (Boolean),
- `null`,
- Objekte,
- Arrays `[value_1,..., value_n]` aus Werten

sein.

D. h. wir können beliebig verschachtelte, komplexe Strukturen mit JSON kodieren, übertragen und speichern.

Beispiel

```xml
<?xml version ="1.0"?>
<Windrad>
    <Instanz>
        <type>Hersteller</type>
        <ort>Koordinaten</ort>
        <sol>2000-11-11</sol>
    </Instanz>
    <wind>
        <strength>54</strength>
        <power>2.07</power>
    </wind>
    <kontakt>
        <phone-no1>tel1</phone-no1>
        <phone-no2>tel2</phone-no2>
        <mail-no1>benutzer@mail.de</mail-no1>
    </kontakt>
</Windrad>
```

```json
{
    "instanz": {
        "type": "Hersteller",
        "ort": "Koordinaten",
        "sol": "2000-11-11"
    },
    "wind": {
        "strength": "54",
        "power": "2.07"
    },
    "kontakt": [
        "tel1",
        "tel2",
        "besitzer@mail.de"
    ]
}
```

JSON vs XML

Gemeinsamkeiten: Beide sind

- von Menschen lesbar,
- hierarchisch,
- maschinenlesbar und nativ oder per Bibliotheken unterstützt von vielen Programmiersprachen.

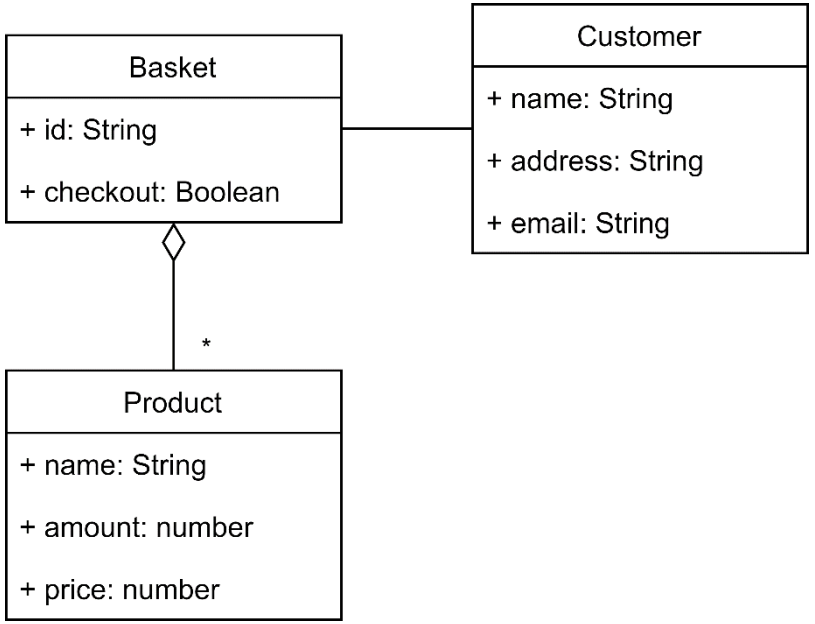
Unterschiede

- JSON ist kompakter.
- JSON ist effizienter zu schreiben und lesen.
- JSON unterstützt Arrays.
- XML lässt Kommentare zu.
- XML besitzt als Metasprache viele Möglichkeiten der Spezialisierung, z. B. Attributierung der Elemente.

Vorteile JSON gegenüber XML

- JSON ist einfacher zu parsen.
- JSON kann in einigen Sprachen nativ verarbeitet werden, d. h. direkte Operationen auf Objekte als Dictionaries und Arrays.

Übung 5: JSON



Legen Sie zu der Warenkorb-Klassen-Struktur links eine beispielhafte JSON-Struktur an (einfach als Text, oder in einer Programmiersprache Ihrer Wahl).

Alternative 1: Ein einzelner Warenkorb.

Alternative 2: Eine Struktur mit mehreren Warenkörben, auf die man über ihre ID zugreifen kann.

Hypertext Markup Language (HTML)

HTML ist eine Sprache zur Beschreibung von Dokumenten wie Web-Seiten:

- Das besteht aus Text und syntaktisch unterscheidbaren Annotationen.
  - Primär zur Beschreibung der Semantik, z. B. `<p>`
  - Teilweise auch Darstellung, z. B. `<b>`
  - Damit Gegenteil von WYSIWYG, hier „Du wolltest es so, hier hast Du es!“
- Seit 1993 öffentlich verfügbar, seit 2000 und HTML 4.01 internationaler Standard, seit 2014 HTML5 als W3C Standard.
- Kennt u. a. die folgenden Elemente:
  - Document Type Definition
  - Markups
  - Attribute
  - Character and Entity references

HTML und Web: Etablierte Begriffe

Webpage

- Einzelne Seite `*.html`
- CSS für Layout
- JavaScript für Verarbeitungslogik

Website

- Sammlung von Seiten
- Unter Domain zusammengefasst
- Thematischer Kontext
- Homogenes Layout und Styling
- Konsistente Navigation

Web App

- Bietet einen oder mehrere Services auf Endgeräten an
- Keine Installation nötig, vom Webserver ausgeliefert
- Look-and-feel wie Desktop / App
- Läuft in einem Browser, der aber in nativer App integriert sein kann
- Hoher Grad an Interaktion, dynamische Updates
- Ggf. größere Datenmengen
- *Progressive* Web Apps arbeiten auch offline mit Service Worker (Caching)

Webserver

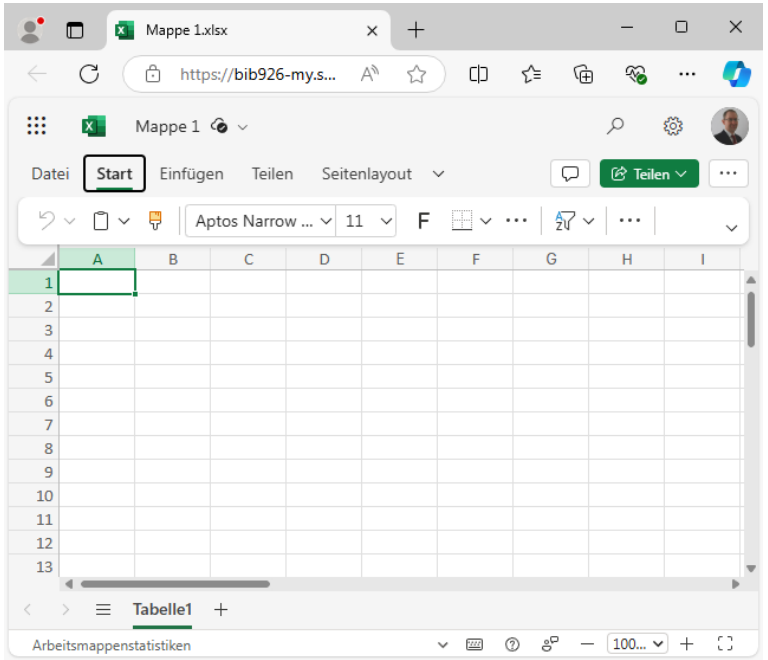
- Hosting einer Website
- Repräsentiert einen damit verbundenen Service
- Nimmt Requests entgegen und erstellt Responses

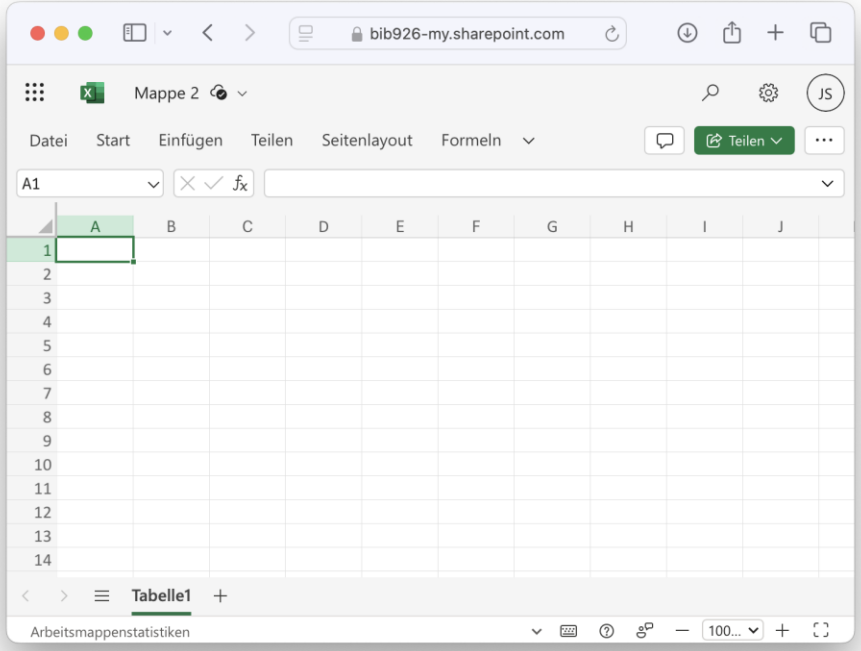
Plattformunabhängigkeit und Interoperabilität

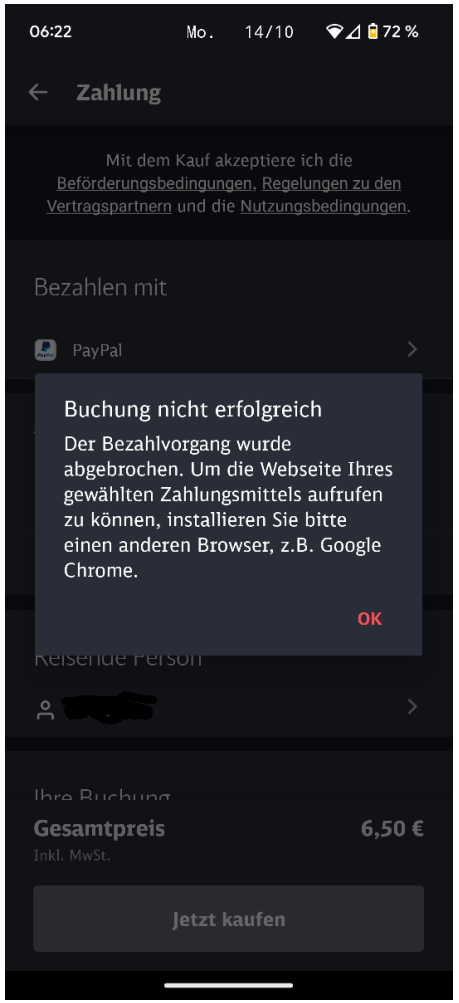
Wichtige Konzepte seit der ersten Stunde des Web:

Inhalte, Strukturen, Anwendungen sollen

- **plattformunabhängig** sein – also auf allen Rechnern und Betriebssystemen laufen, möglichst in ähnlicher Form und ähnlichem Umfang,
- **interoperabel** sein – über die Plattformen hinweg verteilt miteinander arbeiten können.







HTML: Beispiel

Kommentar

XML DTD

Meta-Information

Character Entity Reference

Elements/Tags

Head

Body

Attribute

```
<!-- File: beispiel.html -->
<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8">
        <title>Willi N&#x00dc;&szlig;er sagt:</title>
        <link type="text/css" rel="stylesheet" href="style.css">
    </head>

    <body>
        <p>Hello World!</p>
        <a href="https://fhdw.de">Ein Link!</a>
        <input type="checkbox" />
    </body>
</html>
```

HTML5

Die aktuelle Version HTML5 ist gekennzeichnet durch

- mehr Tags und Tags mit stärkerer Semantik, z. B. `<article>`
- explizitem Multimedia-Support
  - durch Tags wie `<video>` und `<audio>`

```
<video width="320" height="240" controls>
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.ogg" type="video/ogg">
</video>
```

  - `<canvas>` für Zeichnen im Browser via WebGL (= OpenGL ES),
- Schnittstellen, z. B. zum DOM-Zugriff,
- neue APIs für Zugriffe auf Client (Storage, Offline, Sensoren) und Kommunikation (WebSockets).

Beispiele für HTML-Elemente

Tag Name	Beschreibung
`a`	Anchor – Verlinkung z. B. auf andere Websites
`b`	Umschließt fett darzustellenden Text, Inline-Element
`br`	Zeilenumbruch, ohne schließendes Tag
`button`	GUI-Element, unterscheidet sich je *type*-Attribut im Aussehen
`div`	Umschließt einen Bereich, wird zum Styling genutzt
`form`	Sammlung von Eingabefeldern, Abgabe mit `type submit`
`h1 - h6`	Überschriften, groß nach klein
`iframe`	Einbinden anderer Web-Seiten
`img`	Einbinden von Bildern
`input`	Eingabefeld, unterschiedlich dargestellt mit *type*
`ol, ul, li`	(Geordnete) Liste und Listenelemente
`meta`	Informationen über den Inhalt und die Daten.
`p`	Textparagraph, umschließt Text und endet mit Zeilenumbruch

Gesamte Tag-Liste unter `w3schools.com/tags`

Attribute von HTML-Elementen

HTML-Elemente haben Attribute. Diese können in drei Kategorien eingeordnet werden:

Globale Attribute

Können in allen Elementen genutzt werden. Beispiele:

`id` – eindeutiger Identifier

`class` – für einen oder mehrere Klassennamen, durch Leerzeichen getrennt.

`style` – für CSS Styles, welche direkt „inline“ angewendet werden.

Interaktionsattribute

Für alle sichtbaren Elemente. Beispiele:

`onmouseover` – führt Skript beim Mausüberfliegen aus.

`onkeyup` – führt Skript beim Loslassen einer Taste aus.

Eigene Attribute

Werden nur von einigen Elementen unterstützt. Beispiele:

`input` → `maxlength` – maximale Anzahl an erlaubten Zeichen im Eingabefeld.

`img, iframe` → `src` – Quelle des einzubindenden Inhalts, z. B. Bild oder Webseite.

`button, input` → `type` – steuert den Typ und damit das Aussehen des Elements.

Übung 6: Ein Webserver schickt HTML-Seiten

Unseren Webserver gestalten wir nun etwas um: Er soll uns statische HTML-Seiten aus Verzeichnissen schicken.

webserver.py

```python
from flask import Flask, send_from_directory

webserver = Flask(__name__)

@webserver.route('/')
def home():
    return send_from_directory('root-page',
'index.html')

@webserver.route('/<path:path>')
def static_pages(path):
    return send_from_directory('root-page',
path)

if __name__ == '__main__':
    webserver.run(host='0.0.0.0', port=8000)
```

Dockerfile

```dockerfile
FROM python:3.9.20-slim

RUN pip install flask
RUN mkdir /server
COPY ./webserver.py /server
COPY ./root-page /server/root-page
WORKDIR /server

CMD ["python", "webserver.py"]
```

Vagrantfile

```vagrant
Vagrant.configure("2") do |config|

  config.vm.box = "ubuntu/bionic64"
  config.vm.provision :docker

  config.vm.network "private_network", ip: "10.0.0.10"
  config.vm.provision "shell", inline: <<-SHELL
    mkdir ./root-page
    cp -r /vagrant/root-page/* ./root-page/
  SHELL

  config.vm.provision "docker" do |d|
    d.build_image "/vagrant", args: "-t webserver"
    d.run "webserver", args: "-p 8001:8000"
  end

end
```

/root-page/index.html

```html
<!DOCTYPE html>
<html lang="de">
<head>
    <title>Ein Webserver im Container</title>
</head>
<body>
    <h1>Hallo sagt der Python Flask
Webserver!</h1>
    <p>Das ist eine einfache, statische HTML-
Seite, geschickt von Flask in einem
Container.</p>
</body>
</html>
```

/root-page/pages/page1.html

```html
<!DOCTYPE html>
<html lang="de">
<head>
    <title>Weitere Seite</title>
</head>
<body>
    <h1>Eine weitere Beispielseite in
HTML</h1>
    <p>Das ist noch eine einfache,
statische HTML-Seite, geschickt von
Flask in einem Container.</p>
</body>
</html>
```

In Ihrem Arbeitsverzeichnis von Vagrant oder Docker brauchen Sie hier nun ein Unterverzeichnis `/root-page` mit der Struktur von HTML-Seiten für den Webserver.

Document Object Model (DOM)

Die logische Struktur vieler Dokumente kann als Graph dargestellt werden:

Document Object Model (DOM)

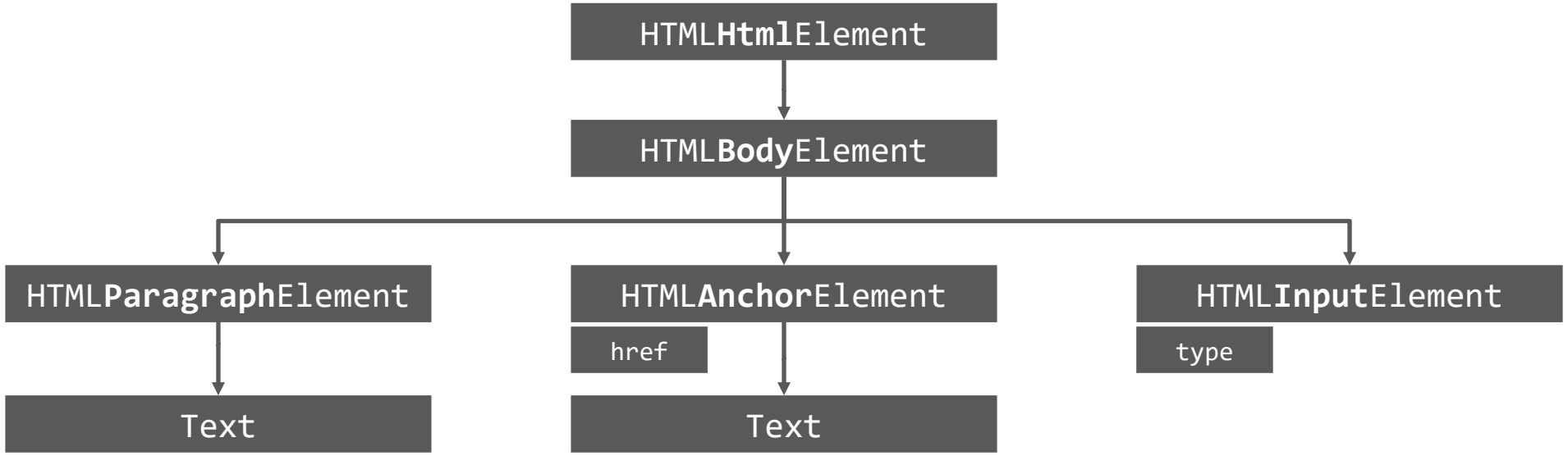
The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document. Quelle: W3C

Es gilt:

- HTML- (und XML-) Dokumente sind sogar Bäume und damit Spezialfälle.
- Das DOM wird von einer Anwendung aus HTML generiert (*Parsing*). Es erlaubt dann die programmatische, also dynamische Anpassung z. B. von Webseiten.
- Wesentlich dafür ist die
  - Auswahl (*selection*) von DOM-Elementen,
  - ggf. nachfolgende Manipulation.
- Dies geschieht über DOM APIs, z. B. in JavaScript.

DOM visualisiert

```
<!DOCTYPE html>
<html>
   <body>
       <p>Hello World!</p>
       <a href="https://fhdw.de">Ein Link!</a>
       <input type="checkbox" />
   </body>
</html>
```



https://developer.mozilla.org/en-US/docs/Web/API/HTML_DOM_API

Cascading Style Sheets (CSS)

HTML gibt – als DOM – primär die **Struktur und Inhalte** des Dokuments an. Die **Darstellung**, Styling und Layout, der Dokument-Teile kann durch **Cascading Style Sheets (CSS)** definiert werden.

CSS

- sind *regel-basiert* `<selector> { property: value; ... }`
- Der `selector` identifiziert die Komponente durch *Element*-, *Klassenname*, *Attribut* oder *ID*, zusätzlich z. B. auch *Laufzeitzustand* wie `:hover`
- Die Eigenschaften sind meist komponenten-spezifisch.
- Wenn mehrere Styles für eine Komponente gelten: Wähle die spezifischste („cascading“).
- Reihenfolge kann umgangen werden (`!important`).

CSS: Einfache Selektoren

Selektor	Beispiel CSS	Beispiel HTML
Element	`h1`	`<h1> ... </h1>`
Klasse (alle Elemente)	`.myClass`	`<div class=„myClass“>...`
Klasse (ein Element)	`p.myClass`	`<p class=„myClass“>...`
Element mit mehreren Klassen	`p.myClass1 ... p.MyClass2`	`<p class=„myClass1 myClass2“>...`
ID	`#myId`	`<div id=„myId“>...`
Attribut	`[href]`	`<a href=„...“>...`
Universell	`*`	`...`

Selektoren ausführlich unter `w3schools.com/css`

CSS: Beispiel

```html
<!-- File: beispiel2_styled.html -->
<!DOCTYPE html>
<html>
    <head>
        <title>WTA Example</title>
        <link type="text/css" rel="stylesheet" href="my_style.css">
    </head>

    <body>
        <h1>Web Technologies and Applications Example</h1>
        <p>Hello World!</p>
        <a class="myLink" href="https://fhdw.de">Mein spezieller Link.</a>
        <br>
        <a href="https://fhdw.de">Unveränderter Link.</a>
        <p class="anders">Hello Galaxy!</p>
    </body>
</html>
```

```css
/* File: my_style.css */
body {
    background-color: lightgray;
}
h1 {
    font-family: sans-serif;
    color: blue;
}
.anders {
    color: red;
    font-weight: bold;
    font-size: x-large;
}
a.myLink {
    font-family: monospace;
    color: black;
}
```

Web Technologies and Applications Example
Hello World!
Mein spezieller Link.
Unveränderter Link.
Hello Galaxy!

Web Technologies and Applications Example
Hello World!
Mein spezieller Link.
Unveränderter Link.
Hello Galaxy!

Schauen Sie sich das mit den Entwicklungs-Tools in einem Browser an.

Responsive Web Design

https://www.vecteezy.com/free-vector/responsive-web-design

CSS können wir nutzen, um eine Website flexibel auf unterschiedliche Geräte bzw. Displays anzupassen:

Responsive Web Design – eine Website adaptiert ihr Layout an unterschiedliche Endgeräte.

(*Reactive Web Design* – es gibt verschiedene, maßgeschneiderte Versionen einer Website für die unterschiedlichen Endgeräte)

CSS Media Queries

```
@media media-type and (media-feature-rule) {
    /* CSS rules... */
}
```

```
@media screen and (max-width: 600px) {
    .container {
        padding: 10px;
    }

    form {
        flex-direction: column;
    }
}
```

Layout mit CSS

Positionierung der Elemente auf einer Webpage – Problem: Jeder Browser rendert ein wenig anders.

- Absolut / relativ zu anderen Elementen
- Einstellbar über `display` und `position`

Normal Flow (Standard)

```
<h1>Überschrift</h1>
<p>Absatz mit Text.</p>
<ul>
    <li>Punkt 1</li>
    <li>Punkt 2</li>
    <li>Punkt 3</li>
</ul>
```

Flexbox

- Flexible Anordnung von Items
- Container `display: flex`
- Items innerhalb des Containers

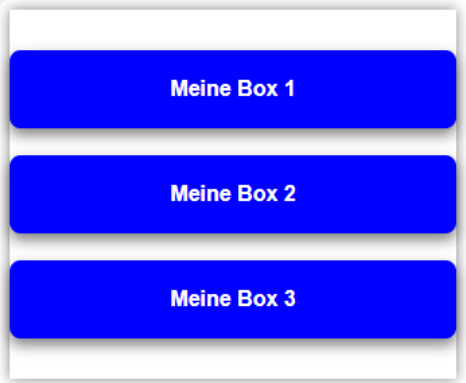
Grid

- Layout basierend auf Zeilen und Spalten
- Container `display: grid`
- Anzahl der Spalten sowie deren Breite: `grid-template-columns`
- Anzahl der Zeilen sowie deren Breite: `grid-template-rows`

Beispiel Flexbox

```css
.container {
    display: flex;
    font-family: sans-serif;
    justify-content: space-around;
    align-items: center;
    width: 80%;
    background-color: white;
    padding: 20px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.5);
}
.box {
    background-color: blue;
    font-weight: bold;
    color: white;
    padding: 20px;
    margin: 10px;
    width: 30%;
    text-align: center;
    border-radius: 8px;
    box-shadow: 0 5px 10px rgba(0, 0, 0, 0.5);
}
@media (max-width: 768px) {
    .container {
        flex-direction: column;
    }
    .box {
        width: 100%;
    }
}
```

```html
<!DOCTYPE html>
<html>
    <head>
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Flexbox Example</title>
        <link type="text/css" rel="stylesheet" href="flexbox_style.css">
    </head>
    <body>
        <div class="container">
            <div class="box">Meine Box 1</div>
            <div class="box">Meine Box 2</div>
            <div class="box">Meine Box 3</div>
        </div>
    </body>
</html>
```



Beispiel Grid

```css
/* Flexbox außenrum */
body {
    font-family: sans-serif;
    margin: 0;
    padding: 0;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    background-color: grey;
}
/* Grid Container */
.container {
    display: grid;
    grid-template-columns: 2fr 1fr; /* Linke Spalte ist doppelt so groß */
    grid-template-rows: 1fr 1fr;    /* fr = fractional unit */
    gap: 20px;
    width: 80%;
    background-color: white;
    padding: 20px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.5);
}
/* Grid Items (Boxes) */
.box {
    background-color: blue;
    color: white;
    padding: 20px;
    text-align: center;
    border-radius: 8px;
    box-shadow: 0 5px 10px rgba(0, 0, 0, 0.5);
}
.box1 {
    grid-column: 1 / 2; /* in der linken Spalte */
    grid-row: 1 / 3; /* über zwei Zeilen */
}
.box2 {
    grid-column: 2 / 3; /* "Start" / "Ende vor" */
    grid-row: 1 / 2;
}
.box3 {
    grid-column: 2 / 3;
    grid-row: 2 / 3;
}
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Grid Layout with Columns</title>
    <link rel="stylesheet" href="grid_style.css">
</head>
<body>
    <div class="container">
        <div class="box box1">Box 1</div>
        <div class="box box2">Box 2</div>
        <div class="box box3">Box 3</div>
    </div>
</body>
</html>
```

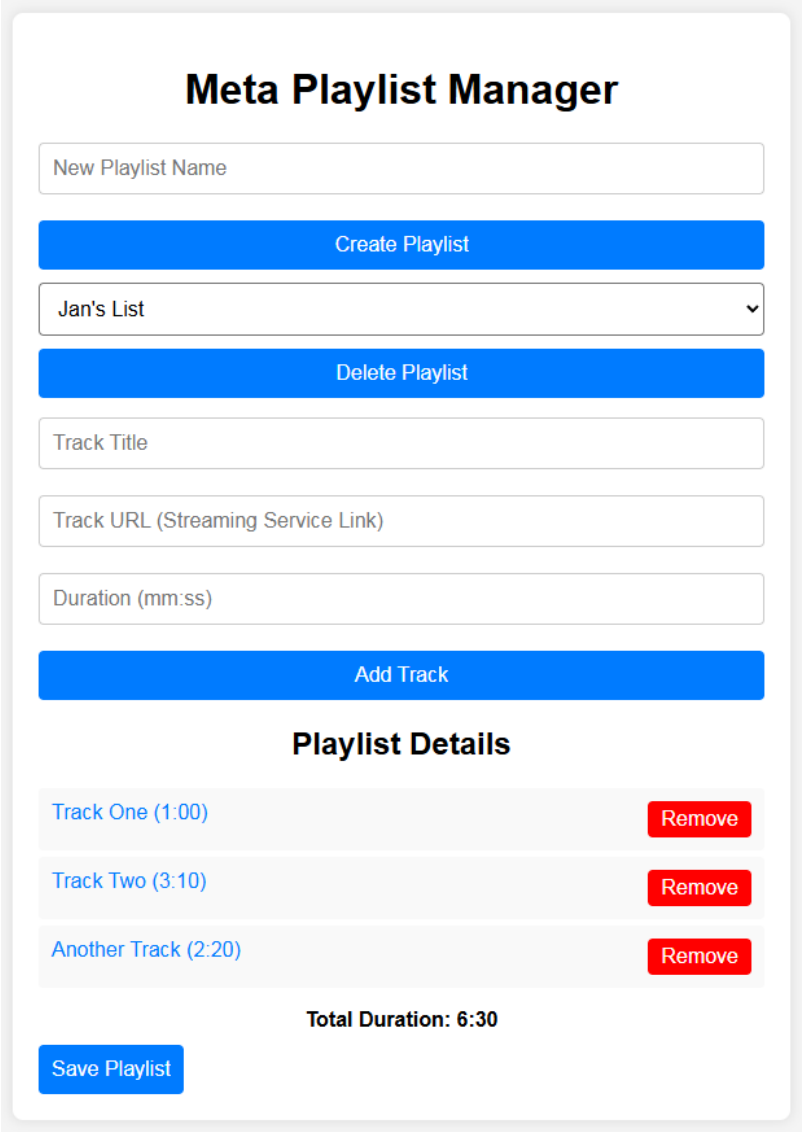
Übung 7: Eine Oberfläche für eine responsive Web App

Implementieren Sie ein **User Interface** für eine **Meta Playlist App**, mit der wir Musik von mehreren Streaming-Diensten verwalten können.

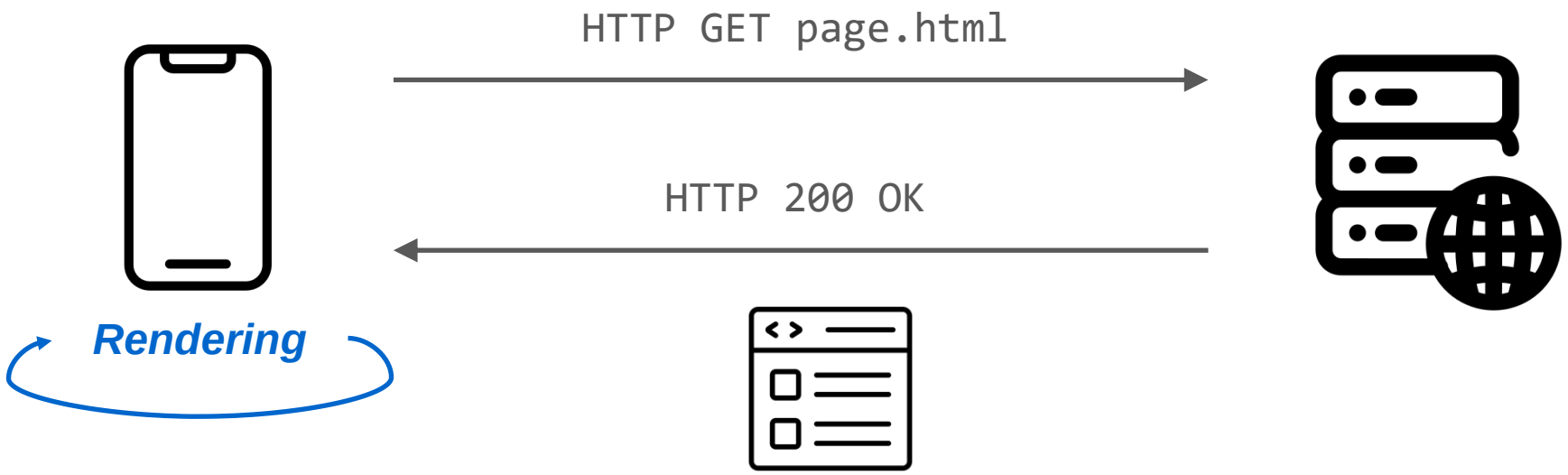
1. Implementieren Sie die notwendigen **Elemente** mit **HTML**.
2. Implementieren Sie ein passendes **Layout** mit **CSS**. Es soll responsiv sein, sich also auf Desktop und Mobilgeräten anpassen.

Sie können das UI selbst schreiben, auf eine Gruppe aufteilen, und/oder einen AI-Assistenten nutzen.

Im Job und in der Klausur müssen Sie aber wissen, wie das Produkt funktioniert, es erklären und Modifikationen beschreiben können.



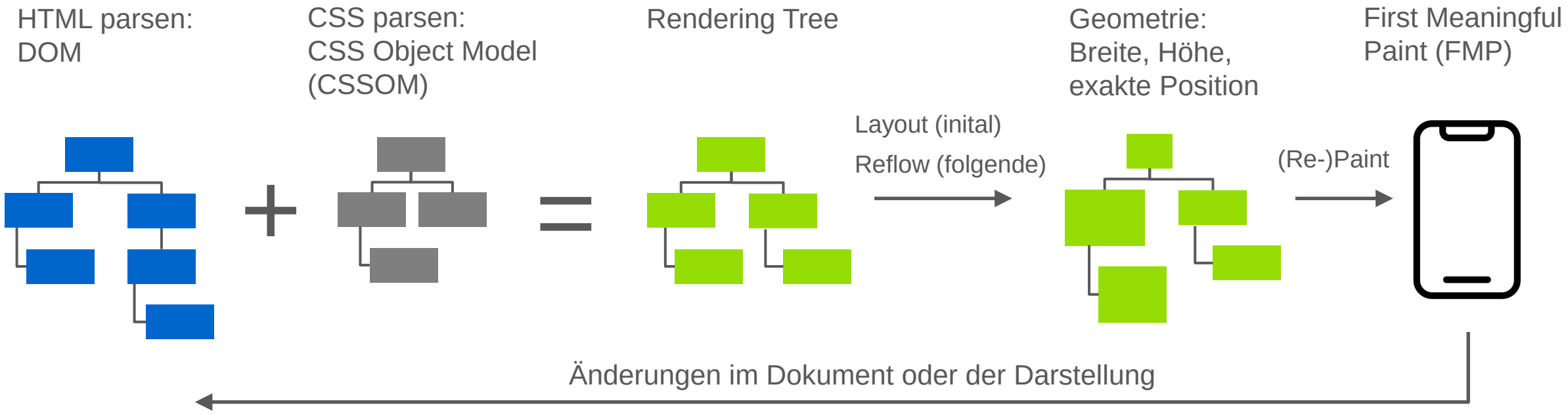
Rendering



Web-Seite im Browser anzeigen: **Critical Rendering Path**

- Vom Empfangen der HTML-Seite bis zur Ausgabe auf dem Screen.
- Nach Veränderung des DOM oder CSSOM – also Content oder Darstellung.

Rendering Tree, Layout/Reflow, Paint



Layout, *Reflow* und *Paint* sind rechenintensive Operationen, die sich stark auf die Performance einer Seite auswirken.

JavaScript: Einführung

Bisher haben wir Inhalte im Web *kodieren*, *strukturieren* und *darstellen* können. Was uns noch fehlt, ist *Verhalten* und *Interaktivität*.

Für die Entwicklung im Web ist **JavaScript (JS)** eine der wichtigsten Sprachen:

- Ab 1995 für den Mozilla-Vorläufer Netscape entwickelt, standardisiert als *ECMAScript*. Syntax ähnlich C, hat trotz des Namens aber nicht viel mit Java zu tun.
- Interpretierte Sprache mit diversen Laufzeitumgebungen und flexibler JIT Compilation, die Änderungen während der Laufzeit berücksichtigt.
  - Im Browser in einer Sandbox.
  - Auf dem Server mit vollem Ressourcenzugriff, z. B. mit *node.js.*
- Multiparadigmatisch, dynamisch typisiert, Garbage Collection, schlanker Sprachkern, erweitert durch Bibliotheken und Frameworks.
- Großer Erfolg und Schwächen von JS haben zu kompatiblen Alternativen geführt, z. B. dem stark typisierten *TypeScript* von Microsoft.

JavaScript: Wichtige Merkmale

Integration mit HTML und CSS

JS kann auf DOM und CSSOM zugreifen und sie lokal verändern, d. h. die Seite muss nicht neu vom Server geladen werden.

Funktionen

Wiederverwendbaren Code können wir als Funktionen implementieren, die einen Namensraum für Variablen bilden. Funktionen sind in JS Objekte erster Klasse.

Frameworks und Bibliotheken

JS besitzt ein sehr umfassendes Ökosystem von Erweiterungen:

- Frameworks für Frontends, z. B. *Angular* und *React*.
- Effizientere Entwicklung, z. B. *jQuery*.
- Serverseitige Ausführung, z. B. *node.js*.
- 3D-Grafik und Spiele, z. B. *Babylon.js*, *Three.js*.
- U. v. m.

Event-driven Programming

Wir können `EventListener` definieren, die uns Mausklicks, Scrollen etc. der HTML-Elemente melden und dazu Code schreiben, der auf die Ereignisse reagiert.

Asynchrone Programmierung

Unser JS läuft als *einzelner Thread*, aber wir können bestimmte Operationen nebenläufig ausführen lassen, z. B. Daten aus einer externen Quelle lesen, damit unsere App dabei nicht blockiert.

JavaScript: Beispiel

```html
<!DOCTYPE html>
<html>
    <head>
        <title>Ein kleines Beispiel zu JavaScript</title>
    </head>
    <body>
        <h1 id="greeting">Hallo Welt!</h1>
        <button id="greetButton">Klick mich!</button>

        <script src="bsp.js"></script>
    </body>
</html>
```

```javascript
// bsp.js

// Zugriff auf das DOM:
const button = document.getElementById("greetButton");

// Funktion zur Reaktion auf einen Klick registrieren:
button.addEventListener("click", function() {
    // Zugriff und Veränderung des DOM:
    document.getElementById("greeting").innerHTML = "Klick erfolgt!";
    button.textContent = "Idempotent.";
    // innerHTML und textContent sind
    // von HTMLElement geerbte Eigenschaften.
});
```

Hallo Welt!

Klick mich!

Klick erfolgt!

Idempotent.

Siehe vorne: Bei Änderung des DOM wird die Seite automatisch neu gerendert.

JavaScript: Cheat Sheet

Grundlegende Syntax

```
// Variablen
let x = 10;       // Gültig in dem Block, in dem sie definiert ist
const y = 20;     // Konstante

// Datentypen
const str = "Hello";      // String
const num = 42;           // Number
const bool = true;        // Boolean
const arr = [1, 2, 3];    // Array
const obj = { name: "Alice", age: 25 }; // Object, siehe JSON

// Funktionen
function add(a, b) {
  return a + b;
}

const multiply = (a, b) => a * b;
```

Kontrollstrukturen

```
// Bedingungen
if (x > 5) {
    console.log("x ist größer als 5");
  } else if (x === 5) {
    console.log("x ist 5");
  } else {
    console.log("x ist kleiner als 5");
  }

// Schleifen
for (let i = 0; i < 5; i++) {
  console.log(i);
}
let i = 0;
while (i < 5) {
  console.log(i);
  i++;
}
for (const num of [1, 2, 3]) {
  console.log(num);
}
```

Fehlerbehandlung

```
try {
    // Hier könnte ein Fehler auftreten:
    let result = riskyFunction();
  } catch (error) {
    console.error("Fehler:", error.message);
  } finally {
    console.log("Das läuft.");
  }
```

Umgang mit JSON

```
// JavaScript-Objekt nach JSON umwandeln
const person = { name: "Alice", age: 25 };
const json = JSON.stringify(person);

// JSON nach JavaScript-Objekt umwandeln
const parsedPerson = JSON.parse(json);
```

Speicher

```
// Lokaler Speicher
localStorage.setItem("name", "Alice"); // Save item
const name = localStorage.getItem("name"); // Retrieve item
localStorage.removeItem("name");       // Remove item

// Sitzungsspeicher (gleiche Funktionen)
sessionStorage.setItem("sessionKey", "value");
```

Gängige Dialoge

```
alert("Dies ist ein wichtiger Hinweis!");
const confirmed = confirm("Bist Du sicher?"); // true oder false
const name = prompt("Gib Deinen Namen ein:"); // Eingabe oder null
```

Debugging

```
console.log("Hallo Welt!"); // Ausgabe auf Kommandozeile
console.error("Fehler!");   // Fehlermeldung
console.warn("Warnung!");   // Warnmeldung

debugger; // Pause, wenn Entwicklungstools offen
```

Ereignisverarbeitung

```
const button = document.getElementById("buttonId");
button.addEventListener("click", () => {
  console.log("Button geklickt!");
});

Gängige Ereignisse
click      Mausklick
mouseover  Zeiger über Element
mouseout   Zeiger verlässt Element
keyup      Taste losgelassen
submit     Webformular abgeschickt
```

DOM-Elemente auswählen

```
// Auswahl nach ID:
const el = document.getElementById("id");
// Auswahl nach Klasse:
const elements = document.getElementsByClassName("class");
// Auswahl nach Tag:
const items = document.getElementsByTagName("tag");
// Einzelnes Element nach CSS selector:
const elQuery = document.querySelector(".class");
// Alle Elemente nach CSS selector
const elAll = document.querySelectorAll(".class");
```

DOM-Elemente bearbeiten

```
el.textContent = "Neuer Text";          // Textinhalt setzen
el.innerHTML = "<b>Fetter Text</b>";    // HTML-Inhalt setzen
el.setAttribute("href", "https://beispiel.de"); // Attribut setzen
el.removeAttribute("href");           // Attribut entfernen
el.style.color = "blue";              // Stil ändern
const newEl = document.createElement("div"); // Neues Element
document.body.appendChild(newEl);            // Element hinzufügen
document.body.removeChild(newEl);            // Element entfernen
```

JavaScript: Client versus Server

Aspekt	Client-side JavaScript	Server-side JavaScript
Verwendung	User Interfaces, interaktive Anwendungen, Präsentation von Content. Daten über `fetch` vom Server.	Komplexe Datenverarbeitung, -auswertung, Management von Diensten und Datenbanken, Middlewares. Events dann Anfragen, Verbindungsstati, Empfang von und Änderungen an Daten.
Laufzeitumgebung	JS Engine im Browser	JS Engine auf Server
Systemzugriff	Beschränkt auf Sandbox, keine direkten Zugriffe auf OS, nur mit Erlaubnis	Alle Ressourcen (Dateisystem, Netzwerk, etc.)
Typische Sicherheitsfragen	XSS-Angriffe (Cross-site Scripting, Injektion von Schadcode)	Netzwerkbasierte Angriffe, Injection, Request Forgery
Skalierbarkeit	Begrenzt durch Endgerät	Hoch, da z. B. in der Cloud flexibel
Datenspeicherung	Lokaler Browser-Speicher, oder extern	Datenbankverbindungen und -operationen
Typische Frameworks	React, Angular, Vue	Express, Koa, NestJS

Übung 8: Das Verhalten unserer responsiven Web App

Erweitern Sie die Meta Playlist App mit JavaScript:

1. Fügen Sie *EventListener* für Funktionen hinzu, die auf die einzelnen Buttons reagieren.
2. Implementieren Sie diese *Funktionen* zum *Anlegen* und *Löschen* einer Playlist, *Hinzufügen* und *Löschen von Songs*, *Berechnung der Gesamtspielzeit*.
3. Verwalten Sie die Playlists intern *in Form von JSON*.
4. *Speichern* Sie die JSON-Struktur *lokal*.

Zwischenstand: Wo sind wir jetzt?

Web Content
Inhalte im Web

Form HTML, JSON, XML	**Verteilung** DNS → IP, Webserver
Darstellung CSS	**Adressierung** URI, URL, URN
Verhalten JS und Weiterentwicklungen	**Datenaustausch** HTTP, WebSockets

Gewohnt sind wir im Alltag Web-Produkte für Development, Office, Gaming, Streaming...
– komplexe, *verteilte* Anwendungen aus vielen Komponenten und Schnittstellen.

Wie nutzen wir also die Bausteine oben in einem größeren Systemkontext?

Architektur und Implementierung von Web-Anwendungen und -Diensten

Begriffe

Bibliotheken

Sammlung von wiederverwendbaren Software-Bausteinen mit einem spezifischen Zweck.

Bsp.: GUI-Bibliotheken wie Qt, wissenschaftliche Bibliotheken wie numpy, scipy etc.

Middleware

Schicht zwischen (verteilter) Anwendung und Low-Level-Schnittstellen und Diensten.

- Stellt vielfältige Funktionen bereit, z. B. für vereinfachte Netzwerk-Kommunikation.
- Beschreibt die mögliche Applikationsarchitektur nur wenig.

Bsp.: Message-Bus- oder RPC-Middleware.

Framework

(Konsistente) Sammlung von Architektur-Vorgaben und Bibliotheken für einen Anwendungstyp.

- Technisches Modell eines Typs wiederverwendbarer Anwendungen.
- Starke Architektur-Vorgaben.
- Verwendet oft *Inversion of Control* (IoC).

Bsp.: Angular, React etc. für Web-Anwendungen.

Konzept: Inversion of Control (IoC)

Konventionell bestimmt und steuert eine Komponente, was sie braucht:

- Kontrollfluss,
- Art und Ort von Ressourcen,
- Lebenszyklen von Ressourcen.

Bsp.: Web-Seiten, Datenbanktabellen.

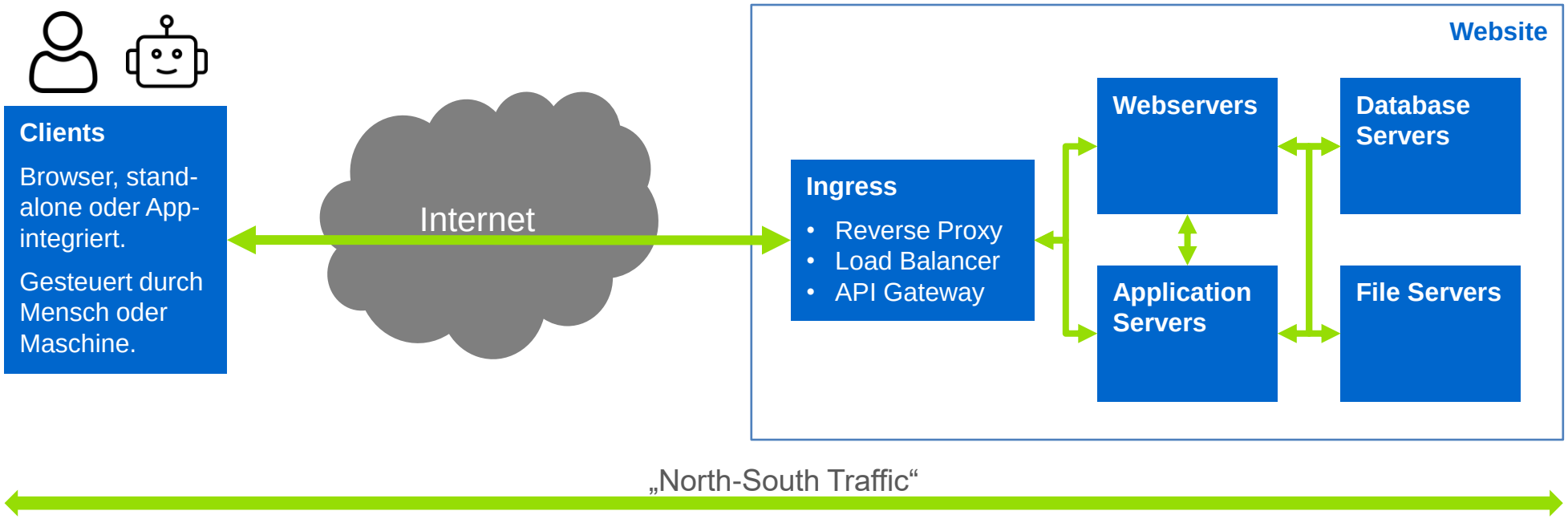
Bei der Nutzung eines Frameworks gibt es der Anwendung vor, was sie bekommt:

- Kontrollfluss,
- die verfügbaren Ressourcen.

Anwendungen in einem Framework werden z. B. durch dessen Events gesteuert.

IoC ist ein prominentes Merkmal vieler Frameworks.

Verteilte Komponenten von Web-Anwendungen



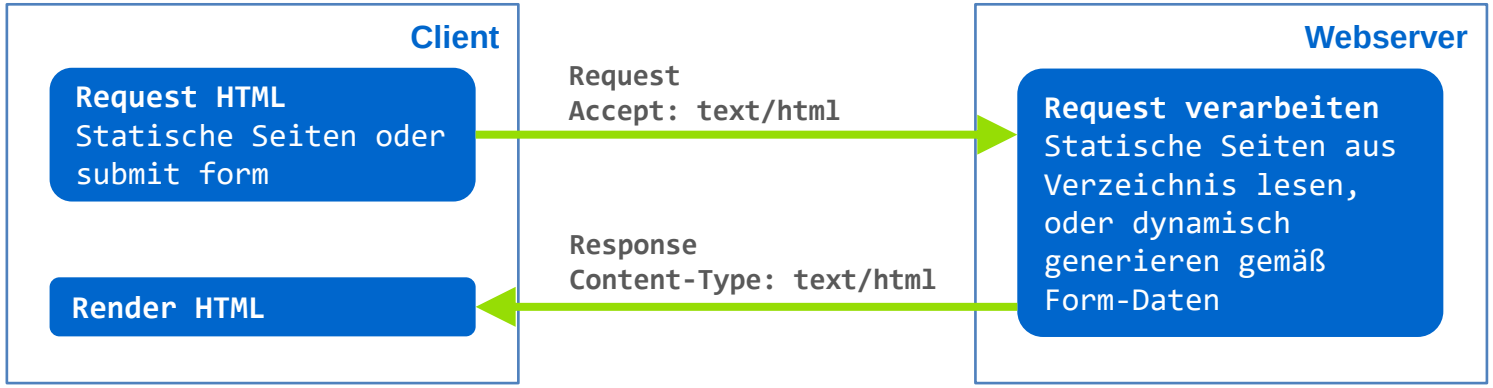
	Reverse Proxy	Load Balancer	API Gateway
Primäre Aufgabe	Stellvertreter für Backend-Systeme, Forwarding von Anfragen	(Last-)Verteilung von Anfragen auf unterschiedliche Server	Management und Routing von Anfragen auf unterschiedliche Dienste
Übliche Funktionen	Caching, SSL Termination, Anonymisierung und Sicherheit, Filterung von Traffic	Monitoring und Health Checks, Skalierung, Gewährleistung von Verfügbarkeit	Authentisierung und Autorisierung, Konvertierung von Formaten, Monitoring und Zugriffsbegrenzung
Schicht	Applikation L5	Transport L4, Applikation L5	Applikation L5

Routing (im Kontext hier)

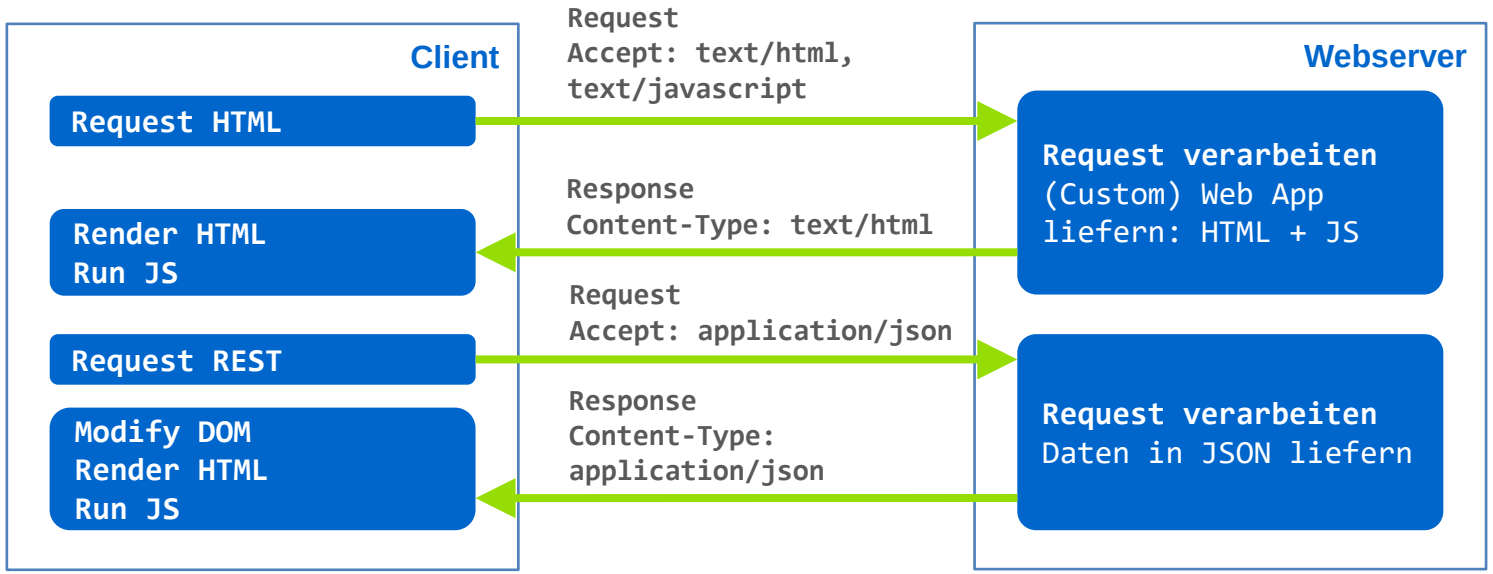
Reaktion auf Navigation des menschlichen oder maschinellen Users: URL- und Pfad-Zugriffe.

- Server-seitig: Abbildung von URL/Path auf Webserver-Funktionen/Ressourcen.
- Client-seitig: Abbildung von Links auf lokale Elemente / Ressourcen.

Web Apps: Einfache und Single Page Applications (SPA)



- Client erhält HTML.
- Client bietet User damit Links und Forms zur Interaktion an.
- Anwendungsverhalten ist nur auf dem Server implementiert.
- Anwendungszustände ergeben sich aus den HTML-Seiten vom Server.



- Client erhält HTML mit JavaScript: Die „Seite“ repräsentiert die vollständige Frontend-Applikation.
- Anwendungsverhalten ist damit auf Client *und* Server implementiert.
- Die Client-Anwendung tauscht Datenobjekte mit dem Server aus.
- Der Client ändert seinen Anwendungszustand lokal, kann also selbst Veränderungen im Frontend darstellen.

Ist das „veraltet“ und „modern“? Was sind hier Vor- und Nachteile?

Konzept: Sessions

In vielen Web-Anwendungen gibt es

Sessions: Menge von logisch zusammengehörenden Transaktionen (Requests, Responses) und ihrer Informationen.

Z. B. Einkauf in einem Web Shop.

HTTP/1.* ist aber zustandslos, so dass die Anwendung selbst für die Definition von Sessions verantwortlich ist: **Session Management**.

Session Management und IDs

Session Management muss einen Kontext zwischen Web Client und Server über mehrere Requests pflegen.

Session Management

1. Beim Zugriff auf eine Web-Applikation wird vom Server eine eindeutige Session ID generiert.
2. Die Session ID wird an den Client geschickt.
3. Der Client nutzt die Session ID in jeder weiteren Request. Der Server nutzt diese ID, um die zum Client passenden Daten zu finden.
4. Eine Session endet, wenn
   - ein User sich abmeldet,
   - über eine definierte Zeit inaktiv ist,
   - der Server sie für ungültig erklärt.

Übliche Implementierungen

Cookies – ID gespeichert in einem Cookie auf dem Client, Cookie enthalten im Header jedes folgenden Requests.

URL-Parameter – ID wird als Query Parameter übertragen.

Verstecktes Formularfeld – in HTML enthalten, also im Request Body.

JSON Web Tokens (JWT) – signiert in jedem Request enthalten, zustandslos, muss vom Server nicht mehr gespeichert werden.

OAuth/OpenID Connect – Session Management wird von einem 3rd Party Identity Provider übernommen. Praktisch für Single Sign-On.

Webserver

Ein Webserver ist ein flexibel einsetzbares, leistungsfähiges Element in unseren Web-Architekturen.

Primäre Aufgaben

Request Handling – Annahme von HTTP/S Requests von Clients.

Content Serving – Lieferung von statischen Ressourcen (HTML, CSS, JavaScript) und dynamischen Inhalten (auch über Interaktion mit Backend-Systemen).

Resource Management – Parallele Verbindungen verwalten und Ressourcennutzung optimieren.

Logging – Aktivitäten aus Requests und Transaktionen über Sessions überwachen.

Damit ist er auch in den Rollen
- Application Server
- Reverse Proxy
- API Gateway

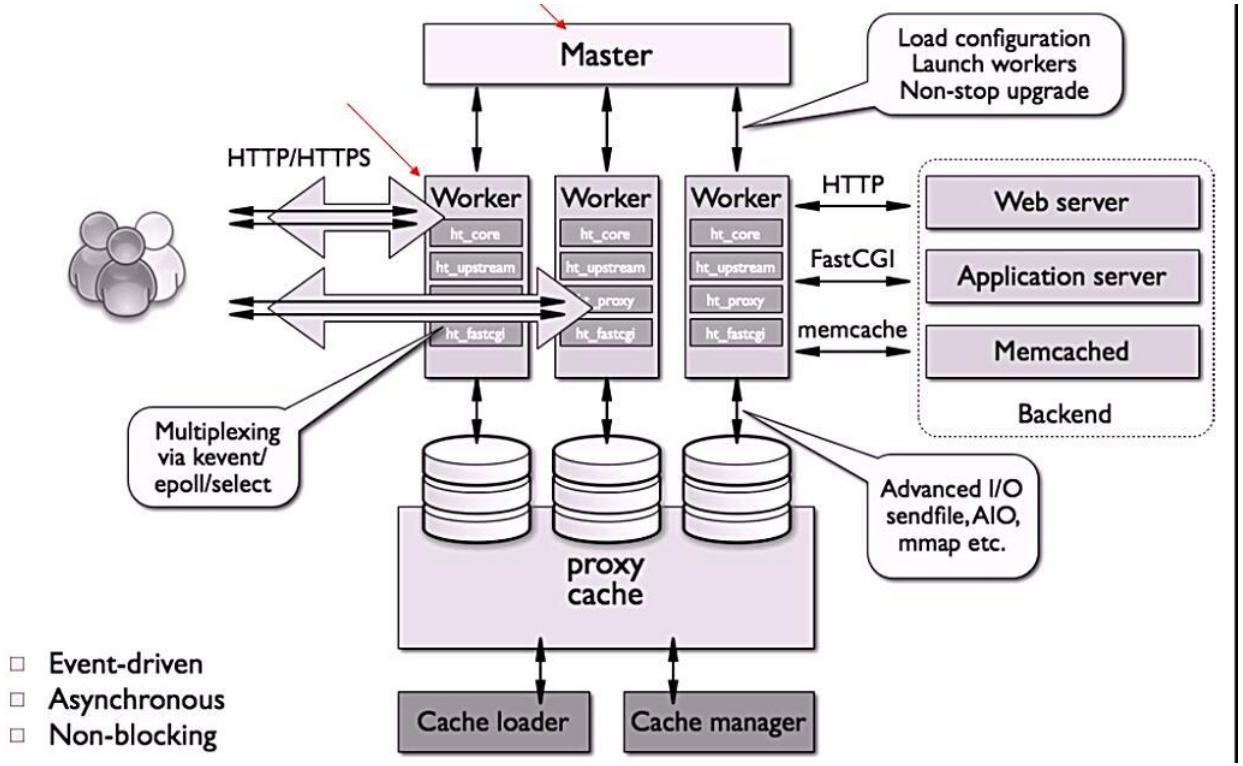
einsetzbar.

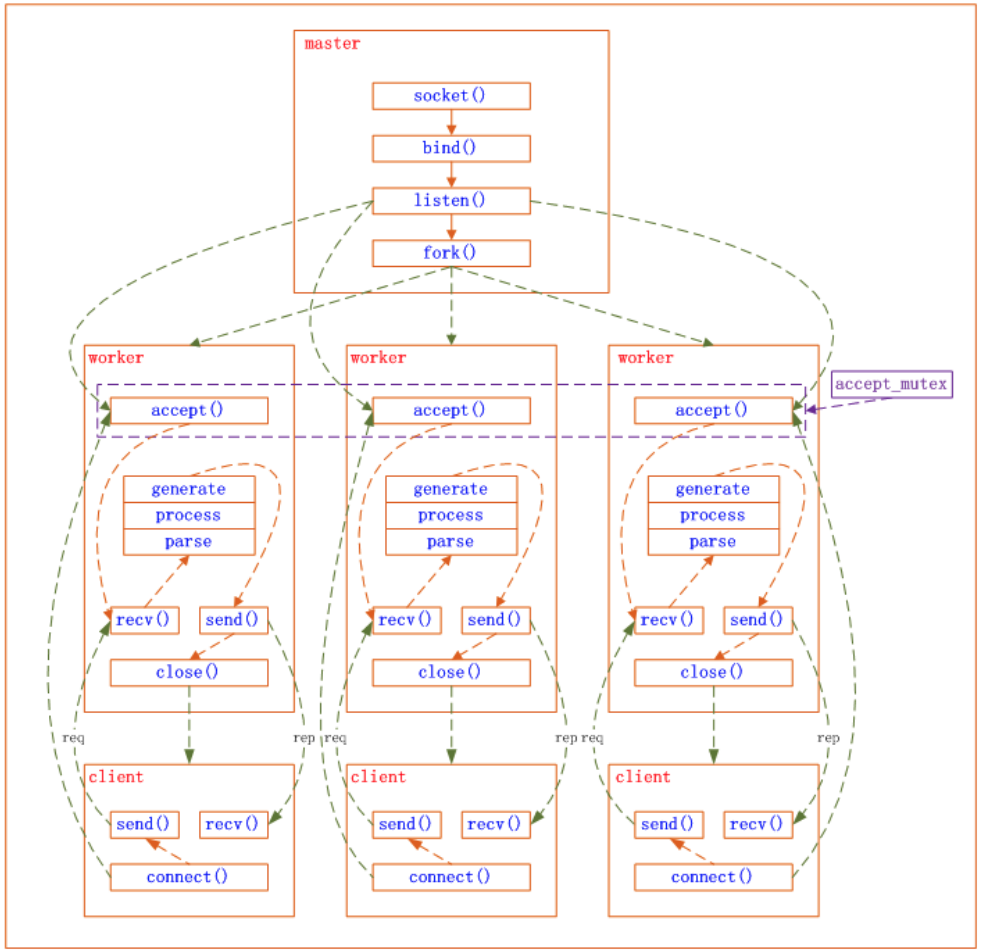
Verarbeitungsabläufe

1. **Annahme von Requests** – Client schickt HTTP/S Request, Webserver identifiziert die angefragte Ressource, die Zugriffsaktion und liest Header und ggf. Body.
2. **Routing und Entscheidungen** –
   - *statische Ressourcen* werden direkt gemäß Pfad aus einem File System gelesen,
   - für *dynamische Inhalte* ermittelt der Webserver einen Pfad zum passenden Dienst und leitet die Anfrage weiter, oder ruft eine passende Funktion auf.
3. **Generierung Response** – in HTTP/S mit dem Status und den angefragten Daten kodiert im Body, Verschlüsselung bei TLS.
4. **Sendung Response** – über den passenden Port zum Client.
5. **Logging und Cleanup** – IP-Adressen, Zeitstempel etc. speichern, nicht mehr benötigte lokale Ressourcen freigeben.

Architektur Webserver: Beispiel *nginx*

nginx („engine x“) ist ein beliebter freier Webserver: `nginx.org`





https://programmer.group/working-principle-of-nginx-and-configuration-of-nginx.html

Konzept: Representational State Transfer (REST)

REST ist ein Architekturstil primär für die Kommunikation von Maschine zu Maschine (M2M) in verteilten Client/Server- bzw. Producer/Consumer-Anwendungen. Er wurde entwickelt von Roy Fielding (`https://ics.uci.edu/~fielding/pubs/dissertation/top.htm`).

REST setzt auf vorhandene Web-Komponenten und -Protokolle auf. Die wesentlichen Eigenschaften:

Zustandslosigkeit – Jede REST-Anfrage enthält alle Informationen, die der Server zur Verarbeitung benötigt. Der Server merkt sich zwischen den Transaktionen keine Client-Zustände. Dadurch ist flexible Lastverteilung möglich.

Einheitliche Schnittstelle –

1. Jede Ressource ist eindeutig identifiziert über eine URI.
2. Egal, wie der Server die Ressourcen intern verwaltet, nach außen sind sie repräsentiert in HTML, JSON oder XML.
3. Die Repräsentation ist vollständig für jegliche Bearbeitungen durch den Client.
4. HATEOAS – siehe rechts.

Mehrschichtige Systeme – hinter der einheitlichen Schnittstelle können komplexe Dienste aus vielen Komponenten laufen.

Hypermedia As The Engine Of Application State (HATEOAS) – weitere Ressourcen und Navigationsmöglichkeiten kommuniziert der Server in seinen Responses.

Führen Ressourcenzugriffe also zu neuen Ressourcen, muss der Client sich diese nicht selbst konstruieren, oder aus anderen Quellen lesen, sondern erhält die Links vom Server.

Das ist für M2M-Kommunikation in JSON die Analogie eines menschlichen Users, der auf der Homepage weitere Links in HTML entdeckt und verfolgen kann.

Endpoints, Zugriffsmethoden und Beispiel vorne im Kapitel HTTP.

HATEOAS: Beispiel


```
{
    "id": 456,
    "title": "Was Sie gerne hören",
    "artist": "Wen Sie mögen",
    "album": "Irgendeins",
    "duration": 233,
    "likes": 12500,
    "is_liked_by_user": true,
    "links": [
      {
        "rel": "self",
        "href": "https://api.musicstreaming.com/songs/456",
        "method": "GET"
      },
      {
        "rel": "like",
        "href": "https://api.musicstreaming.com/songs/456/like",
        "method": "POST"
      },
      {
        "rel": "unlike",
        "href": "https://api.musicstreaming.com/songs/456/unlike",
        "method": "DELETE"
      },
      {
        "rel": "add_to_playlist",
        "href": "https://api.musicstreaming.com/playlists/{playlist_id}/add",
        "method": "POST",
        "templated": true
      },
      {
        "rel": "artist",
        "href": "https://api.musicstreaming.com/artists/123",
        "method": "GET"
      },
      {
        "rel": "album",
        "href": "https://api.musicstreaming.com/albums/789",
        "method": "GET"
      }
    ]
}
```

OpenAPI

Die **OpenAPI Specifikation** (`openapis.org`) ist ein Standard zur Beschreibung von HTTP APIs.

- Sprach-agnostische Beschreibung der Schnittstellen von Web-Services.
- Lesbar für Mensch und Maschine.
- Nutzbar für Dokumentation, Programmgenerierung, Konfiguration, Test.
- Geschrieben in JSON oder YAML.

Siehe `OpenAPI_Bsp.json`

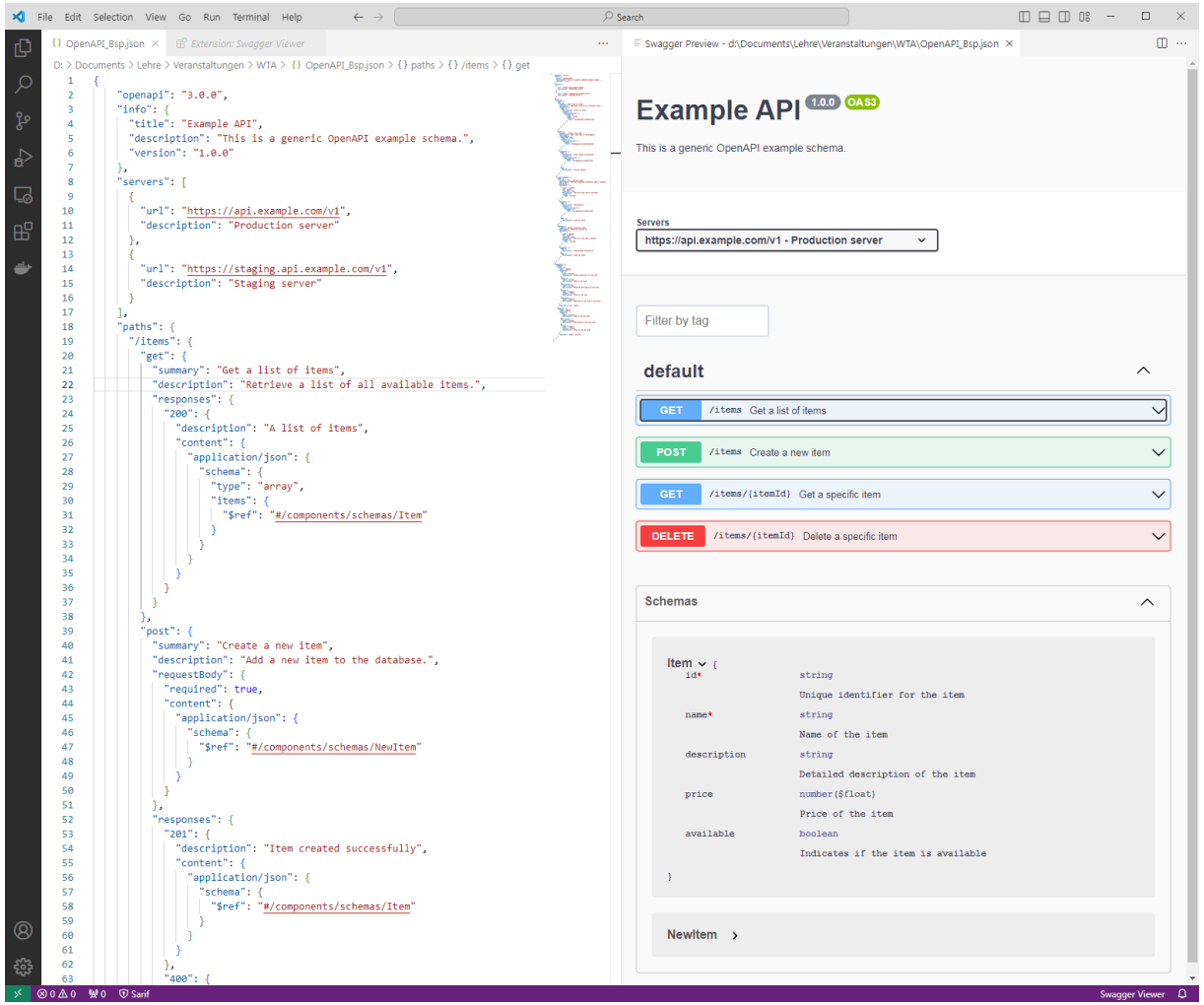
`openapi` – Version der genutzten Spezifikation
`info` – Metadaten über die API
  `title` – Name der API
  `description` – Kurze Beschreibung ihres Zwecks
  `version` – Version der API
`servers` – Liste der URLs der API
  Jeder Server hat eine **`url`** und **`description`**
`paths` – Endpunkte und ihre Operationen
  Für jeden **`/path`** sind die Zugriffsmethoden definiert
  Jede Operation beinhaltet
    `summary` und **`description`**
    `parameters` – Query-, Path- oder Header-Parameter
    `requestBody` – Datenstrukturen z. B. für POST
    `responses` – Mögliche Antworten mit Statuscode und Schema
`components` – Wiederverwendbare, referenzierbare Definitionen von Datenstrukturen und Schemas
  `schemas`
  `$ref`

Swagger

Swagger (`swagger.io`) ist eine Werkzeugsammlung zur Arbeit mit API-Spezifikationen wie OpenAPI.

Swagger™
Supported by SMARTBEAR

- *Editor* mit Formatprüfung, Extensions z. B. für Visual Studio Code.
- *Code-Generierung*: Server Stubs und Client SDKs, u. a. für JS und Python und deren Frontend Frameworks und Webserver-Bibliotheken.
- *Interaktive UI* für API-Dokumentation und Aufrufe der API, Tests.



Swagger Viewer in Visual Studio Code

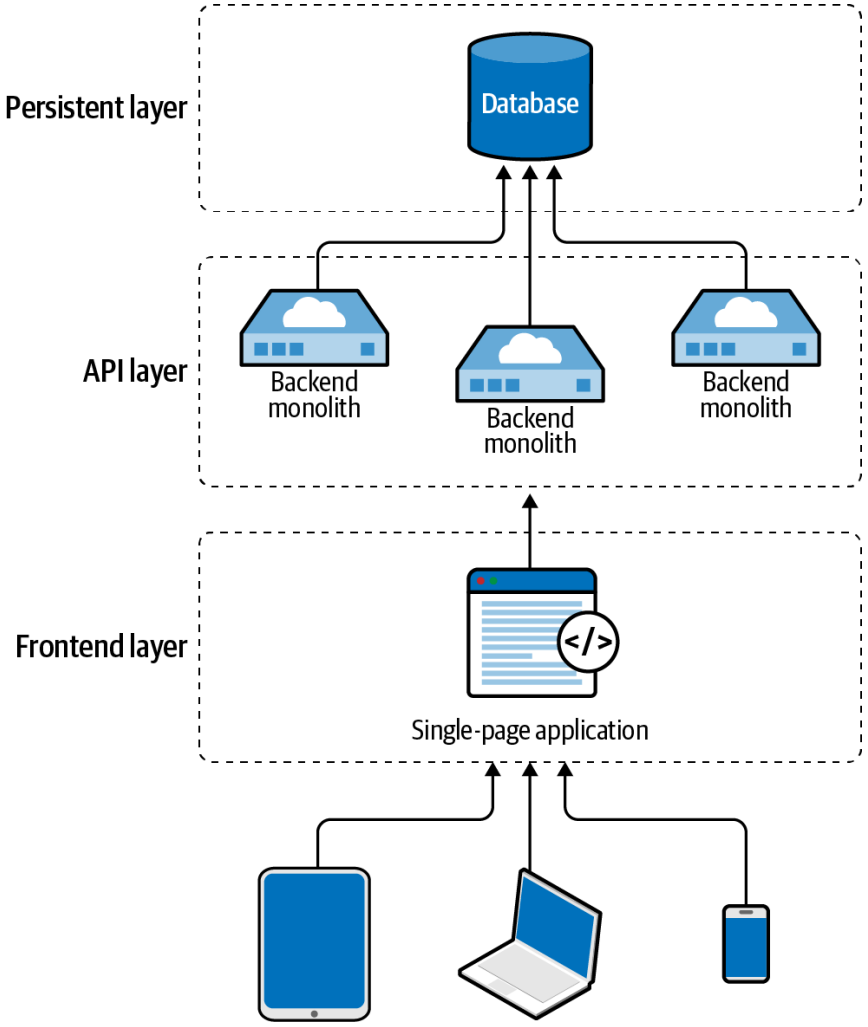
Übung 9: Ein Server für unsere Meta Playlist App

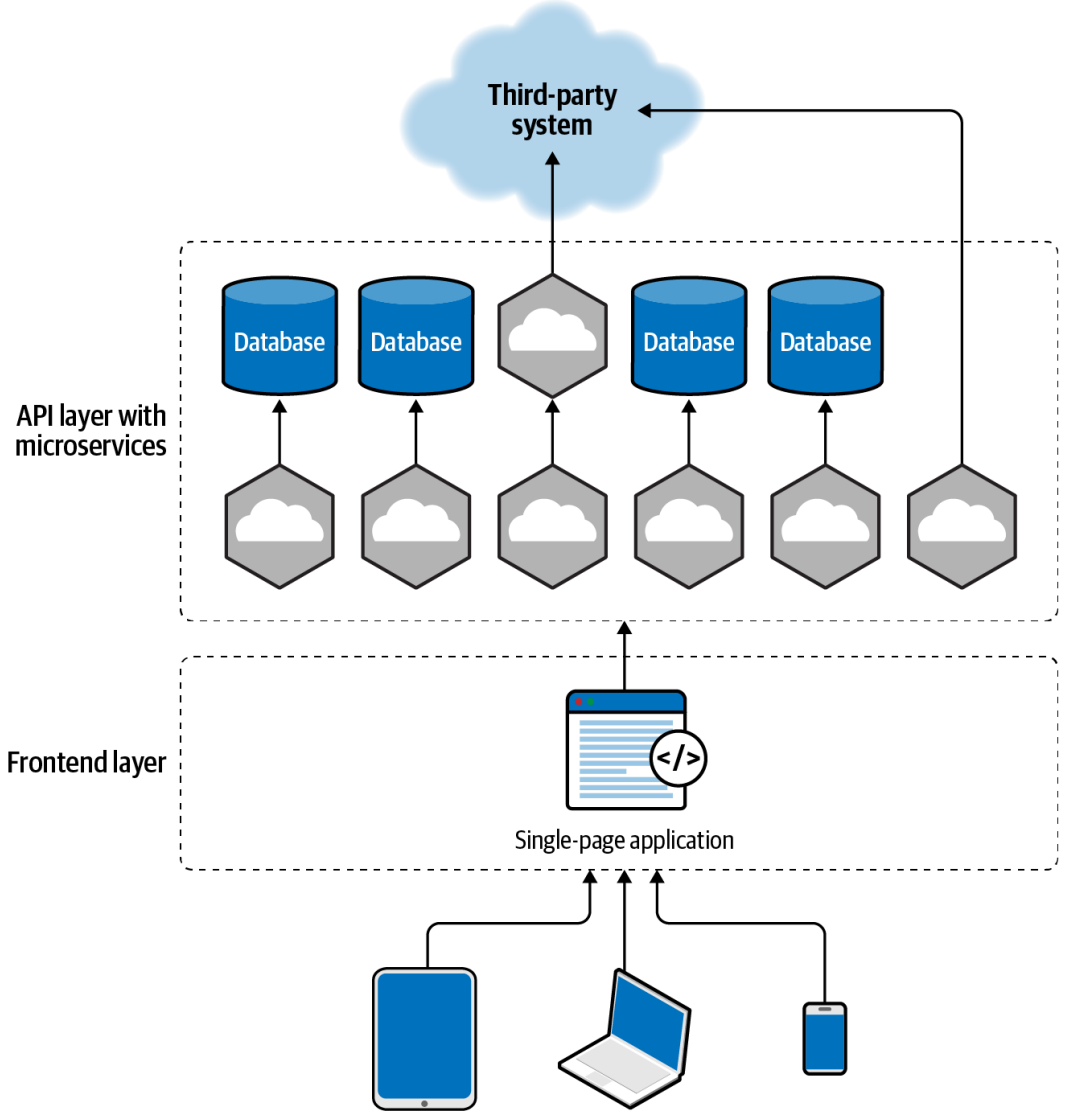
Wir wollen unsere Playlists nicht mehr lokal, sondern auf einem Server speichern. Außerdem wollen wir die Playlist-Verwaltung als Dienst anbieten, der von unserer Web App, aber auch von anderen Clients benutzt werden kann.

1. Spezifizieren Sie für den Server eine REST API im Format von OpenAPI mit folgenden Möglichkeiten: Alle Playlists abrufen, eine Playlist anlegen oder aktualisieren, eine bestimmte Playlist abrufen, eine Playlist löschen. Die Daten sollen in JSON ausgetauscht werden.
2. Ergänzen Sie den Webserver um die passenden Routen und Funktionen. Die Playlists speichern Sie zunächst einfach lokal in einer Variable im Speicher des Servers als JSON-Struktur, wie wir das vorher schon im Client gemacht haben. Den Client liefert der Server als HTML-Dokument zurück, z. B. direkt als `index.html` auf der Default Route `/`.
3. Stellen Sie den Webserver als Container bereit und testen Sie ihn auf geeignete Weise (*curl, Swagger, Postman, o. ä.*) zunächst gesondert als Subsystem.
4. Passen Sie die Client App auf die Schnittstelle des Servers an und testen Sie die Integration.

Sie müssen nicht unbedingt alles ausimplementieren. Wichtig ist es, dass Sie die Konzepte, Funktionsweisen und Abläufe praktisch nachvollziehen.

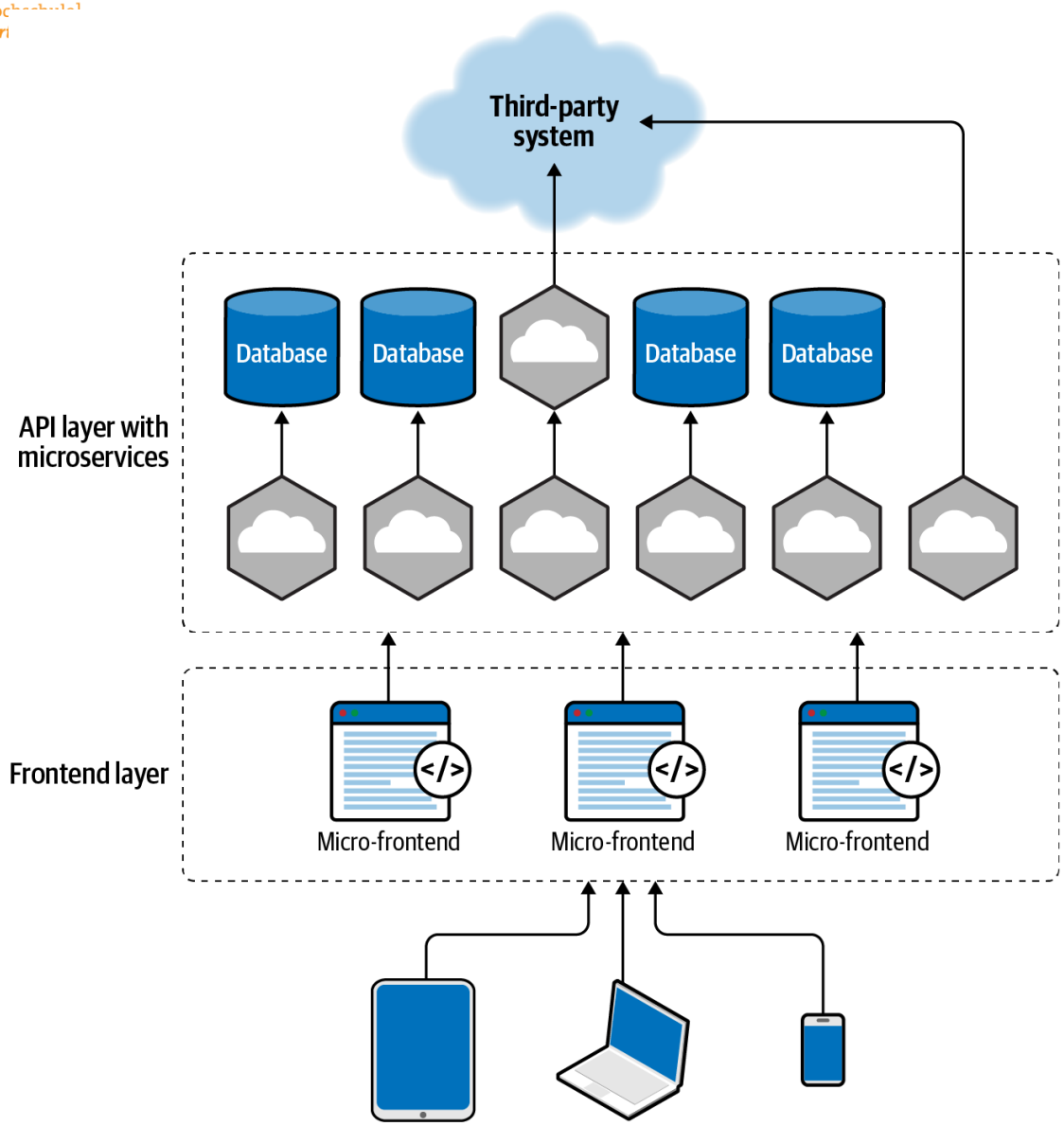
Konzept: Microservices





Luca Mezzalira: Building Micro-Frontends. O'Reilly 2021

Konzept: Microfrontends



Luca Mezzalira: Building Micro-Frontends. O'Reilly 2021

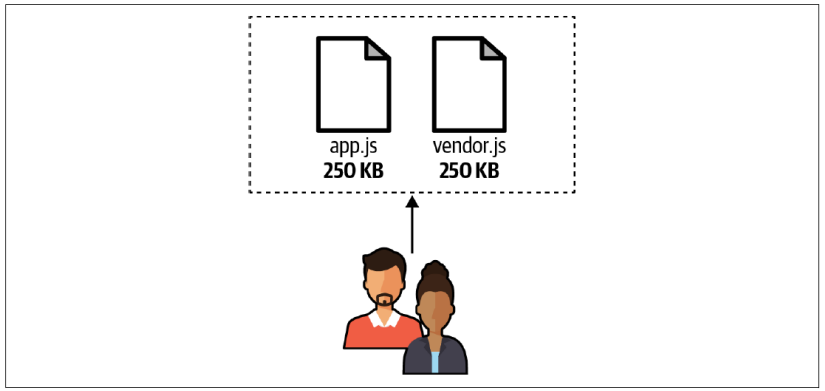


Figure 4-9. Any user of an SPA has to download the entire application regardless of the action they intend to perform in the application

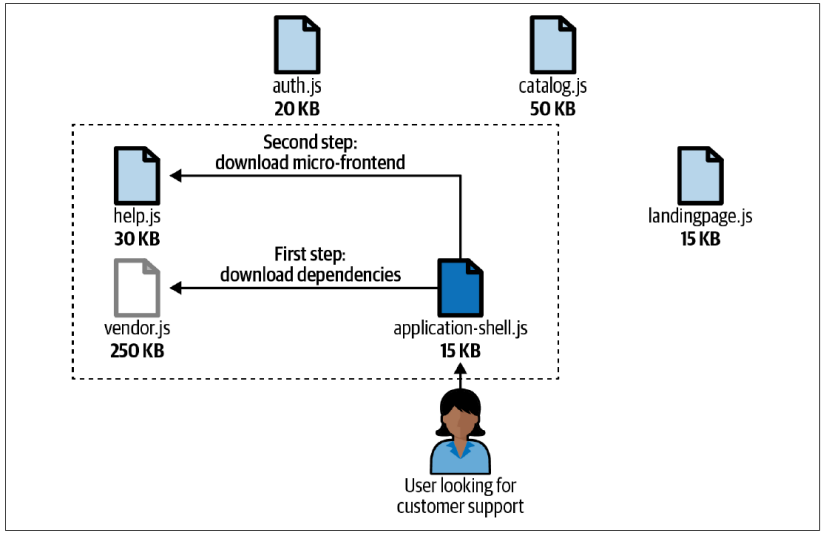


Figure 4-10. A vertical-split micro-frontend enables the user to download only the application code needed to accomplish the action the user is looking for

Microservices und -frontends: Prinzipien

Business-Orientierung

Ein Dienst/Frontend bildet eine Geschäftsfunktion ab.

Dezentralisierung

Teams sind für ihre Komponenten selbst verantwortlich.

Automation

Deployment und Betrieb sind so weit wie möglich automatisiert.

Unabhängige, schnelle Deployments

Keine Gesamtintegration erforderlich, unabhängige Wartung, individuelle Skalierung.

Kapselung

Schnittstellen veröffentlichen, Implementierung abgeschlossen halten, intern passende Technologien nutzen.

Resilienz

Komponentenprobleme sollen andere Teile möglichst wenig beeinflussen.

Hohe Überwachbarkeit

Statt einem geordneten Monolithen haben wir eine hohe Zahl von autonomen Teilen.

- ⇒ Komplexes Systemverhalten
- ⇒ Komplexe Orchestrierung
- ⇒ Hoher Netzwerk-Overhead
- ⇒ Komplexes Datenmanagement durch verteilte Datenhaltung
- ⇒ Konsistenzwahrung im Gesamtsystem nicht trivial
- ⇒ Komplexes, aber sehr wichtiges Monitoring: Konsolidierung und Analyse verteilter Logs und Metriken

Datenhaltung

In Web-Anwendungen müssen wir regelmäßig Daten persistieren, also **dauerhaft über verschiedene Zeitskalen speichern**, z. B. über eine Session oder dauerhaft. Auch benötigen wir Datenhaltungen, um **Daten über Dienste hinweg verfügbar** zu machen, also nicht nur im lokalen Speicher zu bearbeiten.

	SQL-Datenbanken	NoSQL-Datenbanken	Cloud Storage	Verzeichnisdienste
Zweck	(Objekt-)Relationale Datenhaltung in strukturierten Tabellen mit vordefiniertem Schema	Flexible, uneinheitlich, semi-strukturierte oder unstrukturierte Daten, „Dokumente“	Skalierbare Speicherung von unterschiedlichen Datenobjekten, also zu jedem Zweck	Identitäts- und Ressourcenmanagement einer Organisation
Datenstrukturen	Tabellen mit Spalten und Zeilen	JSON, Key-Value, Graphen	Objekte (Media, Logs, Backups) mit Metadaten, Blöcke oder Dateien eines Dateisystems, Managed No-/SQL	Baumhierarchie, auf schnelle und viele Lese-Operationen optimiert, mit Zugriffs-protokollen über Netz
Skalierung	Vertikal und begrenzt horizontal skalierbar	Horizontal skalierbar	Abhängig von Kosten sehr flexibel	Begrenzt skalierbar
Use Cases	ERP, Transaktionen, Analytics	IoT, Echtzeit-Analytics, Social Network Graphs	Archive, Backups, Data Lakes	Authentifikation, Zugriffs-rechte, Netzwerkressourcen
Beispiele	MySQL, PostgreSQL	MongoDB, Cassandra, Redis	Amazon S3, Google Cloud Storage, Azure Blob Storage	OpenLDAP, MS Active Directory

Übung 10: Eine Datenhaltung für unseren Web-Dienst

Wir wollen unsere Playlists in einer Datenhaltung ablegen. Dazu binden wir die dokumentenorientierte Datenbank *CouchDB* (`couchdb.apache.org`) an unseren Webserver an.

1. CouchDB nutzen wir als Container, die Datenbank wird über eine REST API angesprochen. Es gibt ein fertiges Image, das wir einfach zusammen mit unserem Webserver in ein `docker-compose.yaml` integrieren. Diese Datei, mit einigen abhängigen Dateien, ist als Ausgangspunkt gegeben (liegt auf Teams). Wir gehen die zunächst einmal gemeinsam durch.
2. Der Webserver muss nun die JSON-Objekte der Playlists über die REST API an die CouchDB übergeben bzw. von dort abfragen. Die Funktionen des Webservers modifizieren wir entsprechend – finden Sie heraus, wie das geht. Den Client sollte der Webserver wie in Übung 9 liefern.
3. Testen Sie die Gesamtlösung auf geeignete Weisen, also z. B. den Server über seine REST API, dann zusammen mit dem Client. Beantworten Sie dabei folgende Fragen:
   a. Wie genau findet sich hier das Microservice-Konzept umgesetzt?
   b. Welches wichtige Element läuft im CouchDB-Container neben der Datenbank selbst?
   c. Wie sieht die Architektur aus, welche Komponenten gibt es, über welche Schnittstellen kommunizieren die?

Zwischenstand: Wo sind wir?

Web-Anwendungen und -Dienste
Verteilte Systeme im Internet

Komponenten Bausteine und Konzepte	**REST** Konzept und Werkzeuge
Webserver Funktion und Rollen	**Microservices/-frontends** Architekturkonzept
Web Apps Verteilung und Ablauf	**Datenhaltung** Arten und Lösungen

Wir haben nun gesehen, wie eine verteilte Anwendung mit Web-Technologien aufgebaut ist und wie ihre Bestandteile miteinander interagieren.

Wie rollen wir ein solches System aus und wie betreiben wir es?

Deployment und Betrieb von Web-Anwendungen und -Diensten

Konzept: Orchestrierung

In unserem Szenario haben wir eine überschaubare Anzahl an Komponenten: Den Web-/Applikationsserver, die Datenhaltung und Clients – unsere App als Beispiel für ein MMI und z. B. `curl` als Beispiel für M2M-Interaktion.

Im Echtbetrieb können Web-Anwendungen aus hunderten von Diensten bestehen.

Diese vielen Dienste müssen wir über unsere Cloud- oder On-Premise-Infrastruktur aus vernetzten Rechnerknoten verteilen, ebenso wie die im Betrieb anfallende Arbeitslast.

Das ist **Orchestrierung**:

1. **Scheduling** – Entscheidung, welcher Dienst auf welchem Knoten laufen soll.
2. **Skalierung** – Anpassung der Dienste an die Arbeitslast. Siehe auch nächste Seite.
3. **Load Balancing** – gleichmäßige Aufteilung der ankommenden Anfragen.
4. **Verfügbarkeit** – des Gesamtsystems gewährleisten, Dienste im Problemfall neu starten, „Self-Healing“.
5. **Service Discovery** – Netzwerk abstrahieren für einfachere interne Adressierung.
6. **Zustandsmanagement** – die einzelnen Dienste im geforderten Betriebsmodus halten.
7. **Deployment Management** – Rollouts und ggf. Rollbacks neuer Versionen, Downtimes minimieren.
8. **Konfigurationsmanagement** – Secrets, Umgebungsvariablen, Konfig-Dateien über die verteilten Dienste.
9. **Überwachung** – Performance der Komponenten und des Gesamtsystems, Logging.

Konzept: Skalierung

Eine Web-Anwendung ist schwankenden Lasten (*Workloads*) ausgesetzt. Cloud-Infrastrukturen und Container bieten zwei Strategien, flexibel darauf zu reagieren:

Vertikale Skalierung

Hinzufügen oder Entfernen von **Ressourcen** (CPUs, RAM, Massenspeicher, Bandbreite etc.) für eine einzelne Instanz (Server, VM, Container), um deren **lokale Leistungsfähigkeit** anzupassen.

+ Einfach umsetzbar
+ Keine Änderungen an der Architektur, Ideal für Monolithen

- Ressourcenlimits
- I. d. R. Downtimes erforderlich

Horizontale Skalierung

Hinzufügen oder Entfernen von **Instanzen** (Server, VM, Container), um die **Verteilung der Arbeitslast** anzupassen.

+ Keine Ressourcengrenze
+ Verfügbarkeit kann erhalten bleiben

- Hohe Komplexität, Koordination
- Latenz durch aufgeteilte Datenflüsse

Ausgangssituation Orchestrierung

Wir haben ein Netz aus virtuellen oder physischen Rechnerknoten,

- mit lokalen Ressourcen – Prozessorkerne, Speicher,
- adressierbar über IP (Router/Switch implizit).

Unsere Web-Anwendung aus dem Kapitel zuvor besteht aus mehreren Komponenten. Die möchten wir über unsere verfügbaren, vernetzten Rechner verteilen.

Um das über den Betrieb und Lebenszyklen einer solchen verteilten Anwendung möglichst optimal zu gestalten, hilft uns **Kubernetes** (**K8s**).

kubernetes

Kubernetes: Ein virtuelles Netzwerk mit *kube-proxy / CNI*

Node IPs beschreiben unsere Netzwerk-Infrastruktur, on-premise oder von einem Cloud Provider, in jedem Fall das „echte Internet“ (privat oder öffentlich).

Die im folgenden Kontext zusammengehörenden Nodes bezeichnen wir als **Cluster**.

Wir wollen aber mehr Flexibilität in der Vergabe von IP-Adressen und Vereinheitlichung für unsere Orchestrierung, also verwaltet auf jedem Knoten ein **kube-proxy** virtuelle, interne **Cluster IP**s und Regeln zum Routing *Node IPs* ↔ *Cluster IPs* und Load Balancing.

Die Prozesse auf den Knoten nutzen die kube-proxies als virtuelle Router. Die legen so ein virtuelles Netz aus Cluster IPs über das echte Netz.

Für die kube-proxies gibt es Plug-ins für das **Container Network Interface (CNI)**, das das Management der unterliegenden Netzwerkebene regelt (Data Plane aus CI).

Kubernetes: Node-lokales Management mit *kubelets*

Eine Reihe der Knoten wollen wir zur Ausführung von Anwendungen und Diensten in Containern nutzen. Auf diesen Arbeitsknoten installieren wir jeweils ein **kubelet**, das

- die Lebenszyklen der lokalen Container über eine Container Runtime (*Docker, containerd, Podman*) steuert,
- deren geteilte Speicher und Secrets verwaltet,
- die Ressourcennutzung und -verfügbarkeit auf dem Knoten überwacht.

Kubernetes: Cluster Management mit der *Control Plane*

Die nun ausgerüsteten Nodes steuern wir als Cluster mit der **Control Plane** von Kubernetes:

kube-apiserver – die Kernkomponente für das Management aller Aspekte des Clusters. Alle Interaktionen zur Steuerung laufen über ihre REST API, zentrales Gateway und Communication Hub für alle K8s-Komponenten.

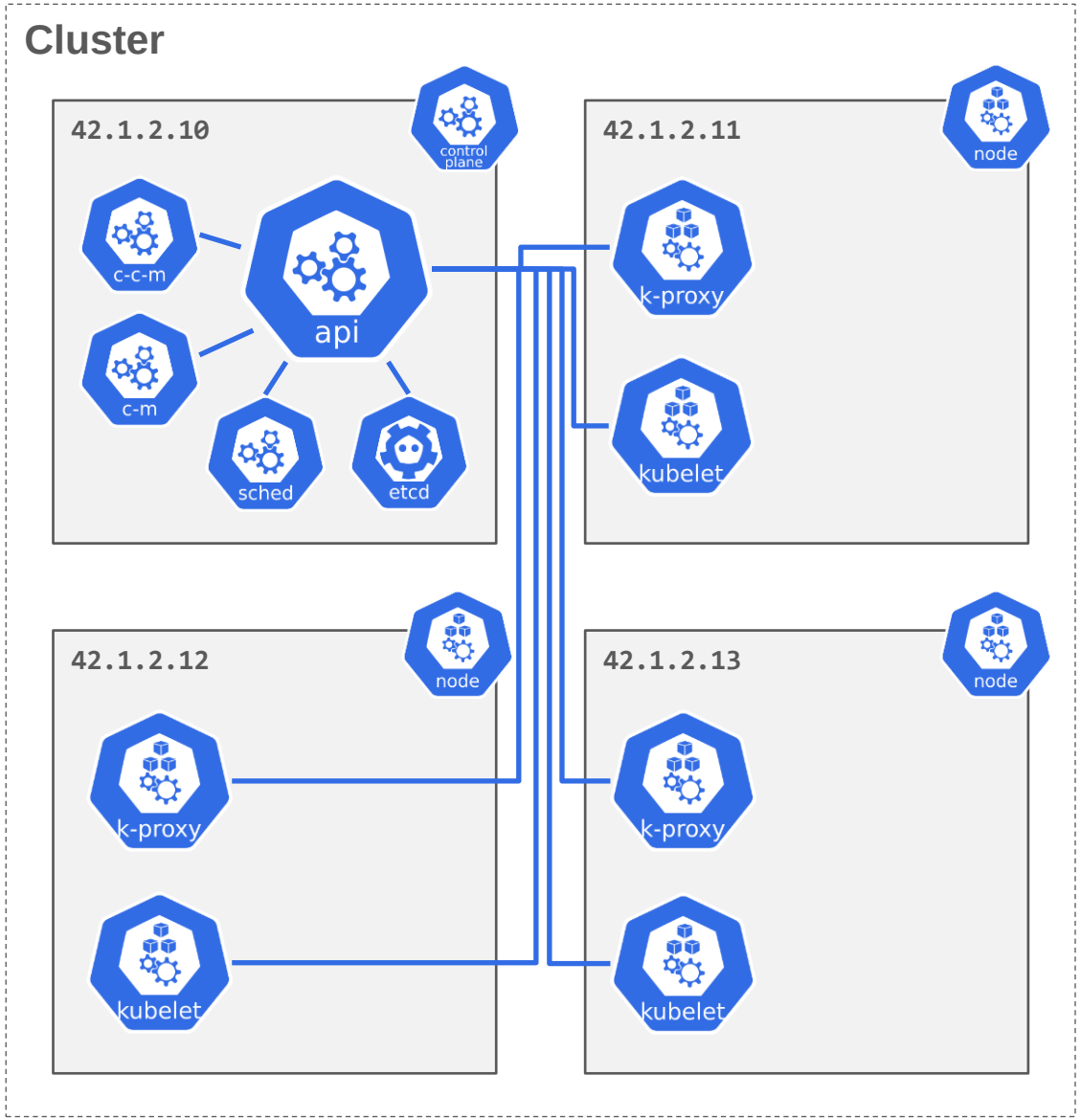
etcd – Key-Value Store für die Speicherung der Soll- und Ist-Zustände aller Komponenten im Cluster und Metadaten des API-Servers.

kube-scheduler – Zuteilung von Workloads an Nodes auf Basis der Ressourcenbedarfe, -verfügbarkeiten und konfigurierter Regeln.

kube-controller-manager – führt *Controller* aus, Kontrollschleifen, um durch passende Eingriffe vom Ist-Zustand den Soll-Zustand des Clusters zu erreichen und aufrecht zu erhalten.

cloud-controller-manager – Integration mit unterliegenden Cloud-Infrastrukturen.

Die Control Plane kann auch woanders laufen (z. B. bei *Managed K8s*) und lässt sich selbst skalieren und verteilen.



Übung 11: K8s zum Ausprobieren

1. Installieren Sie **Minikube**: `https://minikube.sigs.k8s.io/`
2. Installieren Sie **kubectl**: `https://kubernetes.io/docs/tasks/tools/`
3. Starten Sie Ihren K8s Cluster mit `minikube start`
   Unter MacOS und Linux sollten Sie mit `--driver=docker` die Control Plane im Container nutzen.
4. Orientieren Sie sich auf dem Dashboard mit `minikube dashboard` und finden Sie die Informationen zu Ihrem lokalen Node.
   Alternativ auf der Kommandozeile:
   ```
   kubectl get nodes
   kubectl describe node [NAME]
   ```
5. Warum bezeichnet man K8s als „Betriebssystem der Cloud“? Welches sind die Analogien?

Falls Sie alternative Installationen zum lokalen Testen ausprobieren möchten, gibt es z. B. `https://devopscube.com/kubernetes-cluster-vagrant/`

Kubernetes: Workloads und Pods

Auf dem Cluster wollen wir nun die Anwendungen, Dienste, bzw. Prozesse unserer komplexen Web-Applikation laufen lassen – allgemein **Workloads**.

Die kleinste Einheit, die K8s auf einem Cluster laufen lassen kann, ist ein **Pod**:

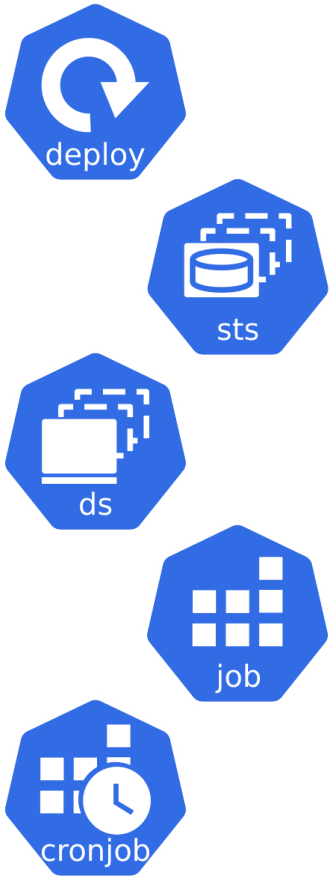
- Eine fachlich/technisch zusammenhängende *Gruppe*
- aus einem oder mehreren *Container/n*,
- die sich *Ressourcen teilen* – Speicher, Netzwerk (gemeinsame Namespaces) – und auf der gleichen Hardware/VM laufen,
- eine *gemeinsame Cluster-weite IP-Adresse* haben und sich *einen Port-Nummernraum* teilen.

Wenn wir eine verteilte Web-Applikation mit K8s betreiben wollen, werden ihre containerisierten Komponenten also immer auf solche Pods aufgeteilt.

Kubernetes: Workload Resources

Müssten wir nun die Pods mikro-managen, wäre gegenüber den Containern aus dem Kapitel vorne nicht viel gewonnen. Wir haben aber die ausgestatteten Nodes und die Control Plane.

Damit bietet K8s **Workload Resources**, die das Management für uns abstrahieren.



Deployment – Management von jederzeit austauschbaren, zustandslosen Applikationen, z. B. Webserver, Frontend-Anwendungen.

StatefulSet – Eindeutig identifizierbare Applikationen mit Zuständen und persistenten Speichern, z. B. Datenbanken, Message Broker.

DaemonSet – Hintergrundanwendungen, die auf allen oder bestimmten Nodes laufen, z. B. Logging, Monitoring, Networking.

Job – kurze, einmal ausgeführte Aufgaben, z. B. Batch Processing für Datenmigration.

CronJob – regelmäßige Verarbeitung zu oder nach bestimmten Zeiten, z. B. Backups, Datenbereinigung.

Workload Resources: Beispiel nginx Webserver

Deklarative Spezifikation als **Manifest**.

`label`s als Key/Value-Paare nutzen wir, um die einzelnen Objekte selektieren und zuordnen zu können, auch über unterschiedliche Manifests hinweg.

`example_deployment.yaml`

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-web-app
  labels:
    app: web-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: web-container
        image: nginx:latest
        ports:
        - containerPort: 80
```

Statt die einzelnen Pods imperativ zu aktivieren, ist es üblich und einfacher, K8s den **gewünschten Zustand** zur Umsetzung zu übergeben,

`kubectl apply -f example_deployment.yaml`

und K8s das Management zu überlassen.

Der kube-scheduler weist dann die angegebenen Workloads als Pods den passenden Nodes zu.

Mit der Anzahl der Replicas können wir das Deployment horizontal skalieren. K8s überwacht und korrigiert bei Bedarf den Zustand.

`kubectl get [deployments|pods] [-o wide]`

`kubectl describe [deployment|pod] [NAME]`

`kubectl delete [deployment|pod] [NAME]`

Was passiert, wenn wir ein Pod löschen?

Was passiert, wenn wir ein `curl` *oder* `ping` *auf einen der Webserver machen?*

Kubernetes: Das Netzwerkmodell

Pods auf einem Cluster

nutzen das *virtuelle Netz der kube-proxies/CNI*, adressieren sich also über ihre *Cluster IPs*.

Container in einem Pod

nutzen den gleichen *Network Namespace*, adressieren sich also über `localhost`.

Services

besitzen eine stabile Cluster-IP-Adresse bzw. einen stabilen DNS-Hostnamen. Ein Service repräsentiert einen Dienst, der durch Pods realisiert wird. Die Pods hinter einem Service können immer mal anders aussehen: Deren Instanzen und IPs können sich im Rahmen des Workload Managements oder aktualisierter Deployments ja ändern. kube-proxies implementieren die Services durch ihr Traffic Management.

Gateways (ehemals Ingress)

nutzen wir als *Single Point of Entry*, um unsere Dienste gut kontrolliert der Außenwelt, also im Internet anzubieten. Wir können dazu einen Ingress Controller und die für den Zugriff erlaubten Protokolle, Ports und HTTP-Routen definieren.

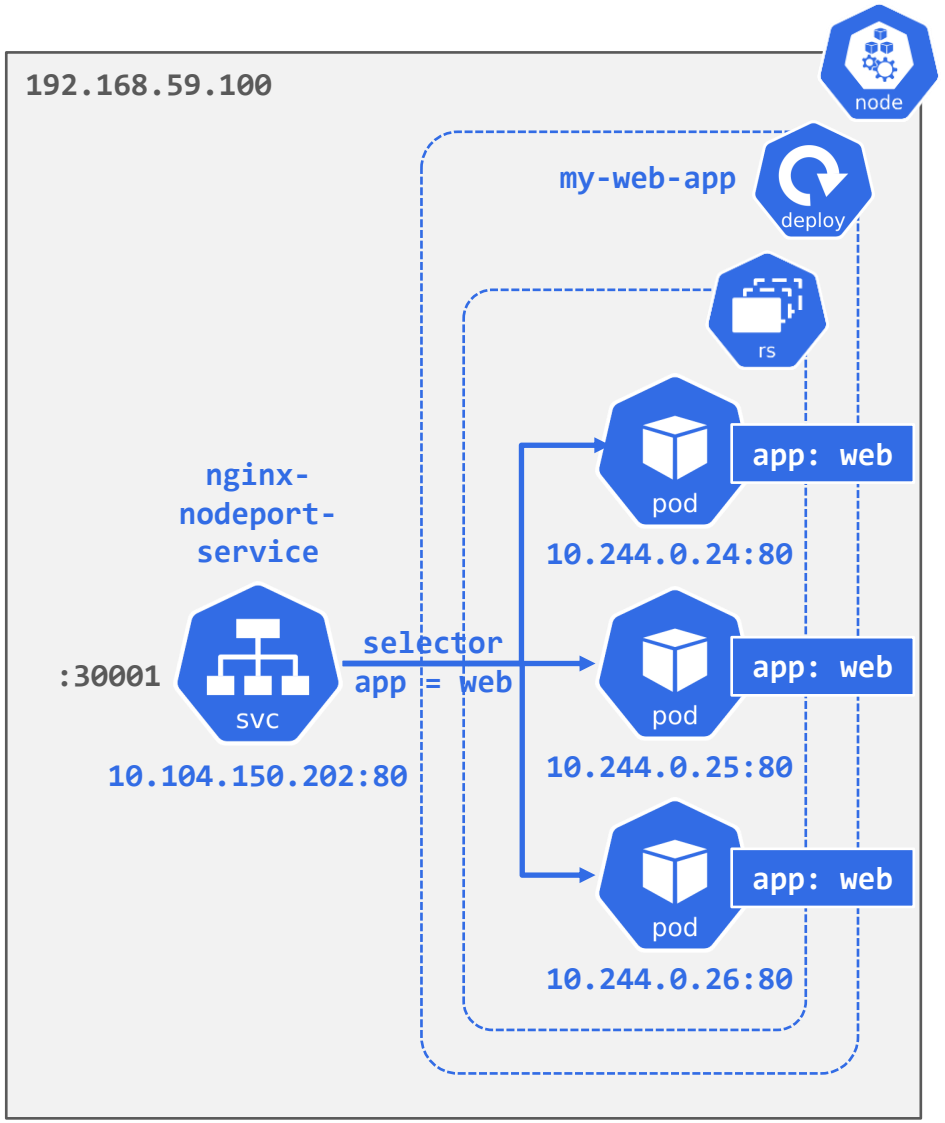
Service: Beispiel nginx Webserver

Für einen Service vom Typ **NodePort** öffnet K8s auf jedem Node einen Port auf diesen Service – von extern erreichbar.

Ist unser K8s bei einem Cloud Provider gehostet, können wir den Typ **LoadBalancer** nutzen. Dann läuft der externe Traffic zu unserem Service über externe Load Balancer.

example_service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport-service
  labels:
    app: web
spec:
  selector:
    app: web
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
    nodePort: 30001
  type: NodePort
```



Manifest anwenden, dann ausprobieren:

```
kubectl get services -o wide
```

```
kubectl describe service nginx-nodeport-service
```

Endpoints anschauen.

```
minikube ip
```

```
curl [minikube ip]:30001
```

Übung 12: Unsere Meta Playlist auf Kubernetes

1. Entwerfen Sie mit den vorgestellten und *ggf. weiteren* K8s Icons eine Zielarchitektur der Komponenten unserer Web-Applikation auf dem Node von Minikube oder Ihren eigenen Nodes (z. B. *draw.io* hat die Icons).
2. Spezifizieren Sie passende Manifests für die Pods und den Service.
3. Wenn möglich: Wenden Sie die Manifests in K8s an und testen Sie das Ergebnis mit dem Web Client.

Manifests und Helm Charts

Mit unseren Manifesten konnten wir unsere Meta Playlist in einem konkreten (minimalen) Netz ausrollen:

- Die Ressourcen unseres Systems sind in den Manifesten direkt für unsere Beispielumgebung spezifiziert.
- Durch sukzessive Anwendung (`kubectl apply`) der einzelnen Manifeste haben wir unser System auf dem Node installiert.

Der Abstraktionsgrad gegenüber der genutzten Infrastruktur ist niedrig. Wollten wir die Meta Playlist in der Cloud für die erwarteten Milliarden von Usern ausrollen, müssten wir diverse Parameter an diversen Stellen ändern. Das wird irgendwann unübersichtlich.

Eine Lösung:

Helm (`https://helm.sh/`)

Ein Paketmanager für wiederverwendbare und parametrierbare K8s-Anwendungen mit komplexen Abhängigkeiten und regelmäßigen Änderungen in Lebenszyklen mit Versionen.

Helm Charts: Pakete aus K8s Manifests mit Templates (`templates/`), Konfigurationen von Werten (`values.yaml`) und Metadaten (`Chart.yaml`).

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.app.name }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Values.app.name }}
  template:
    metadata:
      labels:
        app: {{ .Values.app.name }}
    spec:
      containers:
      - name: {{ .Values.app.name }}-container
        image: {{ .Values.app.image }}
```

Absicherung von Web-Anwendungen und -Diensten

Absicherung von Web-Diensten: Einführung

Schon im Kontext der Sessions vorne kamen die

- **Authentifikation (*authentication*)** – Verifizieren der Identität eines Users oder Systems, *“wer bist du?”* und
- **Autorisierung (*authorization*)** – Feststellung, auf welche Funktionen oder Ressourcen ein User oder System zugreifen darf, *“was darfst du?”,*

vor. Dazu gibt es eine Reihe von Lösungen.

Gerade in den verteilten Web-Anwendungen, die in unserer Veranstaltung Thema sind, ist allerdings das Prinzip des **Single Sign-on** hilfreich: Viele Stellen im System erwarten, dass wir uns identi- und authentifizieren, aber wir müssen das nur einmal tun.

OAuth 2.0: Vorstellung und Elemente

OAuth 2.0 (Open *Authorization*) ist ein Standard für **Autorisierung in verteilten Systemen im Internet**.

Er gewährleistet den sicheren, also kontrollierten Zugriff auf Ressourcen, ohne, dass der zugreifende Client seine Geheimnisse (ID, Passwort, etc.) preisgeben muss.

Wir geben dabei einer Anwendung das Recht, stellvertretend für uns auf Daten oder Funktionen einer anderen Anwendung zuzugreifen (*Delegated Authorization*).

Das Protokoll setzt auf HTTP auf.

Begriffe in OAuth 2.0

Resource Owner – das sind wir als Eigentümer unserer ID, Daten und Operationen, die mit unseren Accounts durchführbar sind.

Client – eine Anwendung, die für den Resource Owner auf Daten oder Funktionen zugreifen möchte.

Authorization Server – die Anwendung, die den Resource Owner bereits kennt bzw. wo dieser ein Account hat.

Resource Server – API bzw. Dienst, den der Client an Stelle des Resource Owner nutzen möchte.

Redirect URI – „Callback URL“, zu der der Authorization Server verweist, nachdem der Owner dem Client eine Erlaubnis erteilt hat.

Response Type – die Art der Information, die der Client erwartet.

Scope – granular vergebene Zugriffsrechte, die der Client benötigt.

Consent – der Authorization Server nimmt den Scope, den der Client anfordert und lässt sich diese Erlaubnisse vom Resource Owner bestätigen.

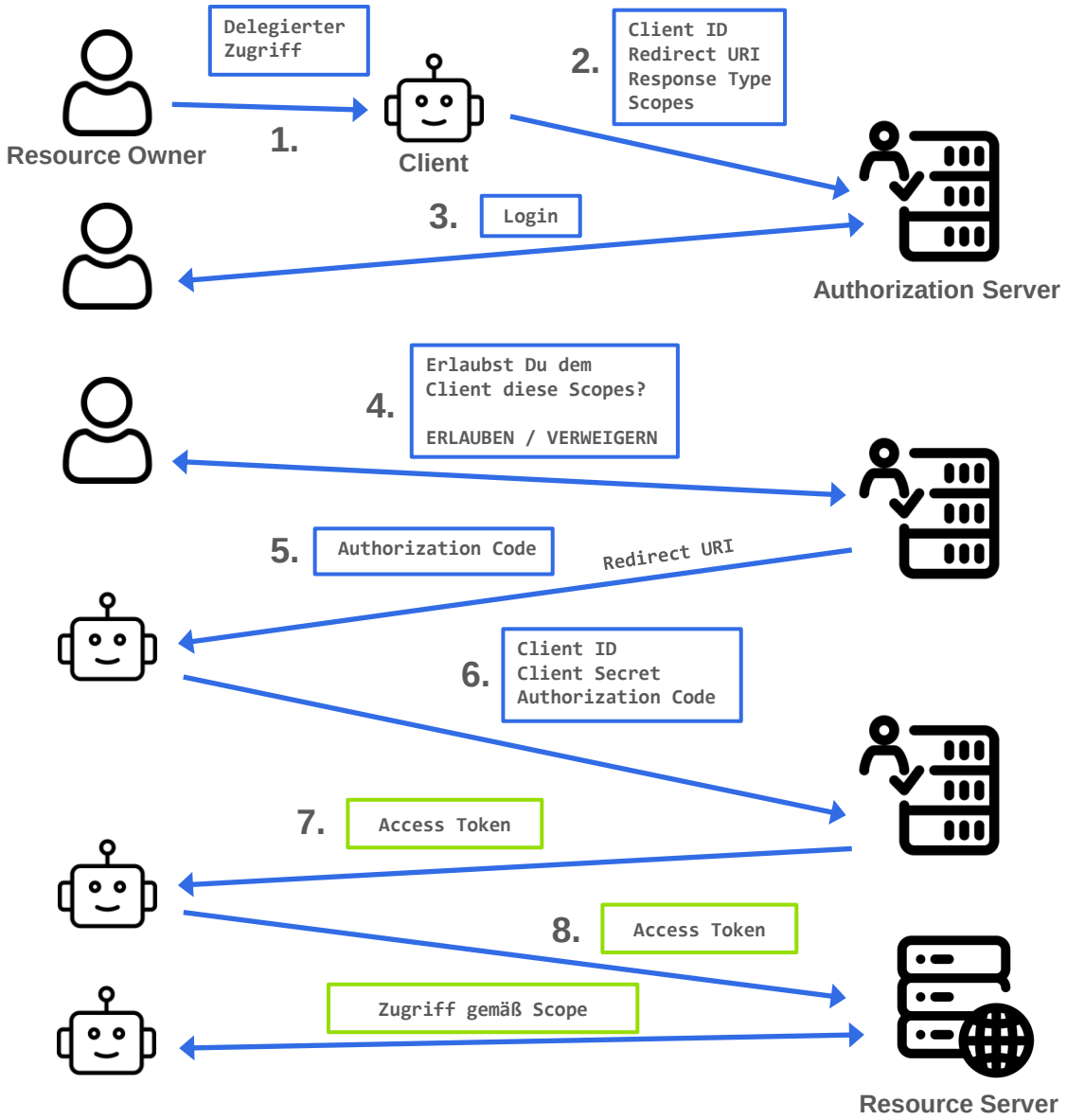
Client ID – zur Identifikation des Clients beim Authorization Server.

Client Secret – zwischen Client und Authorization Server zur geheimen Abstimmung.

Authorization Code – kurzlebiger temporärer Code vom Client an den Authorization Server, der dazu ein Access Token zurückgibt.

Access Token – Schlüssel des Clients zur Interaktion mit dem Resource Server. Bescheinigt die vergebenen Stellvertreterrechte des Clients.

OAuth Flow: Allgemeiner Ablauf



1. Wir als Resource Owner möchten, dass der Client für uns auf Dienste des Resource Servers zugreifen kann.
2. Der Client leitet unsere Anfrage weiter an den Authorization Server, ergänzt um Client ID, Redirect URI, Response Type und benötigte Scopes, um unsere Aufgaben erledigen zu können.
3. Der Authorization Server verifiziert, wer wir sind. Wenn nötig, melden wir uns dort an.
4. Der Authorization Server legt uns die vom Client gewünschten Scopes zur Genehmigung vor (wir erlauben sie).
5. Der Authorization Server leitet zurück zum Client über die Redirect URI, ergänzt um einen Authorization Code.
6. Der Client schickt direkt und abgesichert seine ID, Secret und den Authorization Code an den Server.
7. Nach Verifikation antwortet der Authorization Server mit einem Access Token.
8. Mit dem Access Token kann der Client nun auf den Resource Server zugreifen. Es gibt ihm genau die von uns im Scope genehmigten Rechte.

Übung 13: OAuth Flow für „Sign in with Apple/Google/Whatever“

1. Ordnen Sie die eben eingeführten Komponenten einem Szenario zu, in dem Sie sich mit einer Ihrer schon vorhandenen Identitäten bei Ihrem Streaming-Dienst anmelden möchten.
2. Wie sieht der Ablauf dann aus, wer macht was?

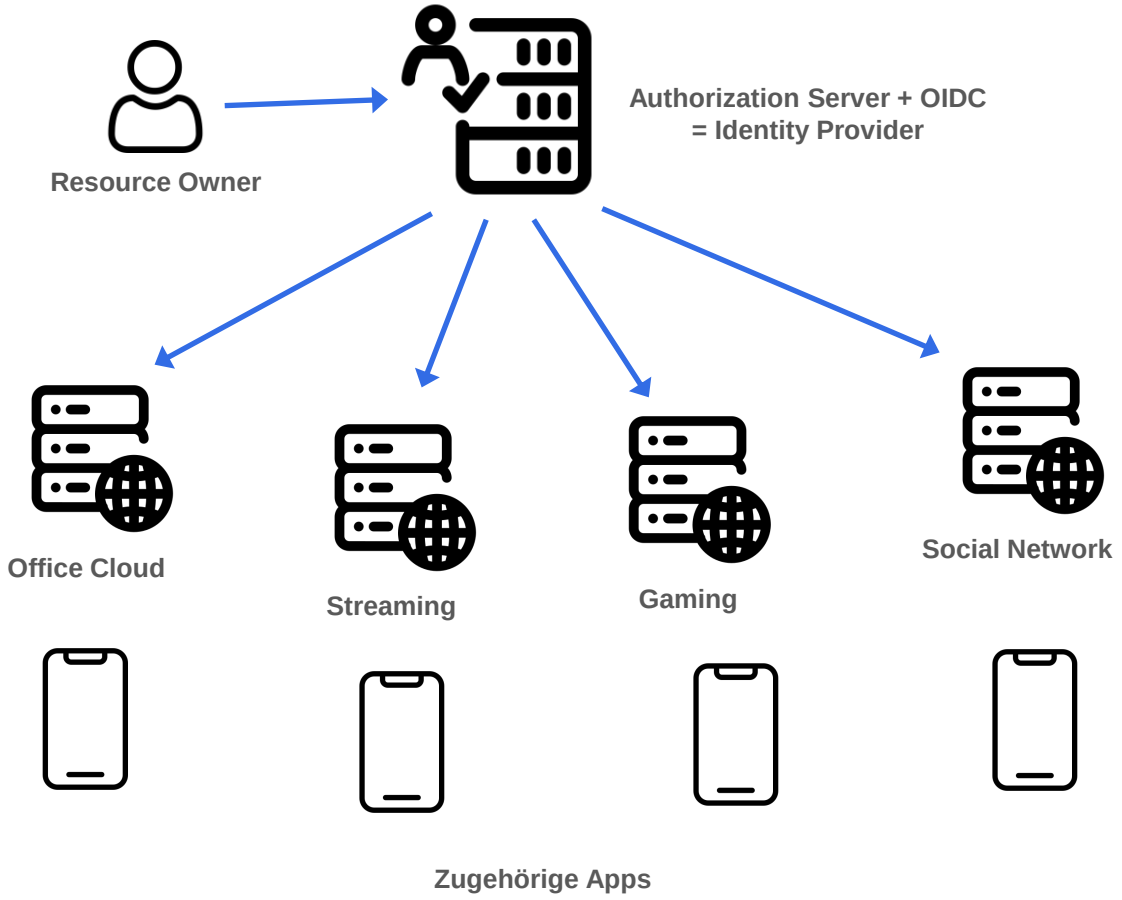
OpenID Connect: Ergänzung von Authentifikation

Genau genommen haben wir uns mit OAuth nur um die *Autorisierung* von Zugriffen gekümmert.

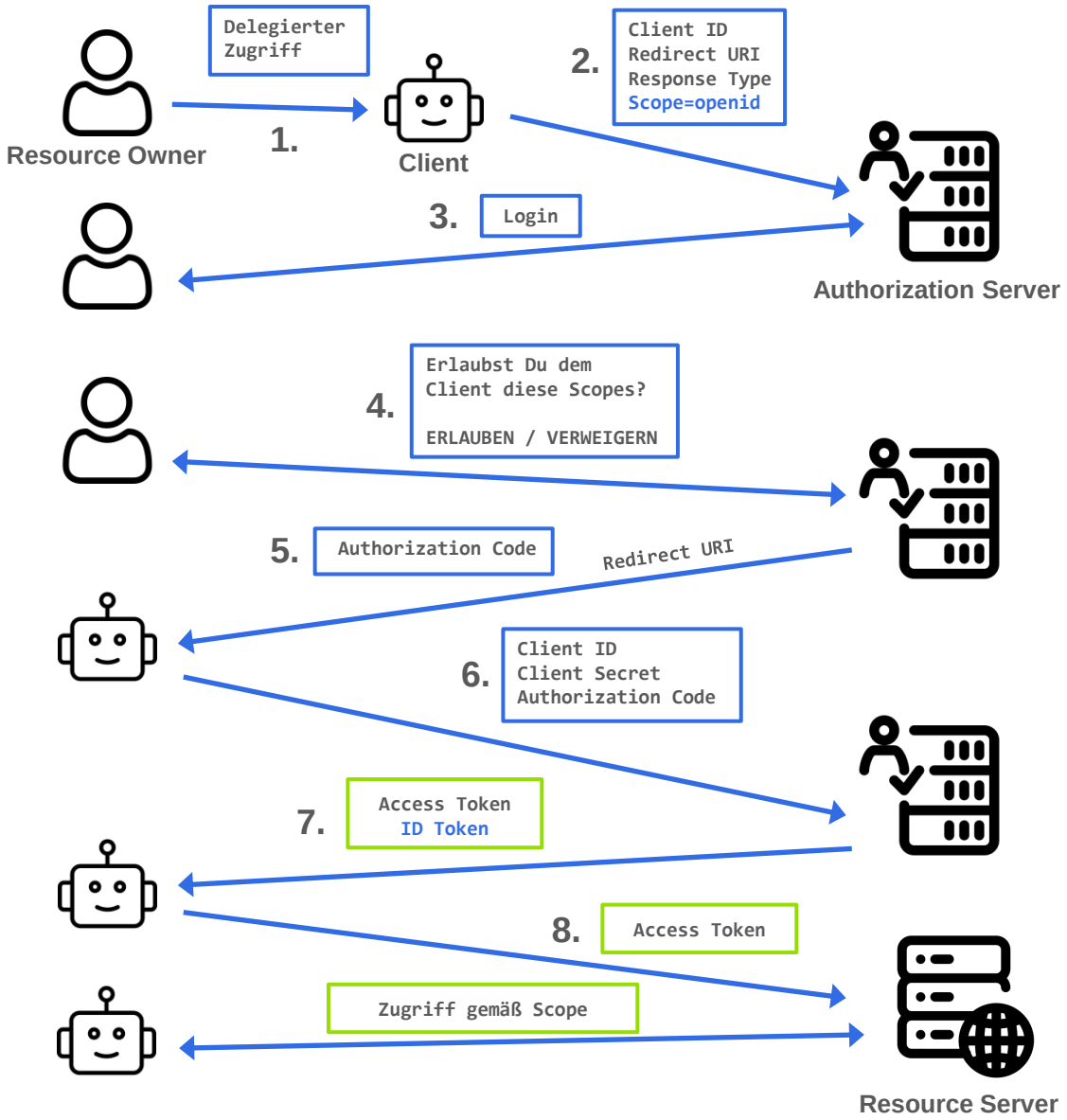
OpenID Connect (OIDC) legt eine weitere, aber dünne Schicht zur Identifikation und Authentifikation auf OAuth.

Wenn ein Authorization Server OIDC unterstützt, wird er zu einem **Identity Provider**, da er Informationen zum Resource Owner geben kann.

Damit können wir ihn für **Single Sign-on (SSO)** nutzen: Wir melden uns mittels OIDC beim Server an, der vergibt die passenden Autorisierungen an verschiedene Clients über OAuth.



OpenID Connect Flow: Allgemeiner Ablauf



Der Nachrichtenaustausch sieht bei OIDC genau so aus, wie bei OAuth. Unterschiede liegen im Detail:

1. Im initialen Request an den Authorization Server gibt es einen spezifischen Scope `openid`.
2. Am Ende erhält der Client sowohl das Access Token, als auch ein **ID Token**. Es enthält *Claims* über den Resource Owner.

Das Access Token ist für den Client selbst nicht weiter auswertbar, er gibt es einfach an den Resource Server weiter. Er kann damit aber auch vom Authorization Server Identitätsinformationen über den Resource Owner abfragen, z. B. Email-Adresse.

Das ID Token ist ein **JSON Web Token (JWT)**. Der Client kann daraus Informationen über den Resource Owner ablesen, z. B. ID, Name, wann wir uns angemeldet haben, Gültigkeit.

Übung 14: OIDC/OAuth für unsere Meta Playlist

Erstellen Sie ein Konzept (keine Implementierung), um OIDC/OAuth in unsere Web-Architektur der Meta Playlists zu integrieren.

1. Welche unterschiedlichen Ansätze und Möglichkeiten gibt es hier?
2. Wie sieht dann jeweils die Zuordnung der Komponenten aus?

In den Unterlagen genutzte Icons

```
<a href="https://www.flaticon.com/free-icons/iphone" title="iphone icons">Iphone icons created by Good Ware - Flaticon</a>
<a href="https://www.flaticon.com/free-icons/webserver" title="webserver icons">Webserver icons created by Danteee82 - Flaticon</a>
<a href="https://www.flaticon.com/free-icons/webpage" title="webpage icons">Webpage icons created by Design View - Flaticon</a>
<a href="https://www.flaticon.com/free-icons/robot" title="robot icons">Robot icons created by Hilmy Abiyyu A. - Flaticon</a>
<a href="https://www.flaticon.com/free-icons/user" title="user icons">User icons created by Freepik - Flaticon</a>
<a href="https://www.flaticon.com/free-icons/validation" title="validation icons">Validation icons created by orvipixel - Flaticon</a>
```